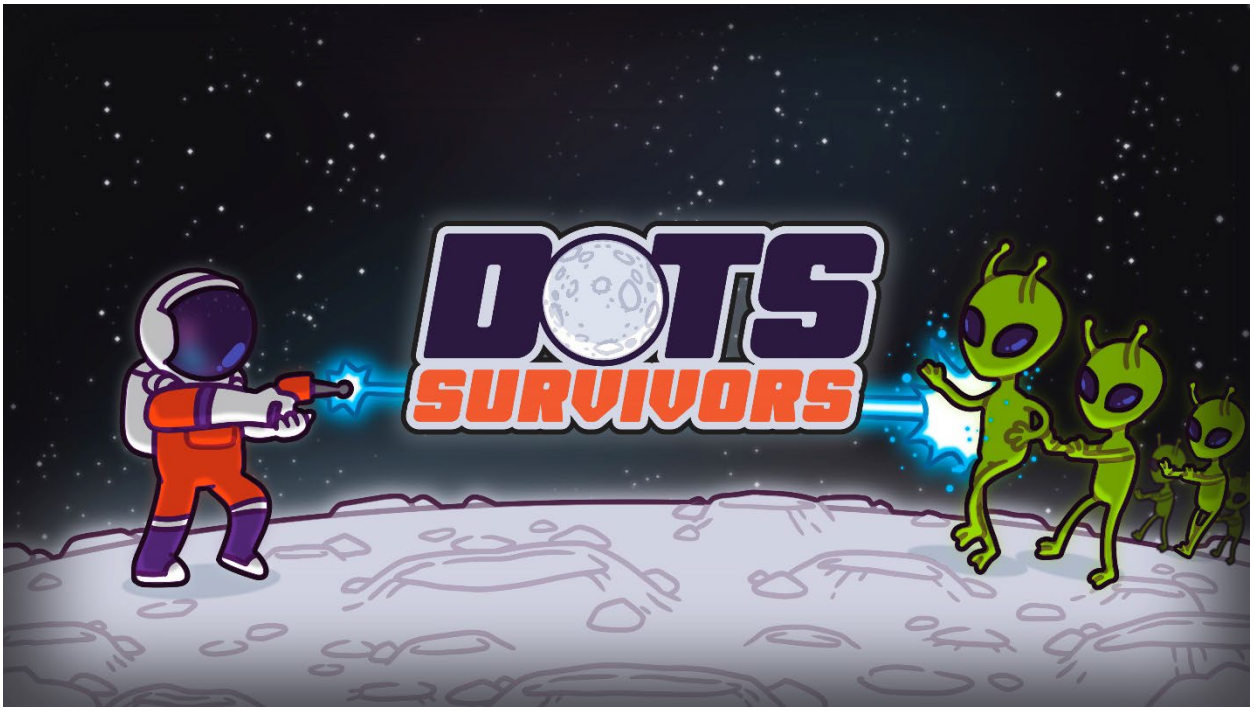

DOTS Survivors API Reference

By: Johnny Thompson – Turbo Makes Games – 2025

Online Documentation: https://survivors.tmg.dev/

Support Discord: https://tmg.dev/support-discord

Support Email: support@tmg.dev

Namespace TMG.DOTSSurvivors

Classes

ArtTestSceneAnimationAuthoring
: Authoring script to set components to the graphics entity of animated characters in the art test scene.

ArtTestSceneAuthoring
: Authoring script to set values on the ArtTestSceneData used for initializing the secret art test scene.

ArtTestScenePauseMenuUIController
: UI controller for the super secret art test scene.

AttractableItemAuthoring
: Authoring script to add components required for item attraction.

BeamInEffectController
: MonoBehaviour to control the beam in effect that is played when the player initially spawns into a level.

BeginGameOverSystem
: System to invoke the begin game over sequence.

BlackHoleEntityAuthoring
: Authoring script to add components necessary for black hole in-world attack entity to function.

BlackHoleWeaponAuthoring
: Authoring script to add BlackHoleWeaponTag to the entity.

BonusItemProperties
: ScriptableObject to define properties related to bonus items. Bonus items are items shown in the level up and crate UI if no further upgrades to attacks or passive items are available to the player.

BoomerangMovementAuthoring
: Authoring script to add the BoomerangMovementData to an entity.

BounceOnEnvironmentAuthoring
: Authoring script to add the BounceOnEnvironmentTag to an entity.

CO2CloudEntityAuthoring
: Authoring script to add components necessary for CO2 cloud in-world attack entity to function.

CO2CloudWeaponAuthoring
: Authoring script to add CO2CloudWeaponTag to the entity.

CameraAspectRatioFixer

MonoBehaviour script to add letterboxing to screens outside supported aspect ratios.

CameraTargetAuthoring

Authoring script to initialize the InitializeCameraTarget component on the player entity.

CameraTargetObject

Singleton MonoBehaviour to identify this GameObject as the target to use for the main Cinemachine camera.

CapabilityResultController

UI controller associated with a single capability element on the end results panel.

CapabilityStatusUIController

UI controller associated with an individual capability to be displayed in the game pause menu.

CapabilityUpgradeController

Managed controller to handle upgrading player capabilities from the GameObject side. This script is required to serve as a bridge between GameObject and ECS sides of the game. UI elements will call methods in this script to configure things on both GameObject and ECS sides related to upgrading capabilities (weapons and passive items).

CapabilityUpgradeUIController

UI controller associated with a singular capability level up UI element that is shown in the level up UI.

CharacterAnimationAuthoring

Authoring script to add components required for character animation.

CharacterAuthoring

Authoring script to add necessary components to character entities.

CharacterProperties

ScriptableObject to define properties for player characters. Certain fields are used just for UI display, while others affect gameplay.

CharacterSelectButtonUIController

UI controller associated with a single UI element for a character that can be selected in the main menu UI.

CharacterSelectPanelUIController

UI controller associated with the panel to select which character to play as in the main title screen menu.

CompanionGameObjectAuthoring

Authoring script to set values for CompanionGameObjectInitializationData which in turn will initialize values of CompanionGameObjectData in the UpdateCompanionGameObjectSystem.

ConstantRotationAuthoring
Authoring script to add the ConstantRotationAuthoring component to entities.

CounterRotationAuthoring
Authoring script to initialize CounterRotationData on the associated entity.

CrateItemAuthoring
Authoring script for crate entity.

CrateUIController
UI controller associated the UI panel that is displayed when the player picks up a loot crate dropped by a boss enemy.

CurrentCapabilitiesUIController
UI Controller associated with the capability status display that is shown in the game pause menu.

CurrentStatsUIController
UI controller associated with the current stats panel that is displayed in the game pause menu.

DS_AttackSystemGroup
System group containing systems related to player attacks.

DS_DestructionSystemGroup
System group containing systems related to entity destruction.

DS_EffectsSystemGroup
System group containing systems that deal with triggering visual and audio effects.

DS_InitializationSystemGroup
System group containing systems related to initialization logic that should be executed towards the beginning of the frame.

DS_InteractionSystemGroup
System group containing systems related to entity interactions.

DS_PhysicsSystemGroup
System group containing systems that deal with physics such as scheduling collision/trigger event jobs and executing physics overlap queries.

DS_TranslationSystemGroup
System group containing systems that deal with moving entities via their LocalTransform component.

DamageNumberUIController

UI controller associated with the damage icons that display above enemy characters when taking damage.

DamageableEntityAuthoring

Authoring script to add components required for entities taking damage.

DealHitPointsOnInteractionAuthoring

Authoring script to add DealHitPointsOnInteraction component to an entity.

DecorationAuthoring

Authoring script to apply the DecorationTag to an entity.

DestroyAfterNumberHitsAuthoring

Authoring script to initialize the DestroyAfterNumberHits component.

DestroyAfterTimeAuthoring

Authoring script to set the initial value for the DestroyAfterTime component.

DestroyAtPositionAuthoring

Authoring component for DestroyAtPosition

DestroyOffCameraAuthoring

Authoring script to initialize values for DestroyOffCameraData

DestroyOnEnvironmentContactAuthoring

Authoring script that adds DestroyOnEnvironmentContactTag component to the entity.

DestroySelfOnInteractionAuthoring

Authoring script to add the DestroySelfOnInteractionTag component to an entity.

DestructibleEntityAuthoring

Authoring component to add DestroyEntityFlag to an entity. This component will be disabled by default to ensure the entity isn't immediately destroyed after instantiation.

DroneEntityAuthoring

Authoring script to add components necessary for drone in-world attack entity to function.

DroneWeaponAuthoring

Authoring script to add DroneWeaponTag to the entity.

DropExperienceOnDestroyAuthoring

Authoring script to add the DropExperienceOnDestroy to an entity.

EnemyAnimationAuthoring

Authoring script to add components necessary for enemy character animation.

EnemyAuthoring
Authoring script to set initial values for enemies.

EnemyBlackHoleResistAuthoring
Authoring component to add EnemyBlackHoleResistTag to an enemy.

EnemyLinearMovementAuthoring
Authoring script to initialize the angle of travel for EnemyLinearMovement. Sets SetEnemyLinearMovementFlag to true to initialize CharacterMoveDirection.

EnemyMoveToPlayerAuthoring
Authoring script to add EnemyMoveToPlayerTag to entity.

EnemySineWaveMovementAuthoring
Authoring script to initialize values for EnemySineWaveMovement.

EnemySpawnAuthoring
Authoring script to define behavior of enemy spawning for a particular level.

EnemySpawnEventProperties
Authoring for spawn events. Spawn event are certain events that have a chance of occurrence within a spawn wave. When a spawn event happens, enemies will be spawned in a certain SpawnFormation and have unique behavior.

EnemySpawnWaveProperties
ScriptableObject used to author information about spawn waves.

EntityInteractionAuthoring
Authoring component to add the EntityInteraction buffer to an entity.

EntityRandomAuthoring
Authoring script to add EntityRandom component to an entity. This can also add the InitializeEntityRandom component to trigger initialization of random seed that cannot be set during baking i.e. using milliseconds of the system when the entity spawns.

EnvironmentAuthoring
Authoring script to apply the EnvironmentTag to an entity.

ExperienceGemItemAuthoring
Authoring script to set initial experience point value for an experience point gem.

ExplodeOnTimerAuthoring
Authoring script to add components necessary for explosion to an entity.

FadeAttackInOutAuthoring
: Authoring script to add components required for fading the attack visual in and out.

FadeCompanionTrailRendererTagAuthoring
: Authoring script to add the InitializeFadeCompanionTrailRenderer component to the entity.

FlashColorOnDamageAuthoring
: Authoring script to initialize values for the FlashColorOnDamageData component and add other necessary components.

GameAudioController
: MonoBehaviour for controlling audio in game.

GameAuthoring
: Authoring script to add components necessary for controlling the ECS side of the game.

GameOverUIController
: UI controller associated with the game over screen that is displayed after the player character is destroyed.

GetPlayerInputSystem
: System for reading input from the input system package and assigning it to the player's Character MoveDirection.

GlobalTimeUpdateSystem
: System to update the global time shader property. This is required so that when the game pauses, any animated shaders that rely on this property will also pause as this system is part of a group that will be disabled when the game is paused due to level up or pressing the ESC key (PauseManager).

GrantMoneyOnInteractionAuthoring
: Authoring script to initialize the value of a coin item.

GrantMoneyOnInteractionSystem
: System to grant money to the player when the coin item is collected.

GraphicsEntityAuthoring
: Authoring script to add GraphicsEntity component to an entity.

HUDUIController
: UI controller associated with the in-game HUD UI.

HandleCrateItemInteractionSystem
: System containing logic for when the player collects a crate.

HandlePlayerExperienceThisFrameSystem

System to add player experience from the player's PlayerExperienceThisFrame dynamic buffer to the player's TotalPoints. This system also determines if the player should level up. If the player should level up, then this system will also determine a weapon or passive ability to upgrade.

HealOnInteractionAuthoring
Authoring script to initialize value of health item.

InstantDestroyOnInteractionAuthoring
Authoring script to add InstantDestroyOnInteractionTag component to entity.

ItemAuthoring
Authoring component to add ItemTag to the baked entity.

JetpackEntityAuthoring
Authoring script to add components necessary for jetpack in-world attack entity to function.

JetpackTrailRendererController
MonoBehaviour component attached to companion trail renderers of jetpack attack entities. Provides some nice visual effects to fade the trail renderer in and out.

JetpackWeaponAuthoring
Authoring script to add JetpackWeaponData to the entity.

KnockbackAuthoring
Authoring script to add components required for knockback

KnockbackOnInteractionAuthoring
Authoring script to add KnockbackOnInteractionData to an entity.

LaserStrikeEntityAuthoring
Authoring script to add components necessary for laser strike in-world attack entity to function.

LaserStrikeWeaponAuthoring
Authoring script to add components necessary for executing laser strike attacks.

LevelGenerationAuthoring
Authoring script to add components necessary for level generation.

LevelUpUIController
UI controller associated with the level up UI that is displayed when the player levels up.

LinearMovementAuthoring
Authoring script to add LinearMovementSpeed component to an entity, so it will constantly move along its forward axis in the LinearMovementSystem

LinearScaleTransformationAuthoring

Authoring script to add LinearScaleTransformationData to an entity.

LoadingScreenUIController

UI controller for the loading screen that is displayed when switching between title scene and gameplay scenes.

MainMenuAudioController

MonoBehaviour for controlling audio in main menu.

ManagedGameData

Managed data component to store managed types required for gameplay.

OxygenHoseEntityAuthoring

Authoring script to add components necessary for oxygen hose in-world attack entity to function.

OxygenHoseWeaponAuthoring

Authoring script to add OxygenHoseWeaponData to the entity.

ParabolicMovementAuthoring

Authoring script to define initial values for ParabolicMovementState.

PassiveEntityAuthoring

Authoring script to add required components to passive entity.

PassiveUpgradeProperties

ScriptableObject to define the properties for a passive item and its upgrade path.

PauseManager

Singleton MonoBehaviour to control the pause state of the game.

PauseMenuUIController

UI controller associated with the pause menu that is displayed when the player pauses the game.

PersistentDataManager

Singleton MonoBehaviour to handle saving, loading, and processing, persistent data relevant to gameplay.

PhysicsHelper

Static helper class used to implement helpers not present in Unity's API.

PlasmaBlastEntityAuthoring

Authoring script to add components necessary for plasma blast in-world attack entity to function.

PlasmaBlastWeaponAuthoring

Authoring script to add PlasmaBlastWeaponTag to the entity.

PlayAudioClipOnDamageAuthoring
Authoring script to add components to entity required for playing audio clips when taking damage.

PlayAudioClipOnDestroyAuthoring
Authoring script to initialize PlayAudioClipOnDestroyData.

PlayAudioClipOnSpawnAuthoring
Authoring script to add necessary components for playing audio clips when the entity is spawned.

PlayParticleSystemOnDamageAuthoring
Authoring script to add components necessary for playing a particle system when an entity takes damage.

PlayerAnimationAuthoring
Authoring script to initialize and add components required for player character animation.

PlayerAttractionAreaAuthoring
Authoring script to initialize data for the player's attraction area.

PlayerAuthoring
Authoring script to add components specific to the player entity.

PlayerInitializationSystem
System to initialize values pertaining to player entities.

PlayerStatStatusUIController
UI controller associated with individual stat elements that display the player's current modifiable stats.

PlayerUpgradeData
Managed data component to store data related to player upgrades.

RadioactiveWasteDropEntityAuthoring
Authoring script to add components necessary for radioactive waste drop in-world attack entity to function.

RadioactiveWasteSpillEntityAuthoring
Authoring script to add components necessary for oxygen hose in-world attack entity to function.

RadioactiveWasteWeaponAuthoring
Authoring script to add RadioactiveWasteWeaponData to the entity.

RandomItemDropProperties
ScriptableObject to define which items can be dropped when an entity destroyed.

ResultsPanelUIController

UI controller associated with the results panel that is displayed after the game over screen.

SatelliteEntityAuthoring

Authoring script to add components necessary for satellite in-world attack entity to function.

SatelliteWeaponAuthoring

Authoring script to add SatelliteWeaponData to the entity.

SawBladeEntityAuthoring

Authoring script to add components necessary for saw blade in-world attack entity to function.

SawBladeWeaponAuthoring

Authoring script to add SawBladeWeaponData to the entity.

ScaleOscillationAuthoring

Authoring script to initialize values on the ScaleOscillationData of an entity.

ScreenEdgeBounceAuthoring

Authoring script to add ScreenEdgeBounceTag to entity.

ScrewdriverEntityAuthoring

Authoring script to add components necessary for screwdriver in-world attack entity to function.

ScrewdriverWeaponAuthoring

Authoring script to add ScrewdriverWeaponTag to the entity.

SelectionIconUIController

UI controller associated with the selection icons that display on either side of the current highlighted element.

ShowDamageNumberOnDamageAuthoring

Authoring script to initialize components necessary for displaying damage numbers above the entity.

ShowDamageNumberOnHealAuthoring

Authoring script to initialize components necessary for displaying "damage" numbers above the entity when healing.

SpaceAgencyFlagController

Singleton MonoBehaviour to control setting of the space agency flag in each level with the flag of the nation associated with the current character.

SpaceAgencyProperties

ScriptableObject to define properties related to an individual space agency.

SpawnOnDestroyAuthoring

Authoring script to initialize the entity prefab stored in the SpawnOnDestroy component.

SpawnRandomOnDestroyAuthoring

Authoring component to associate a RandomItemDropProperties ScriptableObject with an entity. This authoring component will also convert GameObject prefabs set in the RandomItemDropProperties and store them in a temporary SpawnRandomPrefabHelper DynamicBuffer.

StageProperties

ScriptableObject to store properties related to stages.

StageSelectButtonUIController

UI controller associated with the individual UI elements to display a stage available to be chosen.

StageSelectPanelUIController

UI controller associated with the stage select panel displayed in the main menu of the title scene.

StatModifierController

Stat modifier controller singleton. Objects with this component will be marked as Don't Destroy on Load as all scenes will need access to StatModifierProperties ScriptableObjects loaded from this controller.

StatModifierHelper

Helper class to add extension methods to StatModifierType and StatModifierCalculationType enums.

StatModifierProperties

ScriptableObject to author properties related to stat modifiers. Certain properties are used for runtime data and others are used for displaying information in UI elements.

TemporaryStatModifierAuthoring

Authoring script to initialize data components on temporary stat modifier entities.

TitleScreenUIController

UI controller associated with the main menu in the title screen.

UpdateGameTimeSystem

Updates the in-game time during gameplay.

UpdateUISystem

System to update UI elements on screen.

UpgradeProperties

Abstract class of a ScriptableObject to define the upgradable properties for weapons and passive items that can be upgraded during gameplay.

WeaponAuthoring
Base authoring script to add components all weapon entities need.

WeaponUpgradeProperties
ScriptableObject to define the properties for a weapon and its upgrade path.

WorldUICanvasController
UI controller associated with the world space UI canvas used to display damage numbers above enemies taking damage.

WrenchEntityAuthoring
Authoring script to add components necessary for wrench in-world attack entity to function.

WrenchWeaponAuthoring
Authoring script to add WrenchWeaponData to the entity.

Structs

ActiveStatModifierEntity
Dynamic buffer to hold the currently active stat modifier entities.

AliensDefeatedCount
Data component to store the running count of aliens defeated in a run.

AlreadyStruckEntity
Dynamic buffer to store entities that have already been struck by the current group of laser strikes.

AnimationIndexOverride
Material property override for the animation index. Setting this value will set the animation index on the material instance for this entity.

AnimationOffsetOverride
Material property override for the animation offset of the enemy. Setting this value will update the _SpawnTime property in the material instance of this entity. This provides an offset for the animation so all enemies can be on their own unique animation frame and will not appear to be synchronized.

ArtTestSceneData
Data component to store data pertinent to the secret art test scene.

AttackPrefab
Prefab of an in-world attack entity that will be spawned by an attack system.

AttractableItemSystem
System that moves attractable items from their start position to the player.

AttractableItemTag

Tag component to identify this entity as an attractable item.

BaseHitPoints

Data component storing the base hit points of a damageable entity.

BeginGameOverTag

Tag component to signify the BeginGameOverSystem should begin execution.

BlackHoleAttackSystem

System to handle instantiating AttackPrefabs of black holes into the game world.

BlackHoleData

Data component to store persistent data for black hole attacks.

BlackHoleState

Data component to store information about the current state of the in-world black hole entity.

BlackHoleSystem

System that controls the behavior of the black hole in-world attack entity.

BlackHoleWeaponTag

Tag component to signify this entity as the weapon entity to spawn black holes into the game world.

BoomerangMovementData

Data related to boomerang movement.

BoomerangMovementSystem

System to implement the boomerang movement.

BounceOnEnvironmentJob

Trigger events job to detect trigger events between entities tagged with BounceOnEnvironmentTag and EnvironmentTag and will raise an EntityInteraction on the entity tagged with BounceOnEnvironmentTag. Scheduled by the BounceOnEnvironmentJob.

BounceOnEnvironmentSystem

System to schedule the BounceOnEnvironmentJob to detect trigger events between entities tagged with BounceOnEnvironmentTag and EnvironmentTag and will raise an EntityInteraction.

BounceOnEnvironmentTag

Tag component to signify this entity should bounce off environment entities with a collider.

CO2CloudAttackSystem

System to handle instantiating AttackPrefabs of CO2 clouds into the game world.

CO2CloudWeaponTag

Tag component to signify this entity as the weapon entity to spawn CO2 clouds into the game world.

CameraTarget

Component attached to the player entity. Used for systems that need information about the camera.

CapabilityUpgradeLevel

Helper struct to pair an UpgradeProperties ScriptableObject with the index of its next level.

CharacterAnimationSystem

System to handle changes to character materials to facilitate animation visual effects. This system has 4 foreach loops that do the following:

1. Initializes AnimationOffsetOverride for enemy entities so all their animations are not synchronized.
2. Initializes the EnhanceThicknessOverride to apply a visual enhancement outline on the outside of an enemy to denote it as an enhanced entity.
3. Set which animation is currently playing for the player entity. Used to switch between walk and idle animations depending on current input (CharacterMoveDirection) and previous input (PreviousPlayerInput).
4. Plays the dissolve effect when the character is destroyed. (see DissolveData and DissolveOverride)

CharacterBaseMoveSpeed

Component to hold the character's base movement speed in units per second.

CharacterDefaultModificationValues

Component to hold the default modification values for each StatModifierType.

CharacterEntity

Component for various entities to reference their associated character entity.

CharacterHealthRegenerationState

Component to hold the character's hit point accumulator. When component is enabled, health regeneration will take place in the CharacterHealthRegenerationSystem.

CharacterHealthRegenerationSystem

System to handle regenerating character health.

CharacterInitializationSystem

System to initialize values pertaining to game characters.

CharacterMoveDirection

Component defining the current move direction of a character.

CharacterMoveSystem

System to move characters in the game world based off input from their CharacterMoveDirection. Uses PhysicsVelocity to apply movement so collisions with other entities are properly handled.

CharacterStatModificationState

Data component to store the current stat modification values for a character. Several systems reference this component to calculate effective stat data (i.e. take the base value of a stat, and add or multiply the relevant stat modification value as required).

CoinsCollected

Data component to store the running count of coins collected in a run.

CompanionGameObjectData

Data related to the companion GameObject that will be matching the position of the associated entity.

CompanionGameObjectInitializationData

Initialization data for CompanionGameObjectData.

ConstantRotationData

Data component to store data related to constant rotation of an entity.

ConstantRotationSystem

System to apply a constant rotation to entities.

CounterRotationData

Data component stores data related to counter rotation. Counter rotation is when the rotation of a child entity applies a rotation opposite to that of the parent entity, giving it the visual appearance of having a fixed orientation.

CounterRotationSystem

System to apply counter rotation to an entity. Counter rotation is when the rotation of a child entity applies a rotation opposite to that of the parent entity, giving it the visual appearance of having a fixed orientation.

CrateItemTag

Tag component to identify crate entities.

CurrentHitPoints

Data component storing the current hit points of a damageable entity.

DamagePlayerOutsideSafeBoundsSystem

System to apply damage to the player when it is outside the LevelSafeBounds for the level.

DamagePlayerOutsideSafeBoundsTimestamp

Component to hold the timestamp for when the player can be damaged again by the DamagePlayerOutsideSafeBoundsSystem.

DamageThisFrame

Dynamic buffer to store damage accumulated in the current frame.

DealHitPointsOnInteraction

Data component to define the number of hit points to deal when the associated entity interacts with another.

DealHitPointsOnInteractionSystem

System to add damage hit points to the target entity's DamageThisFrame buffer.

DecorationTag

Tag component to signify this entity is a decoration entity.

DestroyAfterNumberHits

Entities with this component will be destroyed when HitsRemaining reaches 0.

DestroyAfterNumberHitsSystem

System that determines when the entity has hit the max number of entities. Each hit will decrement HitsRemaining until it reaches zero. Once the counter reaches zero, the entity's DestroyEntityFlag will be enabled.

DestroyAfterTime

Entities with this component will be destroyed once this time value expires.

DestroyAfterTimeSystem

System to destroy entities after a given time. This system ticks DestroyAfterTime Value down and will set the DestroyEntityFlag to true once the timer value has reached zero. This will count only in-game time and not be affected by game pause.

DestroyAtPosition

Entities with this component will get destroyed once they reach this world position.

DestroyAtPositionSystem

This system will set the DestroyEntityFlag to true once the associated entity reaches a target world position. It determines if an entity has reached the position by comparing the current distance to the target with the previous distance. If the current distance is greater than the previous distance, it knows the entity has passed the target position. The distance should be less than two units to reduce the possibility of false positives. This method of determining if an entity has reached a target position is to ensure fast moving entities that aren't guaranteed to be within a small tolerance of the target position are accounted for.

DestroyEntityFlag

An entity with this component is capable of being destroyed by various systems throughout the game. This component should be disabled by default and once enabled it will be destroyed at the end of the frame in the DestroyEntitySystem

DestroyEntitySystem

This system is responsible for destroying all entities with the DestroyEntityFlag enabled. There are various checks to determine if any additional logic should be executed when the entity is destroyed. This system should be the last system to execute in the DS_DestructionSystemGroup, which itself updates at the end of the SimulationSystemGroup. The only system to run after this system in the SimulationSystemGroup should be the EndSimulationEntityCommandBufferSystem.

DestroyOffCameraData

Entities with this component will be destroyed when the entity has left the camera bounds on one (or more) of the specified sides.

DestroyOffCameraSystem

System that will enable the to destroy the entity at the end of the frame if the entity is offscreen. Direction can be used to specify the directions off camera the entity will be cleaned up.

DestroyOnEnvironmentContactSystem

System that schedules the DestroyOnEnvironmentContactTriggerJob to destroy an entity upon contact with an environment entity.

DestroyOnEnvironmentContactTag

Entities with this component will be destroyed upon contact with an environment entity.

DestroyOnEnvironmentContactTriggerJob

Trigger Event Job scheduled by the DestroyOnEnvironmentContactSystem. Enables the DestroyEntityFlag component on an entity tagged with the DestroyOnEnvironmentContactTag component that comes in contact with an entity with an EnvironmentTag component.

DestroyPreviousWaveEnemySystem

System to destroy off-screen enemies that were spawned in a previous wave.

DestroySelfOnInteractionSystem

System that enables the DestroyEntityFlag on an entity tagged with the DestroySelfOnInteractionTag when an interaction is raised.

DestroySelfOnInteractionTag

Entities with this component will self-destruct upon raising an interaction.

DetectCapabilityTriggerJob

Job to detect trigger events between capabilities and damageable entities.

DetectCapabilityTriggerSystem

System to schedule the DetectCapabilityTriggerJob.

DetectItemAttractionSystem

System to detect attractable items.

DissolveData

Component data relevant to dissolve visual effect for characters upon destruction

DissolveOverride

Material property override for the dissolve property of the character. Setting this value will update the _Dissolve property in the material instance of this entity. This will make the character sprite appear to dissolve upon destruction.

DroneAttackSystem

System to handle instantiating AttackPrefabs of drones into the game world.

DroneWeaponTag

Tag component to signify this entity as the weapon entity to spawn drones into the game world.

DropExperienceOnDestroy

Entities with this component have a chance to drop an experience point of a specified value when the entity is destroyed via the DestroyEntitySystem.

EnemyAttackCollisionJob

Collision Event Job scheduled by EnemyAttackSystem. This job is ran on all collision events raised by the physics system for this physics step. The Execute method will first determine which of the entities in the collision event is the player entity and which is the enemy entity. If the collision event is not between exactly 1 player and 1 enemy, the job will early out. Next the job will determine if the enemy is on cooldown, if the enemy is not on cooldown, it will apply damage to the player via the Damage ThisFrame Dynamic Buffer and reset the EnemyAttackCooldownExpiration for the enemy.

EnemyAttackCooldownExpiration

Component to store the timestamp of when the enemy attack cooldown expires. If the game time is greater than or equal to this value, the enemy is able to attack.

EnemyAttackData

Data component containing data related to enemy attacking.

EnemyAttackSystem

System that schedules EnemyAttackCollisionJob which detects collisions between enemies and the player and deals damage to the player. Updates in the DS_PhysicsSystemGroup so collision events for this frame have been raised.

EnemyBlackHoleResistTag

Enemies tagged with this component are resistant to black holes. Currently only used on ending reaper enemy.

EnemyHitPointsScaleWithPlayerLevelTag

Tag component to identify this entity as an enemy that should scale its health with the player level. Level scaling is defined by Enemy Base Hit Points multiplied by Player Level.

EnemyInitializationSystem

System to control initialization steps pertaining to enemy entities.

EnemyLinearMovement

Data component defining the angle in radians the player will move.

EnemyLinearMovementSystem

System to set CharacterMoveDirection based off angle stored in EnemyLinearMovement when Set EnemyLinearMovementFlag is enabled for an entity.

EnemyMoveToPlayerJob

Job to calculate the direction from the enemy to the player and set the CharacterMoveDirection to move the enemy towards the player.

EnemyMoveToPlayerSystem

System to schedule to EnemyMoveToPlayerJob which constantly updates the enemies' CharacterMove Direction to move the enemy towards the player.

EnemyMoveToPlayerTag

Enemies tagged with this component will constantly update their CharacterMoveDirection to move towards the player's current position.

EnemySineWaveMovement

Component defining the movement properties for enemy sine wave movement.

EnemySineWaveMovementSystem

System to set the CharacterMoveDirection for an enemy entity based on values calculated using EnemySineWaveMovement.

EnemySineWaveStartTime

Component containing the start time of the sine wave so the enemy can move along the wave over time.

EnemySpawnData

Data pertaining to enemy spawning. This data is specific to a level and will remain unchanged through gameplay of a specific level.

EnemySpawnSystem

System to spawn enemies as defined by spawn waves and spawn events.

EnemySpawnTimer

Timer to inform the EnemySpawnSystem when to spawn the next batch of enemies for a given wave.

EnemySpawnWave

Dynamic buffer component to hold all the EnemySpawnWaveProperties ScriptableObjects for a given stage.

EnemySpawnWaveData

Data pertaining to the currently active spawn wave. This data will be updated at the change of each spawn wave.

EnemySpawnerState

Holds the current state data of the enemy spawner.

EnemyTag

Empty Tag component used to identify Enemy entities.

EnemyTypeInitialization

Component containing the EnemyType value. This component is only used for initialization and will be removed from the enemy prefab before any enemies are instantiated. See GameStartSystem for usage.

EnemyWaveSystem

System to advance to the next spawn wave once the WaveTimer expires.

EnhanceThicknessOverride

Material property override for the spawn time of the enemy. Setting this value will update the _SpawnTime property in the material instance of this entity. This provides an offset for the animation so all enemies can be on their own unique animation frame and will not appear to be synchronized.

EntityInteraction

Dynamic buffer component to store entity interactions.

EntityRandom

Component to store Unity.Mathematics.Random type used for random number generation to be held by a specific entity.

EnvironmentTag

Tag component to signify this entity is an environment entity.

ExperienceGemItemData

Data related to experience gems that have a chance to be dropped when an enemy is destroyed.

ExperiencePointColor

Data struct used to assist with authoring experience point gem colors in the Unity editor.

ExperiencePointTable

Data component to store information related to the amount of experience points required for the player to level up to the next level.

ExplodeOnTimerData

Data component to store data relevant to entities that can explode.

ExplosionGraphicsData

Component to store data relevant to the flashing of a GraphicsEntity for exploding entities that have the ExplodeOnTimerData component.

ExplosionGraphicsInitializationData

Data used to initialize the ExplosionGraphicsData.

FacingDirectionOverride

Material property override for the facing direction of the character. Setting this value will update the _FacingDirection property in the material instance of this entity. This will flip the character animation so the character faces to the left or right depending on the direction of movement.

FadeAttackInOutSystem

System to process fading the attack visuals in and out.

FadeAttackInOutTotalTimeToLive

Component to hold the total time this entity will exist in the game world. Used to calculate the alpha value for fading the attack visual in and out.

FadeCompanionTrailRenderer

Cleanup component to call the EndTrailRenderer() method when the main jetpack entity is destroyed. This provides a nice visual fade out of the trail renderer on jetpacks.

FadeCompanionTrailRendererSystem

System to initialize the FadeCompanionTrailRenderer cleanup component. Also upon entity destruction, this system will call the EndTrailRenderer() method which provides a nice fade out effect of the trail renderer.

FixCharacterPositionSystem

This system fixes the character's position to 0 in the y-axis as this game takes place on the x and z 2D axes.

FlashColorOnDamageData

Component to hold data related to visual effect that flashes the graphics entity of a character when taking damage.

FlashColorOnDamageSystem

This system will flash the graphics entity black and then a color defined in Color when the parent character entity takes damage. This provides a nice visual feedback for the player when an entity takes damage.

FlashColorOnDamageTimer

Timer used to display the flash effect for a certain period of time. This timer will count down from FlashTime to zero.

FlashOnTimerSystem

System to handle initialization of ExplosionGraphicsData on the GraphicsEntity and flash the graphics entity during the flash phase.

GameEntityPrefabs

Data component to entity prefabs utilized throughout the game.

GameEntityTag

Tag component to signify this entity as the entity containing data related to gameplay.

GameStartSystem

System to initialize things for the start of the game.

GameStartSystem.Singleton

Singleton component associated with this system to store the game index.

GameTime

Data component to store the current game time in seconds.

GrantMoneyOnInteraction

Component defining the value of a coin item.

GraphicsEntity

Data component to store a reference to the graphics entity for this entity.

GraphicsEntityPlayDestroyEffectTag

Tag component to inform the DestroyEntitySystem this character has a destruction animation that should play when the character is destroyed.

HandleBounceOnEnvironmentSystem

System to handle EntityInteractions between environment entities (EnvironmentTag) and entities tagged with BounceOnEnvironmentTag)

HandleEntityInteractionSystem

System to mark the IsHandled field to true.

HandleExperienceGemInteractionSystem

System to add player experience points when the player collects an experience point gem.

HandleTemporaryStatModifierInteractionSystem

System to create temporary stat modifier entities that will last for a certain period of time as defined by Value. Created as an interaction system as the TemporaryStatModifierActiveTime is applied to items that are picked up by the player.

HealOnInteraction

Hit points to be granted if player collects a health item.

HealOnInteractionSystem

System to grant health to the player when a health item is collected.

InitializeAnimationOffsetOverrideFlag

Flag component to signify this entity needs its AnimationOffsetOverride component initialized.

InitializeArtTestSceneSystem

System to initialize data for the secret art test scene.

InitializeArtTestSceneTag

Tag component to determine if the secret art test scene should be initialized

InitializeCameraTarget

Component used to initialize the CameraTarget component attached to the player entity.

InitializeCameraTargetSystem

System to initialize data for the CameraTarget by locating the CameraTargetObject and using data from InitializeCameraTarget.

InitializeCharacterFlag

Flag component to signify this character entity needs to be initialized via the CharacterInitialization System.

InitializeEnhancementMaterialFlag

Flag component to signify the EnhanceThicknessOverride component should be enabled.

InitializeEntityRandom

Helper data component to signify this entity needs its EntityRandom component initialized in the InitializeEntityRandomSystem.

InitializeEntityRandomSystem

System to initialize EntityRandom based on the EntityRandomInitializationType stored in InitializeEntityRandom.

InitializeExperienceGemFlag

Flag to initialize the experience gem.

InitializeExperienceGemSystem

System to initialize the color of the experience point gem based on its value.

InitializeFadeAttackInOutFlag

Flag component to signify this entity needs its FadeAttackInOutTotalTimeToLive component initialized.

InitializeFadeCompanionTrailRenderer

Component to signify the associated entity needs its FadeCompanionTrailRenderer component initialized.

InitializeGameTag

Component to signify the game needs to be initialized.

InitializePlayAudioClipOnDestroyData

Data component to initialize PlayAudioClipOnDestroyData.

InitializeSpawnRandomOnDestroySystem

This system is responsible for creating a blob asset containing a blob array of RandomItemToSpawn to be stored in a RandomItemToSpawnBlob component.

InstantDestroyEntitySystem

System responsible for immediately destroying entities with the InstantDestroyTag. This system will execute at the end of the DS_DestructionSystemGroup to ensure all systems that may add this tag have executed but before the DestroyEntitySystem to avoid the possibility of executing any additional destruction logic in that system.

InstantDestroyOnInteractionSystem

System to add the InstantDestroyTag to an entity tagged with the InstantDestroyOnInteractionTag when an EntityInteraction is raised.

InstantDestroyOnInteractionTag

Entities with this component wll be instantly destroyed when an interaction is raised.

InstantDestroyTag

When this component is added to an entity, it will be destroyed at the end of the frame, ignoring any additional destruction logic that would normally be triggered in the DestroyEntitySystem

InvincibilityExpirationSystem

System to take the player off invincibility once the invincibility expiration timestamp is reached. As InvincibilityExpirationTimestamp is an enableable component, entities are only considered invincible when the component is enabled; foreach loop will also only iterate over entities with the component enabled.

InvincibilityExpirationTimestamp
Timestamp for when the associated entity is no longer invincible to incoming damage. This component will be enabled when the entity is invincible to incoming damage and disabled for normal behavior where the entity can receive damage.

IsAttractedToPlayerData
Enableable component to signify when the item is attracted to the player.

IsOpeningCrateFlag
Flag component to signify if the player is currently opening a loot crate.

ItemCollectionJob
Job that checks for collision events between the player and items in the game world. Valid collisions will add an interaction to the EntityInteraction dyanmic buffer so subsequent systems can apply additional logic pertaining to the specific item.

ItemCollectionSystem
System to schedule the ItemCollectionJob. Job is a collision event job so it is scheduled from the DS_PhysicsSystemGroup.

ItemMoveSpeed
Component that defines the move speed of attractable items from their starting position to the player.

ItemTag
Tag component to identify this as an item entity.

JetpackAttackSystem
System to handle instantiating AttackPrefabs of jetpacks into the game world.

JetpackWeaponData
Data component to store data relevant to jetpack weapon.

KnockbackOnInteractionData
Data component to store data related to knockback force to give.

KnockbackOnInteractionSystem
System to apply knockback force to an entity.

KnockbackState

Data component to hold current state to knockback.

KnockbackSystem

System to apply knockback force to an entity.

KnockbackTakenMultiplier

Multiplier to change the strength of knockback received.

LaserStrikeAttackSystem

System to handle instantiating AttackPrefabs of laser strikes into the game world.

LaserStrikeWeaponData

Data component containing data related to the laser strike weapon attack.

LevelElementData

Dynamic buffer data for level elements. Level elements are decoration and environment entities that will spawn in the level.

LevelElementInfo

Data struct used to author a collection of level elements in the Unity editor.

LevelGenerationData

Data related to level generation.

LevelGenerationSystem

System to randomly generate a level with decoration and environment entities.

LevelSafeBounds

Data component used to store the safe bounds of the play area.

LinearMovementSpeed

Data component to store speed of linear movement. Linear movement is when an entity will move along its forward axis constantly at the speed defined here.

LinearMovementSystem

System to move an entity along its forward axis, with data defined in LinearMovementSpeed.

LinearScaleTransformationData

Data component to store data related to linear scale transformation. Linear scale transformation is when the scale of an entity changes from the start scale to the end scale in the duration listed.

LinearScaleTransformationEndTimestamp

Data component to hold the end timestamp of when the linear scale transformation will end.

LinearScaleTransformationSystem

System to perform the linear transformation from the data defined in LinearScaleTransformationData.

LoopingAudioSourceIndex
Component to store the index of the looping audio source in TMG.DOTSSurvivors.GameAudio Controller._loopingAudioSources.

MoveCameraSystem
This system updates the position of the GameObject camera target to match that of the player's position plus an additional offset along the y-axis.

OnDamageParticleSystemPrefab
GameObject prefab for the particle system that should play when this entity takes damage.

OxygenHoseAttackSystem
System to handle instantiating AttackPrefabs of oxygen hoses into the game world.

OxygenHoseWeaponData
Data component to store data relevant to spawning oxygen hose attacks into the game world.

ParabolicMovementState
Data component to hold current state of parabolic movement.

ParabolicMovementSystem
System to apply parabolic movement to entities. Parabolic movement gives the effect that an entity will move towards the bottom of the screen as if screen-space gravity is being applied.

PassiveLevelIndex
Current level index of the passive ability. Level 1 (lowest level) is index 0.

PassiveLevelInfo
Data struct used to store StatModifier array and description of a passive item at a single level of the item's upgrade path.

PassiveUpgradePropertiesReference
Component to reference the associated PassiveUpgradeProperties via UnityObjectRef.

PersistentGameData
Persistent game data that will be saved to disk.

PlasmaBlastAttackSystem
System to handle instantiating AttackPrefabs of plasma blasts into the game world.

PlasmaBlastWeaponTag
Tag component to signify this entity as the weapon entity to spawn plasma blasts into the game world.

PlayAudioClipOnDamageData

Data component for defining the audio clip to play when the associated entity takes damage.

PlayAudioClipOnDamageSystem

System for playing audio clips when an entity takes damage.

PlayAudioClipOnDestroyData

Data component holding the audio clip that will be played when the associated entity is destroyed.

PlayAudioClipOnDestroySystem

System to handle playing audio clip when entity is destroyed.

PlayAudioClipOnSpawnData

Data component holding the audio clip that will be played when the associated entity is spawned.

PlayAudioClipOnSpawnSystem

System to play audio clips when an entity is spawned.

PlayParticleSystemOnDamage

Cleanup component to hold a reference to the particle system that will play when this entity takes damage.

PlayParticleSystemOnDamageFlag

Enableable component to signify this entity should begin playing the particle system as it is taking damage.

PlayParticleSystemOnDamageSystem

System to manage the lifecycle of particle systems that play when an entity takes damage. Has 3 foreach loops that do the following:

1. Instantiate an instance of the particle system prefab from OnDamageParticleSystemPrefab and store a reference to it in PlayParticleSystemOnDamage.
2. Sync the transform of the particle system to the entity's transform. If PlayParticleSystem OnDamage is enabled, it will play the particle system.
3. When the entity is destroyed, destroy the particle system GameObject.

PlayerAttractionAreaData

Component containing the data related to the player's attraction area.

PlayerExperienceState

Current state of the player's experience.

PlayerExperienceThisFrame

Dynamic buffer to queue experience points to add to the player in a given frame.

PlayerTag

Tag component to identify this as the player entity.

PlayerWorldUI

Data component to hold Unity object references to the transform and slider for the player's health bar which exists on a world space canvas.

PlayerWorldUISystem

System to initialize, update, and cleanup the player UI - used for player's in-world health bar.

PreviousPlayerInput

Data component to store the player's previous input.

ProcessDamageThisFrameSystem

System to process damage accumulated in the DamageThisFrame dynamic buffer and apply that damage towards the entity's current hit points.

RadioactiveWasteAttackSystem

System to handle instantiating AttackPrefabs of radioactive waste drops into the game world.

RadioactiveWasteWeaponData

Data component to store data relevant to spawning radioactive waste drops attacks.

RandomItemDropInfo

Serializable helper struct to facilitate Unity editor authoring of many of these elements contained in the ItemDrops List.

RandomItemToSpawn

Struct to store information related to items that can be spawned when this entity is destroyed. Used as the base element of the RandomItemToSpawnBlob blob array.

RandomItemToSpawnBlob

Component containing the blob array of RandomItemToSpawn elements. Used in the DestroyEntitySystem to determine which entity prefab to instantiate when the entity with this component is destroyed.

RecalculateStatsFlag

Enableable component to signify this entity needs their CharacterStatModificationState need to be recalculated in the RecalculateStatsSystem.

RecalculateStatsSystem

System to recalculate the character's current stat modifications stored in CharacterStatModificationState.

RegenerateLevelFlag

Enableable component to inform the LevelGenerationSystem that the level should be regenerated.

SatelliteAttackSystem

System to handle instantiating AttackPrefabs of satellites into the game world.

SatelliteWeaponData

Data component to store data relevant to spawning satellite attacks.

SawBladeAttackSystem

System to handle instantiating AttackPrefabs of satellites into the game world.

SawBladeWeaponData

Data component to store data relevant to spawning saw blade attacks.

ScaleOscillationData

Data component to store data related to scale oscillation. Scale oscillation is when the scale of an entity's LocalTransform is scaled up and down in an sine wave pattern.

ScaleOscillationSystem

System to update the uniform scale value of an entity based on data in ScaleOscillationData.

ScaleOscillationTimer

Timer used to evaluate the sine wave to determine scale at a given time.

ScreenEdgeBounceSystem

System to evaluate the position of entities tagged with a ScreenEdgeBounceTag and bounce them in a similar effect to a DVD screensaver.

ScreenEdgeBounceTag

Tag component to signify this entity should bounce once it reaches the edge of the screen.

ScrewdriverAttackSystem

System to handle instantiating AttackPrefabs of into the game world.

ScrewdriverWeaponTag

Tag component to signify this entity as the weapon entity to spawn screwdrivers into the game world.

SelectedCharacterReference

Data component to store a reference to the CharacterProperties of the player to be spawned.

SetEnemyLinearMovementFlag

Enableable component used to inform the EnemyLinearMovementSystem to update the Character MoveDirection to a new value. Used to improve efficiency so CharacterMoveDirection is only set when needed.

ShouldLoopAudioClipTag

Tag component to signify this entity should loop their associated audio clip for its full lifecycle.

ShowDamageNumberOnDamage

Component to hold data related to displaying damage numbers above characters when taking damage.

ShowDamageNumberOnDamageSystem

System to display damage numbers above entities when taking damage. Only will run on entities with their ShowDamageNumberOnDamage component enabled.

ShowDamageNumberOnHeal

Component to hold data related to displaying green "damage" number above characters when healing.

ShowDamageNumberOnHealSystem

System to display "damage" numbers above enemies when healing. Only will run on entities with the ShowDamageNumberOnHeal component enabled.

SpawnEvent

Dynamic buffer containing information pertaining to the spawn events for the current wave.

SpawnOnDestroy

Component to store the entity prefab that will be instantiated when the entity with this component is destroyed.

SpawnRandomPrefabHelper

Dynamic Buffer component which is required because there is no way to "covert" an GameObject into an Entity at runtime. As such the entity conversion will take place in the SpawnRandomOnDestroy Authoring.Baker and those prefabs will be stored in this Dynamic Buffer. At runtime, a blob asset will be generated storing these entity prefabs.

SpawnRandomPropertiesHashCode

Data component to store the hash code of the RandomItemDropProperties ScriptableObject associated with this entity.

SpawnWaveEnemyPrefab

Dynamic Buffer to hold all the enemy prefabs that will be spawned in the current wave.

StagePropertiesReference

Data component to store Unity Object reference to StageProperties ScriptableObject associated with the current stage.

StatModifier

Dynamic buffer to hold stat modifier types and values.

StatModifierEntityTag
Tag component to identify a stat modifier entity.

StatModifierInfo
Helper struct to author stat modifications in the editor.

TemporaryStatModifierActiveTime
Data component defining the time in seconds this temporary stat modifier entity should remain active.

TileScaleOverride
Material property override to set the tile scale factor.

UpdateCompanionGameObjectSystem
System to handle the lifecycle of companion GameObjects. Companion GameObjects are GameObjects that will match the position and rotation of their associated entity. This system has 3 foreach loops that do the following:

1. Instantiate the companion GameObject from Prefab assign values to a CompanionGameObject Data and add it to the entity.
2. Update the position and rotation of the companion GameObject from values in the entity's LocalToWorld component.
3. When the entity is destroyed, destroy the companion GameObject. Depending on settings, this may happen after a delay set in DelayCleanupTime.

UpdatePlayerHealthUIFlag
Flag component to inform the PlayerWorldUISystem to update the health slider.

UpgradePassiveFlag
Enableable component to signify this passive ability should be upgraded to the next level in the UpgradePassiveSystem.

UpgradePassiveSystem
System to upgrade passive to the next level.

UpgradeWeaponFlag
Flag component to signify this weapon should be upgraded to the next WeaponLevelData level.

UpgradeWeaponSystem
System to upgrade runtime weapon data when the UpgradeWeaponFlag is set to true.

WaveIndexOfSpawn
Shared component to store the index of the wave the enemy was spawned in.

WeaponActivationSystem
System to determine when a weapon should become active.

WeaponActiveFlag
Flag component to signify this weapon is currently active.

WeaponLevelData
Data component to store data related to player weapons.

WeaponLevelInfo
Data struct used to store weapon data and a description string for a single level of a weapon's upgrade path.

WeaponState
Data component to hold the current state of a weapon.

WeaponUpgradeData
Data component to hold blob asset reference for blob array of WeaponLevelData components for a weapon's upgrade path.

WeaponUpgradePropertiesReference
Data component to reference the WeaponUpgradeProperties for a weapon.

WeightedUpgradeProperties
Helper struct used for a pairing of UpgradeProperties with a weighted random value. Used in the PlayerUpgradeData to retrieve a random upgrade.

WrenchAttackSystem
System to handle instantiating AttackPrefabs of wrenches into the game world.

WrenchWeaponData
Data component to store data relevant to spawning wrench attacks.

Enums

AudioPriority
Enum used to set the priority of audio clips being played.

BonusItemType
Enum to define the type of bonus item. Used in BonusItemProperties.

CameraDirection
Byte flag definition for camera direction.

CharacterID

ID of the character, used for serialization.

EnemyType
: Enum used to identify unique alien types. This enum is set in the Unity Editor on the prefab for a given alien type. This enum will also serve as a key to map to the baked entity prefab. This is so EnemySpawnWaveProperties and EnemySpawnEventProperties ScriptableObjects can define which enemy types are present in the current wave or event and the EnemySpawnSystem can spawn the appropriate prefabs.

EntityRandomInitializationType
: Enum denoting the type of initialization that should be used when determining the initial seed of EntityRandom.

PlayerAnimationIndex
: Enum to define the indices for player animations.

SineWaveMoveDirection
: Enum type to define the direction of travel the enemy will move in as it also moves in a sine wave pattern.

SpawnFormation
: Defines the formation enemies for a SpawnEvent will spawn in. The formation will also determine certain aspects of their behavior in the game.

StageID
: ID of the stage, used for serialization.

StatModifierCalculationType
: Defines the calculation type of the stat modifier.

StatModifierType
: Enum containing all stat modifier types. Used as a way to author stat modifications in the editor and used as a key in several places in code to access information for a given stat modifier type via the associated StatModifierProperties

WeaponType
: Defines the type of the weapon. Often used as a key to obtain types relevant to the weapon in managed collections.

Struct ActiveStatModifierEntity

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer to hold the currently active stat modifier entities.

```
[InternalBufferCapacity(0)]
public struct ActiveStatModifierEntity : IBufferElementData
```

Remarks

Stat modifier entities each have their own StatModifier dynamic buffer which contains the specific stats that entity will be modifying. The purpose of having stat modifier entities is to enable the ability to have temporary stat modifications so that for example, a stat modifier entity can be destroyed after a set amount of time and all other stat modifications will remain in effect. Internal buffer capacity is set to 0, meaning all elements of the dynamic buffer will live outside the chunk. This is acceptable for this case as the buffer is only walked occasionally when a character's stats need to be recalculated in the Recalculate StatsSystem.

Fields

Value

```
public Entity Value
```

Field Value

Entity

Struct AliensDefeatedCount

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store the running count of aliens defeated in a run.

```
public struct AliensDefeatedCount : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public int Value
```

Field Value

int

Struct AlreadyStruckEntity

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer to store entities that have already been struck by the current group of laser strikes.

```
[InternalBufferCapacity(16)]
public struct AlreadyStruckEntity : IBufferElementData
```

Remarks

This collection is used to determine if a randomly selected enemy has already been struck so that it does not strike an enemy twice in a single group of attacks. As the laser strike weapon entity sits alone it its own chunk so a large internal buffer capacity can be allocated without issue. This buffer is larger than the theoretical worst case.

Fields

Value

```
public Entity Value
```

Field Value

Entity

Struct AnimationIndexOverride

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Material property override for the animation index. Setting this value will set the animation index on the material instance for this entity.

```
[MaterialProperty("_AnimationIndex", -1)]
public struct AnimationIndexOverride : IComponentData, IQueryTypeParameter
```

Remarks

Your IDE may gray out the Value field as this value is not used in our code. However, Unity uses it to apply this value to the material property defined in the MaterialProperty attribute. Be sure the MaterialProperty string exactly matches the reference string defined in the shader as this will silently fail if there is a typo.

Fields

Value

```
public float Value
```

Field Value

float

Struct AnimationOffsetOverride

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Material property override for the animation offset of the enemy. Setting this value will update the _SpawnTime property in the material instance of this entity. This provides an offset for the animation so all enemies can be on their own unique animation frame and will not appear to be synchronized.

```
[MaterialProperty("_SpawnTime", -1)]
public struct AnimationOffsetOverride : IComponentData, IQueryTypeParameter
```

Remarks

Your IDE may gray out the Value field as this value is not used in our code. However, Unity uses it to apply this value to the material property defined in the MaterialProperty attribute. Be sure the MaterialProperty string exactly matches the reference string defined in the shader as this will silently fail if there is a typo.

Fields

Value

```
public float Value
```

Field Value

float

See Also

InitializeAnimationOffsetOverrideFlag
CharacterAnimationSystem

Class ArtTestSceneAnimationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to set components to the graphics entity of animated characters in the art test scene.

```
public class ArtTestSceneAnimationAuthoring : MonoBehaviour
```

Fields

IsEnhanced

```
public bool IsEnhanced
```

Field Value

bool

IsFacingLeft

```
public bool IsFacingLeft
```

Field Value

bool

IsIdle

```
public bool IsIdle
```

Field Value

bool

Class ArtTestSceneAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to set values on the ArtTestSceneData used for initializing the secret art test scene.

```
public class ArtTestSceneAuthoring : MonoBehaviour
```

Fields

CautionPrefab

GameObject prefab of the caution tape entity that will be spawned around the edge of the play area.

```
public GameObject CautionPrefab
```

Field Value

GameObject

LevelSize

Level is created in a square, this defines the size of each side of the square.

```
public float LevelSize
```

Field Value

float

PlayerPrefab

GameObject prefab of the player entity.

```
public GameObject PlayerPrefab
```

Field Value

GameObject

Struct ArtTestSceneData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data pertinent to the secret art test scene.

```
public struct ArtTestSceneData : IComponentData, IQueryTypeParameter
```

Fields

CautionPrefab

Entity prefab for the caution tape graphics entity that is displayed around the far edges of the play area.

```
public Entity CautionPrefab
```

Field Value

Entity

LevelSize

Level is created in a square, this defines the size of each side of the square.

```
public float LevelSize
```

Field Value

float

PlayerPrefab

Prefab of the player entity.

```
public Entity PlayerPrefab
```

Field Value

Entity

See Also

InitializeArtTestSceneSystem

Class ArtTestScenePauseMenuUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller for the super secret art test scene.

```
public class ArtTestScenePauseMenuUIController : MonoBehaviour
```

Remarks

Provides basic functionality to pause, resume, and quit the art test scene.

Struct AttackPrefab

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Prefab of an in-world attack entity that will be spawned by an attack system.

```
public struct AttackPrefab : IComponentData, IQueryTypeParameter
```

Remarks

Many of these in-world attacks are projectiles that collide with enemies, however some like the black hole attack have slightly different behavior.

Fields

Value

```
public Entity Value
```

Field Value

Entity

Class AttractableItemAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components required for item attraction.

```
[RequireComponent(typeof(ItemAuthoring))]
public class AttractableItemAuthoring : MonoBehaviour
```

Remarks

ItemAuthoring is a required component to add other components necessary for items.

Fields

ItemMoveSpeed

Move speed of attractable items from starting position to the player.

```
public float ItemMoveSpeed
```

Field Value

float

Struct AttractableItemSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System that moves attractable items from their start position to the player.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct AttractableItemSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System uses an easing function to move attractable item first away from the player then towards the player. This easing function was chosen for aesthetic purposes as it gives visual feedback to the player when the item begins to attract to the player.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

DetectItemAttractionSystem

Struct AttractableItemTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to identify this entity as an attractable item.

```
public struct AttractableItemTag : IComponentData, IQueryTypeParameter
```

Remarks

Attractable items are items that exist in the game world that will move towards the player when the item is within the attractable radius of the player (Radius).

Enum AudioPriority

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enum used to set the priority of audio clips being played.

```
public enum AudioPriority : byte
```

Fields

`AboveAverage = 64`

`Average = 128`

`High = 32`

`Highest = 0`

`Low = 192`

`Lowest = 255`

`NextHighest = 1`

`VeryHigh = 16`

Remarks

When lots of audio sources are being played at once, audio sources with a lower priority value are more likely to play than audio sources with a higher priority value.

See Also

GameAudioController
MainMenuAudioController

Struct BaseHitPoints

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component storing the base hit points of a damageable entity.

```
public struct BaseHitPoints : IComponentData, IQueryTypeParameter
```

Remarks

Essentially max hit points, but max HP can go above this number with modifications

Fields

Value

```
public int Value
```

Field Value

int

See Also

CurrentHitPoints
DamageableEntityAuthoring

Class BeamInEffectController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

MonoBehaviour to control the beam in effect that is played when the player initially spawns into a level.

```
public class BeamInEffectController : MonoBehaviour
```

Fields

Instance

Public singleton access to this MonoBehaviour.

```
public static BeamInEffectController Instance
```

Field Value

BeamInEffectController

Methods

BeginBeamInEffect()

Method to invoke the coroutine to play the beam in effect. Method called from the GameStartSystem once level has loaded.

```
public void BeginBeamInEffect()
```

Class BeginGameOverSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to invoke the begin game over sequence.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
public class BeginGameOverSystem : SystemBase
```

Remarks

This system is required so that DestroyEntitySystem can be burst compiled as it moves the Action event managed type out of that system.

Fields

OnGameOver

This event is invoked as soon as the player is destroyed to trigger the Game Over sequence.

```
public Action OnGameOver
```

Field Value

Action

Methods

OnCreate()

```
protected override void OnCreate()
```

OnStartRunning()

```
protected override void OnStartRunning()
```

OnUpdate()

```
protected override void OnUpdate()
```

Struct BeginGameOverTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify the BeginGameOverSystem should begin execution.

```
public struct BeginGameOverTag : IComponentData, IQueryTypeParameter
```

Struct BlackHoleAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of black holes into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct BlackHoleAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system just spawns the black hole entity, further behavior of the black hole attack can be found in BlackHoleSystem. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its Cooldown Timer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct BlackHoleData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store persistent data for black hole attacks.

```
public struct BlackHoleData : IComponentData, IQueryTypeParameter
```

Fields

CollisionFilter

Collision filter used for detecting entities to be destroyed on screen.

```
public CollisionFilter CollisionFilter
```

Field Value

CollisionFilter

Remarks

The black hole in-world entity does not have a collider so this stores a collision filter used to find all enemies, gems, and items on screen when the attack executes in the BlackHoleSystem.

SaveItems

Boolean to determine if items should not be consumed by the black hole attack.

```
public bool SaveItems
```

Field Value

bool

Remarks

A random chance is evaluated in the BlackHoleAttackSystem to determine the result of this value. If true, only enemies will be consumed by the black hole. If false, items, experience point gems, and crates will be consumed by the black hole.

Class BlackHoleEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for black hole in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
[RequireComponent(typeof(InstantDestroyOnInteractionAuthoring))]
public class BlackHoleEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring, DestructibleEntityAuthoring, and InstantDestroyOnInteraction Authoring to ensure additional components are added to perform desired behaviors.

Fields

DamageTimer

Timer to determine the length of time from when the black hole graphic is shown until damage is actually applied.

```
public float DamageTimer
```

Field Value

float

Remarks

The reason for this timer is also primarily for a visual effect. The black hole attack instantly destroys all enemies, gems, and items on screen and this timer is required so that actual destruction occurs when the black hole is covering the entirety of the screen at max opacity - this effectively hides the destruction of entities so they don't all just blink out of existence at once.

GraphicsEntity

Reference to the graphical portion of the in-world black hole attack.

```
public GameObject GraphicsEntity
```

Field Value

GameObject

Remarks

This is required because the graphics entity will be disabled by default and will become enabled once SpawnGraphicTimer expires.

SpawnGraphicTimer

Timer to determine how long until the black hole should show its graphic.

```
public float SpawnGraphicTimer
```

Field Value

float

Remarks

This is used as a nice visual effect that synchronizes with the audio for the black hole. The audio begins to play for approx. 2 seconds as a bit of a warm-up for the attack, this is timed with a beat in the audio sound effect so it looks/sounds nice in the game when this attack executes.

See Also

BlackHoleSystem
DestroyAfterTime
BlackHoleData
GraphicsEntity

Struct BlackHoleState

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store information about the current state of the in-world black hole entity.

```
public struct BlackHoleState : IComponentData, IQueryTypeParameter, IEnableableComponent
```

Fields

DamageTimer

Timer to determine the length of time from when the black hole graphic is shown until damage is actually applied.

```
public float DamageTimer
```

Field Value

float

Remarks

The reason for this timer is also primarily for a visual effect. The black hole attack instantly destroys all enemies, gems, and items on screen and this timer is required so that actual destruction occurs when the black hole is covering the entirety of the screen at max opacity - this effectively hides the destruction of entities so they don't all just blink out of existence at once.

SpawnGraphicTimer

Timer to determine how long until the black hole should show its graphic.

```
public float SpawnGraphicTimer
```

Field Value

float

Remarks

This is used as a nice visual effect that synchronizes with the audio for the black hole. The audio begins to play for approx. 2 seconds as a bit of a warm-up for the attack, this is timed with a beat in the audio sound effect so it looks/sounds nice in the game when this attack executes.

Struct BlackHoleSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System that controls the behavior of the black hole in-world attack entity.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct BlackHoleSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Once a black hole entity is spawned into the world via the BlackHoleAttackSystem, this system will decrement timers stored in the BlackHoleState of the black hole in-world entity to determine when the graphics should begin to be shown, and when to evaluate the screen for enemies, gems, and items to be consumed by the black hole. Note that due to how the graphics entity is re-enabled by this system (removing the Disabled component), this will also start other effects applied to the graphics entity such as LinearScaleTransformationSystem, ConstantRotationSystem, and FadeAttackInOutSystem.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

```
state SystemState
```

Class BlackHoleWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add BlackHoleWeaponTag to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
[RequireComponent(typeof(EntityRandomAuthoring))]
public class BlackHoleWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring and EntityRandom scripts to ensure all components required for executing attacks are added to the entity.

Struct BlackHoleWeaponTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity as the weapon entity to spawn black holes into the game world.

```
public struct BlackHoleWeaponTag : IComponentData, IQueryTypeParameter
```

Class BonusItemProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject to define properties related to bonus items. Bonus items are items shown in the level up and crate UI if no further upgrades to attacks or passive items are available to the player.

```
[CreateAssetMenu(fileName = "BonusItemProperties", menuName =
"ScriptableObjects/BonusItemProperties")]
public class BonusItemProperties : ScriptableObject
```

Remarks

Currently bonus items can grant additional health points or money.

Fields

ItemDescription

Longer description of the item, to be displayed in UI when bonus item is displayed.

```
public string ItemDescription
```

Field Value

string

ItemIcon

Icon of the item, to be displayed in UI when bonus item is displayed.

```
public Sprite ItemIcon
```

Field Value

Sprite

ItemName

Name of the item, to be displayed in UI when bonus item is displayed.

```
public string ItemName
```

Field Value

string

ItemType

Type of the item.

```
public BonusItemType ItemType
```

Field Value

BonusItemType

ItemValue

Value of hit points or money to be granted if bonus item is selected.

```
public int ItemValue
```

Field Value

int

Enum BonusItemType

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enum to define the type of bonus item. Used in BonusItemProperties.

```
public enum BonusItemType : byte
```

Fields

`Health = 1`

`Money = 2`

`None = 0`

See Also

BonusItemProperties

Class BoomerangMovementAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add the BoomerangMovementData to an entity.

```
public class BoomerangMovementAuthoring : MonoBehaviour
```

Remarks

Note that this authoring component does not set the StartTime as that value would not be known at baking time. So you must initialize this value upon instantiation of the entity.

Fields

BoomerangMoveSpeed

```
public float BoomerangMoveSpeed
```

Field Value

float

Struct BoomerangMovementData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data related to boomerang movement.

```
public struct BoomerangMovementData : IComponentData, IQueryTypeParameter
```

Remarks

Used for wrench attack - WrenchAttackSystem

Fields

MoveSpeed

```
public float MoveSpeed
```

Field Value

float

StartTime

```
public double StartTime
```

Field Value

double

Struct BoomerangMovementSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to implement the boomerang movement.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct BoomerangMovementSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Updates in the DS_TranslationSystemGroup which updates before Unity's TransformSystemGroup, so the LocalTransform for this entity can be safely updated.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class BounceOnEnvironmentAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add the BounceOnEnvironmentTag to an entity.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
public class BounceOnEnvironmentAuthoring : MonoBehaviour
```

Remarks

Requires the EntityInteractionAuthoring component as the bounce behavior is implemented as an entity interaction.

Struct BounceOnEnvironmentJob

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Trigger events job to detect trigger events between entities tagged with BounceOnEnvironmentTag and EnvironmentTag and will raise an EntityInteraction on the entity tagged with BounceOnEnvironmentTag. Scheduled by the BounceOnEnvironmentJob.

```
[BurstCompile]
public struct BounceOnEnvironmentJob : ITriggerEventsJob, ITriggerEventsJobBase
```

Remarks

This trigger job is unique in that when raising EntityInteractions it will not scan the buffer for the existence of the environment entity. This is because the desired behavior is such that if a bounceable entity collided with the same environment entity multiple times, it should bounce off it every time.

Fields

BounceOnEnvironmentLookup

Component lookup used for identifying entities that will bounce off environment entities.

```
[ReadOnly]
public ComponentLookup<BounceOnEnvironmentTag> BounceOnEnvironmentLookup
```

Field Value

ComponentLookup<BounceOnEnvironmentTag>

EnvironmentLookup

Component lookup used for identifying environment entities.

```
[ReadOnly]
public ComponentLookup<EnvironmentTag> EnvironmentLookup
```

Field Value

ComponentLookup<EnvironmentTag>

InteractionBufferLookup

Buffer lookup used for raising entity interactions on the bounceable entity.

```
public BufferLookup<EntityInteraction> InteractionBufferLookup
```

Field Value

BufferLookup<EntityInteraction>

Methods

Execute(TriggerEvent)

```
[BurstCompile]
public void Execute(TriggerEvent triggerEvent)
```

Parameters

`triggerEvent` TriggerEvent

Struct BounceOnEnvironmentSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to schedule the BounceOnEnvironmentJob to detect trigger events between entities tagged with BounceOnEnvironmentTag and EnvironmentTag and will raise an EntityInteraction.

```
[UpdateInGroup(typeof(DS_PhysicsSystemGroup))]
public struct BounceOnEnvironmentSystem : ISystem, ISystemCompilerGenerated
```

Remarks

As the scheduled job is a trigger events job, this system updates in the DS_PhysicsSystemGroup to ensure trigger events for the current physics step have been raised.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct BounceOnEnvironmentTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity should bounce off environment entities with a collider.

```
public struct BounceOnEnvironmentTag : IComponentData, IQueryTypeParameter
```

Struct CO2CloudAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of CO2 clouds into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct CO2CloudAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

CO2 clouds will exist in the transform hierarchy of the player, making them follow the player's movement for as long as they are active. CO2 clouds are affected by modifications to the player's attack area and damage dealt stats. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class CO2CloudEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for CO2 cloud in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
public class CO2CloudEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime

Class CO2CloudWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add CO2CloudWeaponTag to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
public class CO2CloudWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity.

Struct CO2CloudWeaponTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity as the weapon entity to spawn CO2 clouds into the game world.

```
public struct CO2CloudWeaponTag : IComponentData, IQueryTypeParameter
```

Class CameraAspectRatioFixer

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

MonoBehaviour script to add letterboxing to screens outside supported aspect ratios.

```
public class CameraAspectRatioFixer : MonoBehaviour
```

Enum CameraDirection

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Byte flag definition for camera direction.

```
[Flags]
public enum CameraDirection : byte
```

Fields

`Down = 4`

`Left = 8`

`None = 0`

`Right = 2`

`Up = 1`

See Also

DestroyOffCameraData
DestroyOffCameraSystem

Struct CameraTarget

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component attached to the player entity. Used for systems that need information about the camera.

```
public struct CameraTarget : IComponentData, IQueryTypeParameter
```

Fields

HalfExtents

```
public float3 HalfExtents
```

Field Value

float3

Transform

```
public UnityObjectRef<Transform> Transform
```

Field Value

UnityObjectRef<Transform>

YOffset

```
public float YOffset
```

Field Value

float

See Also

InitializeCameraTarget
InitializeCameraTargetSystem
MoveCameraSystem

Class CameraTargetAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize the InitializeCameraTarget component on the player entity.

```
public class CameraTargetAuthoring : MonoBehaviour
```

Fields

YOffset

```
public float YOffset
```

Field Value

float

Class CameraTargetObject

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Singleton MonoBehaviour to identify this GameObject as the target to use for the main Cinemachine camera.

```
public class CameraTargetObject : MonoBehaviour
```

Fields

Instance

```
public static CameraTargetObject Instance
```

Field Value

CameraTargetObject

See Also

CameraTarget
InitializeCameraTargetSystem

Class CapabilityResultController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with a single capability element on the end results panel.

```
public class CapabilityResultController : MonoBehaviour
```

Remarks

Capabilities refer to weapons and passive abilities owned and leveled up by the player. In the end results panel, this will display the icon, name, and level at game over time of each capability owned by the player.

Methods

InitializeUI(Sprite, string, int)

```
public void InitializeUI(Sprite icon, string displayName, int levelIndex)
```

Parameters

`icon` Sprite

`displayName` string

`levelIndex` int

Class CapabilityStatusUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with an individual capability to be displayed in the game pause menu.

```
public class CapabilityStatusUIController : MonoBehaviour
```

Remarks

Capabilities are weapons and passive abilities the player currently has. This controller displays the icon of the capability and uses UI toggles to indicate the number of levels the capability has (via toggle count) and what the current level is (via enabled toggle count).

Methods

DisplayUI(UpgradeProperties, int)

```
public void DisplayUI(UpgradeProperties properties, int currentLevelIndex)
```

Parameters

`properties` UpgradeProperties

`currentLevelIndex` int

See Also

CurrentCapabilitiesUIController

Class CapabilityUpgradeController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Managed controller to handle upgrading player capabilities from the GameObject side. This script is required to serve as a bridge between GameObject and ECS sides of the game. UI elements will call methods in this script to configure things on both GameObject and ECS sides related to upgrading capabilities (weapons and passive items).

```
public class CapabilityUpgradeController : MonoBehaviour
```

Fields

Instance

Public singleton access so UI elements can call methods on this controller.

```
public static CapabilityUpgradeController Instance
```

Field Value

CapabilityUpgradeController

OnAddNewPassive

Event invoked when the player gets a new passive item. UI events listen to this to display the new passive item icon in the HUD.

```
public Action<PassiveUpgradeProperties> OnAddNewPassive
```

Field Value

Action <PassiveUpgradeProperties>

OnAddNewWeapon

Event invoked when the player gets a new weapon. UI events listen to this to display the new weapon icon in the HUD.

```
public Action<WeaponUpgradeProperties> OnAddNewWeapon
```

Field Value

Action <WeaponUpgradeProperties>

Methods

CollectBonusItem(BonusItemType, int)

Method for granting the player a bonus item. A bonus item is shown when a player levels up or obtains a crate when all capability slots are full and at max level.

```
public void CollectBonusItem(BonusItemType itemType, int itemValue)
```

Parameters

`itemType` BonusItemType

Type of bonus item - health or money.

`itemValue` int

Value of health or money to grant to the player.

See Also
BonusItemProperties

UpgradeCapability(UpgradeProperties)

"Entry point" for upgrading a capability. Depending on the type of capability specific methods will be called by this method.

```
public void UpgradeCapability(UpgradeProperties upgradeProperties)
```

Parameters

`upgradeProperties` UpgradeProperties

Upgrade properties ScriptableObject of the capability to be upgraded. Will either be of type Weapon UpgradeProperties or PassiveUpgradeProperties.

Struct CapabilityUpgradeLevel

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Helper struct to pair an UpgradeProperties ScriptableObject with the index of its next level.

```
public struct CapabilityUpgradeLevel
```

Fields

NextLevelIndex

```
public int NextLevelIndex
```

Field Value

int

UpgradeProperties

```
public UpgradeProperties UpgradeProperties
```

Field Value

UpgradeProperties

Class CapabilityUpgradeUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with a singular capability level up UI element that is shown in the level up UI.

```
public class CapabilityUpgradeUIController : MonoBehaviour
```

Remarks

Displays information about a single weapon or passive ability that is available to be leveled up.

Properties

Button

```
public Button Button { get; }
```

Property Value

Button

Methods

DisableInteraction()

```
public void DisableInteraction()
```

SetLevelUpUI(BonusItemProperties)

Alternate method to set UI elements for "bonus items" which will be displayed when the player has all 10 capabilities at max level.

```
public void SetLevelUpUI(BonusItemProperties itemProperties)
```

Parameters

`itemProperties` BonusItemProperties

Properties of the bonus item to be displayed in the UI.

SetLevelUpUI(CapabilityUpgradeLevel)

Sets UI elements to provide information on what will change with the new level. Adds event listeners that will be invoked when the element is clicked.

```
public void SetLevelUpUI(CapabilityUpgradeLevel upgradeLevel)
```

Parameters

`upgradeLevel` CapabilityUpgradeLevel

Helper struct containing the UpgradeProperties associated with the capability to display and the level index it is currently at.

Class CharacterAnimationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components required for character animation.

```
public class CharacterAnimationAuthoring : MonoBehaviour
```

See Also

PlayerAnimationAuthoring
EnemyAnimationAuthoring

Struct CharacterAnimationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle changes to character materials to facilitate animation visual effects. This system has 4 foreach loops that do the following:

1. Initializes AnimationOffsetOverride for enemy entities so all their animations are not synchronized.
2. Initializes the EnhanceThicknessOverride to apply a visual enhancement outline on the outside of an enemy to denote it as an enhanced entity.
3. Set which animation is currently playing for the player entity. Used to switch between walk and idle animations depending on current input (CharacterMoveDirection) and previous input (PreviousPlayerInput).
4. Plays the dissolve effect when the character is destroyed. (see DissolveData and DissolveOverride)

```
[BurstCompile]
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct CharacterAnimationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system executes in the DS_EffectsSystemGroup which executes towards the end of the frame.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class CharacterAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add necessary components to character entities.

```
[RequireComponent(typeof(DamageableEntityAuthoring))]
[RequireComponent(typeof(GraphicsEntityAuthoring))]
public class CharacterAuthoring : MonoBehaviour
```

Remarks

Requiring the DamageableEntityAuthoring and GraphicsEntityAuthoring to ensure all necessary components are added.

Fields

MoveSpeed

Base movement speed of the character in units per second.

```
public float MoveSpeed
```

Field Value

float

Struct CharacterBaseMoveSpeed

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to hold the character's base movement speed in units per second.

```
public struct CharacterBaseMoveSpeed : IComponentData, IQueryTypeParameter
```

Remarks

Movement speed can be temporarily or permanently modified via stat modification (see MoveSpeed)

Fields

Value

```
public float Value
```

Field Value

float

Struct CharacterDefaultModificationValues

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to hold the default modification values for each StatModifierType.

```
public struct CharacterDefaultModificationValues : IComponentData, IQueryTypeParameter
```

Remarks

This is used as a baseline component to initialize CharacterStatModificationState before calculating stat modifications.

Fields

AdditionalAttackProjectiles

Number of additional attack projectiles to spawn for each applicable attack.

```
public int AdditionalAttackProjectiles
```

Field Value

int

AdditionalHitPoints

Additional hit points to increase the character's maximum hit points.

```
public int AdditionalHitPoints
```

Field Value

int

AttackArea

Attack area modification. This value is multiplied by the base attack area for an attack in various attack systems to calculate the current scale of the attack.

```
public float AttackArea
```

Field Value

float

AttackCooldown

Attack cooldown modification. This value is multiplied by the base cooldown for an attack in various attack systems to calculate the current effective cooldown for that attack.

```
public float AttackCooldown
```

Field Value

float

AttackDuration

Attack duration modification. This value is multiplied by the base attack duration for an attack in various attack systems to calculate the current effective duration for how long the attack entity exists in the world.

```
public float AttackDuration
```

Field Value

float

AttackProjectileSpeed

Attack projectile speed modification. This value is multiplied by the base projectile speed for an attack in various attack systems to calculate the current effective projectile speed of attack entities.

```
public float AttackProjectileSpeed
```

Field Value

float

DamageDealt

Damage dealing modification. This value is multiplied by the base damage value for an attack in various attack systems to calculate the current effective damage to be assigned to that attack entity.

```
public float DamageDealt
```

Field Value

float

DamageReceived

Reduce incoming damage points by this integer value.

```
public int DamageReceived
```

Field Value

int

ExperiencePointGain

Experience point gain modifier. This value is multiplied by the value of experience points collected by the player in the HandlePlayerExperienceThisFrameSystem to calculate the effective experience point gain for the collected gem.

```
public float ExperiencePointGain
```

Field Value

float

HealthRegeneration

Health regeneration modification. This value represents how many hit points per second are restored in the CharacterHealthRegenerationSystem.

```
public float HealthRegeneration
```

Field Value

float

ItemAttractionRadius

Item attraction radius modifier. This value is multiplied by the player's Radius in the DetectItemAttraction System to calculate the effective attraction area for the player.

```
public float ItemAttractionRadius
```

Field Value

float

MoveSpeed

Move speed modification. This value is multiplied by the character's CharacterBaseMoveSpeed in the CharacterMoveSystem to calculate the current effective move speed.

```
public float MoveSpeed
```

Field Value

float

Methods

GetDefaultValue(StatModifierType)

Helper method to easily get the default value of a given StatModifierType.

```
public float GetDefaultValue(StatModifierType type)
```

Parameters

`type` StatModifierType

Stat modifier type to get the default value of.

Returns

float

Default value of the desired stat modifier type.

Exceptions

ArgumentOutOfRangeException

Throws if invalid stat modifier type is passed into this function.

SetDefaultValue(StatModifierType, float)

Helper method to easily initialize values for a given StatModifierType.

```
public void SetDefaultValue(StatModifierType type, float value)
```

Parameters

`type` StatModifierType

Type of stat modification to initialize.

`value` float

Value of the stat modifier type.

Exceptions

ArgumentOutOfRangeException

Throws if stat modifier type is invalid.

Struct CharacterEntity

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component for various entities to reference their associated character entity.

```
public struct CharacterEntity : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public Entity Value
```

Field Value

Entity

Struct CharacterHealthRegenerationState

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to hold the character's hit point accumulator. When component is enabled, health regeneration will take place in the CharacterHealthRegenerationSystem.

```
public struct CharacterHealthRegenerationState : IComponentData,
IQueryTypeParameter, IEnableableComponent
```

Fields

HitPointAccumulator

Holds fractional values of hit points to recover. Once value is >= 1, 1 hit point will be added the character's CurrentHitPoints until it reaches the player's maximum hit point value.

```
public float HitPointAccumulator
```

Field Value

float

Remarks

This is required because hit points are stored as integers, but health regeneration recovers fractions of a hit point per second and occurs continuously when the player is not at max health.

Struct CharacterHealthRegenerationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle regenerating character health.

```
[UpdateBefore(typeof(ProcessDamageThisFrameSystem))]
public struct CharacterHealthRegenerationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System will only run on entities with the CharacterHealthRegenerationState component enabled. CharacterHealthRegenerationState holds fractions of a hit point that are constantly added to while system is active. Once a whole number is reached, the whole hit point integer will be added to the character's current hit points. Updates before the ProcessDamageThisFrameSystem to ensure regenerated hit points are added to the DamageThisFrame buffer before processing damage for the current frame.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Enum CharacterID

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ID of the character, used for serialization.

```
public enum CharacterID : byte
```

Fields

`Daxode = 4`

`Felikss = 16`

`Hazel = 8`

`Mockarutan = 2`

`None = 0`

`Turbo = 1`

`Zenna = 32`

Remarks

Each character is assigned a bit value so they can be used as bitflags in UnlockedStageFlags.

Struct CharacterInitializationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize values pertaining to game characters.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
[UpdateBefore(typeof(GameStartSystem))]
public struct CharacterInitializationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Important that this updates "before" the GameStartSystem as that is where the player spawns and a 1 frame delay is required before calling UpgradeCapability on the EntityUpgradeControllerManaged otherwise some exceptions are thrown when subscene is open.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct CharacterMoveDirection

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component defining the current move direction of a character.

```
public struct CharacterMoveDirection : IComponentData, IQueryTypeParameter
```

Remarks

Character move direction is set in various systems depending on the type of character. Player entities get their value set in GetPlayerInputSystem, while enemies get their values set in systems like EnemyMoveToPlayerJob or EnemySineWaveMovementSystem. Non-zero values should be normalized.

Fields

Value

```
public float2 Value
```

Field Value

float2

Struct CharacterMoveSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to move characters in the game world based off input from their CharacterMoveDirection. Uses PhysicsVelocity to apply movement so collisions with other entities are properly handled.

```
[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
[UpdateBefore(typeof(PhysicsSystemGroup))]
public struct CharacterMoveSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system initially moved characters by adding to the LocalTransform.Position of a character. However, this lead to some undesired behavior where entities could overlap each other and move through other physical colliders. Converting this system to modify the PhysicsVelocity component resolved this issue as collisions are properly accounted for and resolved. As this system modifies physics components, it updates in the FixedStepSimulationSystemGroup before the PhysicsSystemGroup.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class CharacterProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject to define properties for player characters. Certain fields are used just for UI display, while others affect gameplay.

```
[CreateAssetMenu(fileName = "-CharacterProperties", menuName = "ScriptableObjects/Character
Properties")]
public class CharacterProperties : ScriptableObject
```

Fields

BuffDescription

Description of the initial buff(s) for the character to be displayed in the character selection UI.

```
[TextArea]
public string BuffDescription
```

Field Value

string

CharacterID

ID of the character, used for serialization.

```
public CharacterID CharacterID
```

Field Value

CharacterID

CharacterMaterial

Material of the character that references the appropriate sprite sheet.

```
public Material CharacterMaterial
```

Field Value

Material

CharacterName

Name of the character.

```
public string CharacterName
```

Field Value

string

CharacterSprite

Sprite of the character to be shown in the character selection UI.

```
public Sprite CharacterSprite
```

Field Value

Sprite

HitPoints

Base hit points of the character.

```
public int HitPoints
```

Field Value

int

SpaceAgencyProperties

Properties of the space agency to aid with displaying flag and name of the space agency in the character selection screen and results panel.

```
public SpaceAgencyProperties SpaceAgencyProperties
```

Field Value

SpaceAgencyProperties

StartingWeapon

Starting weapon the player spawns with.

```
public WeaponUpgradeProperties StartingWeapon
```

Field Value

WeaponUpgradeProperties

StatOverrides

Initial stat modifications the player spawns with.

```
public StatModifierInfo[] StatOverrides
```

Field Value

StatModifierInfo[]

UnlockCost

Cost to unlock this character.

```
public int UnlockCost
```

Field Value

int

Methods

GetCurrentValue(StatModifierType, float)

Method used to get the current value of a given StatModifierType. Only used in the character selection panel to display stats unique to this character.

```
public float GetCurrentValue(StatModifierType statModifierType, float defaultValue)
```

Parameters

`statModifierType` StatModifierType

Requested stat modification type.

`defaultValue` float

Default value of this stat modification. Used so differences can be added to default value.

Returns

float

Value of the given stat modification type for this character.

Class CharacterSelectButtonUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with a single UI element for a character that can be selected in the main menu UI.

```
public class CharacterSelectButtonUIController : MonoBehaviour
```

Properties

SelectionButton

```
public Button SelectionButton { get; }
```

Property Value

Button

Methods

ShowCharacterUI(CharacterProperties, bool, UnityAction)

Sets UI elements associated with the stage available to be selected.

```
public void ShowCharacterUI(CharacterProperties characterProperties, bool isUnlocked,
UnityAction showSelectedCharacterInformationAction)
```

Parameters

`characterProperties` CharacterProperties

ScriptableObject containing information and assets pertaining to this character.

`isUnlocked` bool

Flag to denote if the character is unlocked. If the character is locked, a lock icon will be shown with the price to unlock.

`showSelectedCharacterInformationAction` UnityAction

Method that will be called when the main button associated with this UI element is clicked. Will set UI elements in SelectCharacter(CharacterProperties, bool, bool)

See Also

CharacterSelectPanelUIController

See Also

CharacterSelectPanelUIController

Class CharacterSelectPanelUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the panel to select which character to play as in the main title screen menu.

```
public class CharacterSelectPanelUIController : MonoBehaviour
```

Methods

HideCharacterSelectPanel()

```
public void HideCharacterSelectPanel()
```

ShowCharacterSelectPanel()

```
public void ShowCharacterSelectPanel()
```

Struct CharacterStatModificationState

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store the current stat modification values for a character. Several systems reference this component to calculate effective stat data (i.e. take the base value of a stat, and add or multiply the relevant stat modification value as required).

```
public struct CharacterStatModificationState : IComponentData, IQueryTypeParameter
```

Constructors

CharacterStatModificationState(CharacterDefaultModificationValues)

Constructor for easy creation of initial CharacterStatModificationState using CharacterDefaultModificationValues.

```
public CharacterStatModificationState(CharacterDefaultModificationValues defaultValues)
```

Parameters

`defaultValues` CharacterDefaultModificationValues

Default values for each modification state.

Remarks

Used during CharacterInitializationSystem and RecalculateStatsSystem to get a baseline of stat values before applying modifications stored in the StatModifier buffers of stat modifier entities stored in ActiveStatModifierEntity.

Fields

AdditionalAttackProjectiles

Number of additional attack projectiles to spawn for each applicable attack group.

```
public int AdditionalAttackProjectiles
```

Field Value

int

AdditionalHitPoints

Additional hit points to increase the character's maximum hit points.

```
public int AdditionalHitPoints
```

Field Value

int

AttackArea

Attack area modification. This value is multiplied by the base attack area for an attack in various attack systems to calculate the current scale of the attack.

```
public float AttackArea
```

Field Value

float

AttackCooldown

Attack cooldown modification. This value is multiplied by the base cooldown for an attack in various attack systems to calculate the current effective cooldown for that weapon.

```
public float AttackCooldown
```

Field Value

float

AttackDuration

Attack duration modification. This value is multiplied by the base attack duration for an attack in various attack systems to calculate the current effective duration for how long the attack entity exists in the world.

```
public float AttackDuration
```

Field Value

float

AttackProjectileSpeed

Attack projectile speed modification. This value is multiplied by the base projectile speed for an attack in various attack systems to calculate the current effective projectile speed of attack entities.

```
public float AttackProjectileSpeed
```

Field Value

float

DamageDealt

Damage dealing modification. This value is multiplied by the base damage value for an attack in various attack systems to calculate the current effective damage to be assigned to that attack entity.

```
public float DamageDealt
```

Field Value

float

DamageReceived

Reduce incoming damage points by this integer value.

```
public int DamageReceived
```

Field Value

int

ExperiencePointGain

Experience point gain modifier. This value is multiplied by the value of experience points collected by the player in the HandlePlayerExperienceThisFrameSystem to calculate the effective experience point gain for the collected gem.

```
public float ExperiencePointGain
```

Field Value

float

HealthRegeneration

Health regeneration modification. This value represents how many hit points per second are restored in the CharacterHealthRegenerationSystem.

```
public float HealthRegeneration
```

Field Value

float

ItemAttractionRadius

Item attraction radius modifier. This value is multiplied by the player's Radius in the DetectItemAttraction System to calculate the effective attraction area for the player.

```
public float ItemAttractionRadius
```

Field Value

float

MoveSpeed

Move speed modification. This value is multiplied by the character's CharacterBaseMoveSpeed in the CharacterMoveSystem to calculate the current effective move speed.

```
public float MoveSpeed
```

Field Value

float

Methods

AddStat(StatModifier)

Method to add stat modifiers to the character's current set of stat modifications. This method will also clamp the stat modification values to ensure they stay within an acceptable range to facilitate balanced gameplay.

```
public void AddStat(StatModifier statModifier)
```

Parameters

`statModifier` StatModifier

Stat modifier to add to the character's current stat modifications.

Remarks

Certain values are stored as integers and must be casted to an integer after clamping. Minimum and maximum stat modification values are authored in StatModifierProperties, stored in TMG.DOTSSurvivors.StatModifierController._statModifierPropertiesLookup, and clamped in ClampModificationValue(StatModifierType, ref float).

Exceptions

ArgumentOutOfRangeException

Thrown if an invalid StatModifierType is set on this StatModifier element.

GetCurrentValue(StatModifierType)

Helper function to get the current modification value for a given StatModifierType.

```
public float GetCurrentValue(StatModifierType type)
```

Parameters

`type` StatModifierType

Type of stat modifier to be returned.

Returns

float

The current value of the requested stat modifier. Note that this always returns a float although certain stats are stored as integers.

Exceptions

ArgumentOutOfRangeException

Throws if an invalid StatModifierType is passed into this function.

Struct CoinsCollected

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store the running count of coins collected in a run.

```
public struct CoinsCollected : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public int Value
```

Field Value

int

Class CompanionGameObjectAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to set values for CompanionGameObjectInitializationData which in turn will initialize values of CompanionGameObjectData in the UpdateCompanionGameObjectSystem.

```
public class CompanionGameObjectAuthoring : MonoBehaviour
```

Fields

CompanionPrefab

GameObject prefab that will be spawned during initialization. Instance of this prefab will match the position of the associated entity.

```
public GameObject CompanionPrefab
```

Field Value

GameObject

DelayCleanupTime

Time in seconds to delay destruction of the companion GameObject.

```
public float DelayCleanupTime
```

Field Value

float

Struct CompanionGameObjectData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data related to the companion GameObject that will be matching the position of the associated entity.

```
public struct CompanionGameObjectData : ICleanupComponentData,
IComponentData, IQueryTypeParameter
```

Remarks

This component is a cleanup component which cannot be added to the entity through baking. As such CompanionGameObjectInitializationData is required to initialize this component.

Fields

DelayCleanupTimer

Time in seconds to delay destruction of the companion GameObject.

```
public float DelayCleanupTimer
```

Field Value

float

Transform

Transform reference of the instantiated GameObject that will match the position of the associated entity.

```
public UnityObjectRef<Transform> Transform
```

Field Value

UnityObjectRef<Transform>

Struct CompanionGameObjectInitializationData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Initialization data for CompanionGameObjectData.

```
public struct CompanionGameObjectInitializationData : IComponentData, IQueryTypeParameter
```

Remarks

Defines a GameObject to spawn and match the position of the associated entity. Initialization data is required as CompanionGameObjectData is an ICleanupComponentData which cannot be added to an entity through baking.

Fields

DelayCleanupTime

Time in seconds to delay destruction of the companion GameObject.

```
public float DelayCleanupTime
```

Field Value

float

Prefab

GameObject prefab that will be spawned during initialization. Instance of this prefab will match the position of the associated entity.

```
public UnityObjectRef<GameObject> Prefab
```

Field Value

UnityObjectRef<GameObject>

Class ConstantRotationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add the ConstantRotationAuthoring component to entities.

```
public class ConstantRotationAuthoring : MonoBehaviour
```

Fields

EulerDegreesPerSecond

Euler rotation to be applied along each axis, authored in degrees per second and converted to radians per second during baking.

```
public Vector3 EulerDegreesPerSecond
```

Field Value

Vector3

RotationOrder

Order the euler axes will be applied when converting to a quaternion before applying the rotation.

```
public math.RotationOrder RotationOrder
```

Field Value

math.RotationOrder

See Also

ConstantRotationSystem

Struct ConstantRotationData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data related to constant rotation of an entity.

```
public struct ConstantRotationData : IComponentData, IQueryTypeParameter
```

Remarks

ConstantRotationSystem will apply constant rotation in the euler defined in this component as long as this component is on the entity. Used mostly for visual effects where an attack entity may appear to constantly rotate through its full lifecycle.

Fields

EulerRadiansPerSecond

Euler rotation to be applied along each axis, stored in radians per second.

```
public float3 EulerRadiansPerSecond
```

Field Value

float3

RotationOrder

Order the euler axes will be applied when converting to a quaternion before applying the rotation.

```
public math.RotationOrder RotationOrder
```

Field Value

math.RotationOrder

See Also

ConstantRotationAuthoring

Struct ConstantRotationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to apply a constant rotation to entities.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct ConstantRotationSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

ConstantRotationData
ConstantRotationAuthoring

Class CounterRotationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize CounterRotationData on the associated entity.

```
public class CounterRotationAuthoring : MonoBehaviour
```

Fields

EulerOffset

Euler offset to apply additional rotation correction. Authored in degrees, stored in radians.

```
[Tooltip("Enter in Degrees")]
public Vector3 EulerOffset
```

Field Value

Vector3

Struct CounterRotationData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component stores data related to counter rotation. Counter rotation is when the rotation of a child entity applies a rotation opposite to that of the parent entity, giving it the visual appearance of having a fixed orientation.

```
public struct CounterRotationData : IComponentData, IQueryTypeParameter
```

Remarks

Used on drone attack as main entity will constantly rotate around the player and drone entities will stay visually up and down.

Fields

EulerOffset

Euler offset to apply additional rotation correction, stored in radians.

```
public float3 EulerOffset
```

Field Value

float3

Struct CounterRotationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to apply counter rotation to an entity. Counter rotation is when the rotation of a child entity applies a rotation opposite to that of the parent entity, giving it the visual appearance of having a fixed orientation.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct CounterRotationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Used on drone attack as main entity will constantly rotate around the player and drone entities will stay visually up and down. Updates in the DS_TranslationSystemGroup which updates before Unity's TransformSystemGroup, so LocalTransform can be safely modified.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class CrateItemAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script for crate entity.

```
[RequireComponent(typeof(ItemAuthoring))]
[RequireComponent(typeof(DestroySelfOnInteractionAuthoring))]
public class CrateItemAuthoring : MonoBehaviour
```

Remarks

Additional authoring scripts required to add components necessary for desired behavior.

Struct CrateItemTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to identify crate entities.

```
public struct CrateItemTag : IComponentData, IQueryTypeParameter
```

Remarks

Crate entities have a chance to be dropped by boss enemies for a spawn wave. When a crate is picked up by the player a sequence will play and grant the player an upgrade to a random weapon or passive ability they already have selected. If all weapons or passive items the player currently has are fully upgraded, additional money will be granted.

Class CrateUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated the UI panel that is displayed when the player picks up a loot crate dropped by a boss enemy.

```
public class CrateUIController : MonoBehaviour
```

Remarks

Handles displaying different UI elements and invoking timelines and particle systems for that sweet, sweet dopamine rush.

Methods

BeginFinalSequence()

```
public void BeginFinalSequence()
```

StopParticleSystems()

```
public void StopParticleSystems()
```

Class CurrentCapabilitiesUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI Controller associated with the capability status display that is shown in the game pause menu.

```
public class CurrentCapabilitiesUIController : MonoBehaviour
```

Remarks

When the player pauses the game, this status UI will be shown with an icon of all capabilities (weapons and passive abilities) that the character currently has. Each icon will have a series of boxes to indicate the max amount of levels that capability has, and what is the current level.

Methods

HideCapabilitiesUI()

```
public void HideCapabilitiesUI()
```

ShowCapabilitiesUI()

```
public void ShowCapabilitiesUI()
```

See Also

CapabilityStatusUIController

Struct CurrentHitPoints

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component storing the current hit points of a damageable entity.

```
public struct CurrentHitPoints : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public int Value
```

Field Value

int

See Also

BaseHitPoints
DamageableEntityAuthoring

Class CurrentStatsUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the current stats panel that is displayed in the game pause menu.

```
public class CurrentStatsUIController : MonoBehaviour
```

Remarks

Current stats panel displays the current stat value for all 12 modifyable stats in the game. These stats account for modifications the character spawns with and ones modified via passive abilities. Depending on the stat type it will display as an absolute number, absolute difference, or percentage difference.

Methods

HideStatsUI()

```
public void HideStatsUI()
```

ShowStatsUI()

```
public void ShowStatsUI()
```

ShowStatsUIForCharacter(CharacterProperties)

```
public void ShowStatsUIForCharacter(CharacterProperties curCharacterProperties)
```

Parameters

`curCharacterProperties` CharacterProperties

See Also

PlayerStatStatusUIController

Class DS_AttackSystemGroup

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System group containing systems related to player attacks.

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateBefore(typeof(TransformSystemGroup))]
[UpdateBefore(typeof(DS_InteractionSystemGroup))]
public class DS_AttackSystemGroup : ComponentSystemGroup
```

Remarks

Updates in Unity's main SimulationSystemGroup. Must update before Unity's TransformSystemGroup and DS_InteractionSystemGroup as systems in those groups depend on the completion of systems in this group.

Class DS_DestructionSystemGroup

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System group containing systems related to entity destruction.

```
[UpdateInGroup(typeof(SimulationSystemGroup), OrderLast = true)]
[UpdateBefore(typeof(EndSimulationEntityCommandBufferSystem))]
public class DS_DestructionSystemGroup : ComponentSystemGroup
```

Remarks

Updates at the end of Unity's main SimulationSystemGroup to ensure all entities marked for destruction in various earlier systems are accounted for and to ensure entities aren't destroyed prematurely. Updates before Unity's EndSimulationEntityCommandBufferSystem as systems in this group schedule things in that entity command buffer.

Class DS_EffectsSystemGroup

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System group containing systems that deal with triggering visual and audio effects.

```
[UpdateInGroup(typeof(SimulationSystemGroup), OrderLast = true)]
[UpdateBefore(typeof(DS_DestructionSystemGroup))]
public class DS_EffectsSystemGroup : ComponentSystemGroup
```

Remarks

Updates at the end of Unity's main SimulationSystemGroup to ensure any effects that need to be triggered are already known about (typically via enableable component) before executing effects systems. Updates before the DS_DestructionSystemGroup to ensure an entity isn't destroyed before it plays necessary effects for the current frame.

Class DS_InitializationSystemGroup

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System group containing systems related to initialization logic that should be executed towards the beginning of the frame.

```
[UpdateInGroup(typeof(InitializationSystemGroup), OrderLast = true)]
[UpdateBefore(typeof(EndInitializationEntityCommandBufferSystem))]
public class DS_InitializationSystemGroup : ComponentSystemGroup
```

Remarks

Updates at the end of Unity's InitializationSystemGroup to ensure that logic has completed, however it is before the command buffer system at the end of that group in case anything needs to be scheduled in that ECB.

Class DS_InteractionSystemGroup

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System group containing systems related to entity interactions.

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateAfter(typeof(DS_AttackSystemGroup))]
[UpdateAfter(typeof(FixedStepSimulationSystemGroup))]
public class DS_InteractionSystemGroup : ComponentSystemGroup
```

Remarks

Updates in Unity's main SimulationSystemGroup. Must update after Unity's FixedStepSimulationSystemGroup and DS_AttackSystemGroup as systems in those groups raise entity interactions handled by systems in this group.

Class DS_PhysicsSystemGroup

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System group containing systems that deal with physics such as scheduling collision/trigger event jobs and executing physics overlap queries.

```
[UpdateInGroup(typeof(PhysicsSystemGroup))]
[UpdateAfter(typeof(PhysicsSimulationGroup))]
[UpdateBefore(typeof(AfterPhysicsSystemGroup))]
public class DS_PhysicsSystemGroup : ComponentSystemGroup
```

Remarks

Updates in Unity's PhysicsSystemGroup (which is part of Unity's FixedStepSimulationSystemGroup that updates on a fixed tick). Will update after Unity's PhysicsSimulationGroup and before their AfterPhysicsSystemGroup to ensure the appropriate place in the frame for looking at collision/trigger events and overlap queries for the current physics step.

Class DS_TranslationSystemGroup

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System group containing systems that deal with moving entities via their LocalTransform component.

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateBefore(typeof(TransformSystemGroup))]
public class DS_TranslationSystemGroup : ComponentSystemGroup
```

Remarks

Updates in Unity's main SimulationSystemGroup and before their TransformSystemGroup to ensure that any changes to LocalTransform are made before they are transferred to the LocalToWorld component which is used to tell the GPU where to render the entity for the current frame.

Class DamageNumberUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the damage icons that display above enemy characters when taking damage.

```
public class DamageNumberUIController : MonoBehaviour
```

Remarks

Damage text elements are pooled in WorldUICanvasController.

Methods

DisplayDamageNumber(int, bool)

```
public void DisplayDamageNumber(int damageValue, bool isHealing)
```

Parameters

`damageValue` int

`isHealing` bool

Struct DamagePlayerOutsideSafeBoundsSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to apply damage to the player when it is outside the LevelSafeBounds for the level.

```
public struct DamagePlayerOutsideSafeBoundsSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Level safe bounds are identified by a caution tape graphic around the edge of the safe bounds.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct DamagePlayerOutsideSafeBoundsTimestamp

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to hold the timestamp for when the player can be damaged again by the DamagePlayerOutsideSafeBoundsSystem.

```
public struct DamagePlayerOutsideSafeBoundsTimestamp : IComponentData, IQueryTypeParameter
```

Remarks

Component gets added to the DamagePlayerOutsideSafeBoundsSystem SystemHandle entity.

Fields

Value

```
public float Value
```

Field Value

float

Struct DamageThisFrame

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer to store damage accumulated in the current frame.

```
[InternalBufferCapacity(1)]
public struct DamageThisFrame : IBufferElementData
```

Remarks

There are two primary purposes to this approach to handling damage:

1. Damage can be accumulated throughout the duration of the frame from various systems and even parallel jobs without the possibility of overwriting other changes to damage in the current frame.
2. Applying damage to a character takes place in a single system (ProcessDamageThisFrameSystem) which provides a single point for handling all cases related to taking damage, factoring modifiers, etc. rather than having this logic spread out or in multiple systems.

Fields

Value

```
public int Value
```

Field Value

int

Class DamageableEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components required for entities taking damage.

```
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class DamageableEntityAuthoring : MonoBehaviour
```

Remarks

DestructibleEntityAuthoring is a required component as damageable entities should also be destructible.

Fields

HitPoints

Starting number of hit points.

```
public int HitPoints
```

Field Value

int

Remarks

Sets both CurrentHitPoints and BaseHitPoints.

Struct DealHitPointsOnInteraction

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to define the number of hit points to deal when the associated entity interacts with another.

```
public struct DealHitPointsOnInteraction : IComponentData, IQueryTypeParameter
```

Remarks

Typically this component is added to an in-world attack entity and EntityInteractions are raised by trigger events in DetectCapabilityTriggerJob.

Fields

Value

```
public int Value
```

Field Value

int

Class DealHitPointsOnInteractionAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add DealHitPointsOnInteraction component to an entity.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
public class DealHitPointsOnInteractionAuthoring : MonoBehaviour
```

Remarks

Requires the EntityInteractionAuthoring component to ensure EntityInteraction is added to the entity as well.

Fields

HitPoints

Hit points to deal when the associated entity interacts with another entity.

```
public int HitPoints
```

Field Value

int

Struct DealHitPointsOnInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to add damage hit points to the target entity's DamageThisFrame buffer.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public struct DealHitPointsOnInteractionSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Updates in the DS_InteractionSystemGroup to ensure interactions for the current frame have already been added to this entity's EntityInteraction buffer.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class DecorationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to apply the DecorationTag to an entity.

```
public class DecorationAuthoring : MonoBehaviour
```

Struct DecorationTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity is a decoration entity.

```
public struct DecorationTag : IComponentData, IQueryTypeParameter
```

Remarks

Decoration entities are similar to environment entities (EnvironmentTag) however these do not have colliders and do not affect gameplay.

Struct DestroyAfterNumberHits

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Entities with this component will be destroyed when HitsRemaining reaches 0.

```
public struct DestroyAfterNumberHits : IComponentData, IQueryTypeParameter
```

Remarks

Typically used for attack entities that can only hit a max number of entities. For example implementation see SawBladeAttackSystem

Fields

HitsRemaining

Number of hits remaining before entity is destroyed.

```
public int HitsRemaining
```

Field Value

int

See Also

DestroyAfterNumberHitsAuthoring
DestroyAfterNumberHitsSystem

Class DestroyAfterNumberHitsAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize the DestroyAfterNumberHits component.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class DestroyAfterNumberHitsAuthoring : MonoBehaviour
```

Fields

MaxHitCount

Maximum number of hits this entity can have before being destroyed.

```
public int MaxHitCount
```

Field Value

int

See Also

DestroyAfterNumberHits
DestroyEntityFlag

Struct DestroyAfterNumberHitsSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System that determines when the entity has hit the max number of entities. Each hit will decrement Hits Remaining until it reaches zero. Once the counter reaches zero, the entity's DestroyEntityFlag will be enabled.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup), OrderFirst = true)]
public struct DestroyAfterNumberHitsSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system runs at the beginning of the interaction system because if this entity hits multiple entities in a single frame, that puts the hit count OVER the max hit count, additional hits will be ignored by future interactions. i.e. an attack entity won't deal damage to more entities than the max hit count.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

DestroyAfterNumberHits
DestroyAfterNumberHitsAuthoring
DestroyEntityFlag

Struct DestroyAfterTime

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Entities with this component will be destroyed once this time value expires.

```
public struct DestroyAfterTime : IComponentData, IQueryTypeParameter
```

Remarks

Time is tracked in game time, meaning the timer will pause when the game pauses.

Fields

Value

This value will represent the time remaining (in game time) until the entity is destroyed.

```
public float Value
```

Field Value

float

See Also

DestroyAfterTimeSystem
DestroyAfterTimeAuthoring

Class DestroyAfterTimeAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to set the initial value for the DestroyAfterTime component.

```
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class DestroyAfterTimeAuthoring : MonoBehaviour
```

Fields

DestroyAfterTime

Amount of time in seconds the entity should exist in the world.

```
public float DestroyAfterTime
```

Field Value

float

See Also

DestroyAfterTime
DestroyAfterTimeSystem
DestroyEntityFlag

Struct DestroyAfterTimeSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to destroy entities after a given time. This system ticks DestroyAfterTime Value down and will set the DestroyEntityFlag to true once the timer value has reached zero. This will count only in-game time and not be affected by game pause.

```
[UpdateInGroup(typeof(DS_DestructionSystemGroup))]
public struct DestroyAfterTimeSystem : ISystem, ISystemCompilerGenerated
```

Remarks

If an enemy entity is destroyed in this fashion, it should not drop an experience point gem.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

DestroyAfterTime
DestroyAfterTimeAuthoring
DestroyEntityFlag

Struct DestroyAtPosition

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Entities with this component will get destroyed once they reach this world position.

```
public struct DestroyAtPosition : IComponentData, IQueryTypeParameter
```

Fields

LastDistanceSq

Last distance calculated between entity and target position. If this value is decreasing, the entity is still moving towards the target. If this value increases then it means the entity has gone past the TargetPosition and should be destroyed.

```
public float LastDistanceSq
```

Field Value

float

TargetPosition

Target position the entity will be destroyed at.

```
public float3 TargetPosition
```

Field Value

float3

Class DestroyAtPositionAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring component for DestroyAtPosition

```
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class DestroyAtPositionAuthoring : MonoBehaviour
```

Fields

Position

Target world position this entity should be destroyed at.

```
public Vector3 Position
```

Field Value

Vector3

Struct DestroyAtPositionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

This system will set the DestroyEntityFlag to true once the associated entity reaches a target world position. It determines if an entity has reached the position by comparing the current distance to the target with the previous distance. If the current distance is greater than the previous distance, it knows the entity has passed the target position. The distance should be less than two units to reduce the possibility of false positives. This method of determining if an entity has reached a target position is to ensure fast moving entities that aren't guaranteed to be within a small tolerance of the target position are accounted for.

```
[UpdateInGroup(typeof(DS_DestructionSystemGroup))]
public struct DestroyAtPositionSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct DestroyEntityFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

An entity with this component is capable of being destroyed by various systems throughout the game. This component should be disabled by default and once enabled it will be destroyed at the end of the frame in the DestroyEntitySystem

```
public struct DestroyEntityFlag : IComponentData, IQueryTypeParameter, IEnableableComponent
```

See Also

DestroyEntitySystem
DestructibleEntityAuthoring

Struct DestroyEntitySystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

This system is responsible for destroying all entities with the DestroyEntityFlag enabled. There are various checks to determine if any additional logic should be executed when the entity is destroyed. This system should be the last system to execute in the DS_DestructionSystemGroup, which itself updates at the end of the SimulationSystemGroup. The only system to run after this system in the SimulationSystemGroup should be the EndSimulationEntityCommandBufferSystem.

```
[UpdateInGroup(typeof(DS_DestructionSystemGroup), OrderLast = true)]
public struct DestroyEntitySystem : ISystem, ISystemCompilerGenerated
```

Remarks

Examples of additional logic on destroy includes spawning experience gems, playing destruction VFX and SFX, and triggering game over logic.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

```
state SystemState
```

Class DestroyOffCameraAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize values for DestroyOffCameraData

```
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class DestroyOffCameraAuthoring : MonoBehaviour
```

Fields

Direction

Byte flag for defining which directions off screen should be tested when determining if the entity should be destroyed.

```
public CameraDirection Direction
```

Field Value

CameraDirection

Remarks

Example - if Up bit is not set, but down bit is set, then the entity will not be destroyed if the entity goes upwards offscreen, but will be destroyed if the entity goes downwards offscreen.

See Also
CameraDirection

Padding

Additional padding applied to testing if the entity is offscreen. Used so that the entity isn't destroyed when just its origin is offscreen, but some graphics are still on screen.

```
public float Padding
```

Field Value

float

See Also

DestroyOffCameraSystem
DestroyOffCameraData

Struct DestroyOffCameraData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Entities with this component will be destroyed when the entity has left the camera bounds on one (or more) of the specified sides.

```
public struct DestroyOffCameraData : IComponentData, IQueryTypeParameter
```

Fields

Direction

Byte flag for defining which directions offscreen should be tested when determining if the entity should be destroyed.

```
public CameraDirection Direction
```

Field Value

CameraDirection

Remarks

Example - if Up bit is not set, but down bit is set, then the entity will not be destroyed if the entity goes upwards offscreen, but will be destroyed if the entity goes downwards offscreen.

See Also
CameraDirection

Padding

Additional padding applied to testing if the entity is offscreen. An entity will be considered offscreen when its world transform position is outside the screen bounds plus the padding value.

```
public float Padding
```

Field Value

float

Remarks

Used so that the entity isn't destroyed when just its origin is offscreen, but some graphics are still on screen.

See Also

DestroyOffCameraSystem
DestroyOffCameraAuthoring
CameraDirection

Struct DestroyOffCameraSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System that will enable the to destroy the entity at the end of the frame if the entity is offscreen. Direction can be used to specify the directions off camera the entity will be cleaned up.

```
[UpdateInGroup(typeof(DS_DestructionSystemGroup))]
public struct DestroyOffCameraSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Example - if Up bit is not set, but down bit is set, then the entity will not be destroyed if the entity goes upwards offscreen, but will be destroyed if the entity goes downwards offscreen.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

DestroyOffCameraData
DestroyOffCameraAuthoring

Class DestroyOnEnvironmentContactAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script that adds DestroyOnEnvironmentContactTag component to the entity.

```
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class DestroyOnEnvironmentContactAuthoring : MonoBehaviour
```

See Also

DestroyOnEnvironmentContactSystem
DestroyOnEnvironmentContactTag
EnvironmentTag

Struct DestroyOnEnvironmentContactSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System that schedules the DestroyOnEnvironmentContactTriggerJob to destroy an entity upon contact with an environment entity.

```
[UpdateInGroup(typeof(DS_PhysicsSystemGroup))]
public struct DestroyOnEnvironmentContactSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

DestroyOnEnvironmentContactTag
DestroyOnEnvironmentContactTriggerJob
EnvironmentTag

Struct DestroyOnEnvironmentContactTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Entities with this component will be destroyed upon contact with an environment entity.

```
public struct DestroyOnEnvironmentContactTag : IComponentData, IQueryTypeParameter
```

See Also

DestroyOnEnvironmentContactSystem
DestroyOnEnvironmentContactAuthoring
EnvironmentTag

Struct DestroyOnEnvironmentContactTriggerJob

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Trigger Event Job scheduled by the DestroyOnEnvironmentContactSystem. Enables the DestroyEntityFlag component on an entity tagged with the DestroyOnEnvironmentContactTag component that comes in contact with an entity with an EnvironmentTag component.

```
[BurstCompile]
public struct DestroyOnEnvironmentContactTriggerJob : ITriggerEventsJob,
ITriggerEventsJobBase
```

Remarks

This is an ITriggerEventsJob as entities that match the query (typically player attacks) raise trigger events, not collision events.

Fields

DestroyEntityLookup

```
public ComponentLookup<DestroyEntityFlag> DestroyEntityLookup
```

Field Value

ComponentLookup<DestroyEntityFlag>

DestroyOnEnvironmentLookup

```
[ReadOnly]
public ComponentLookup<DestroyOnEnvironmentContactTag> DestroyOnEnvironmentLookup
```

Field Value

ComponentLookup<DestroyOnEnvironmentContactTag>

EnvironmentLookup

```
[ReadOnly]
public ComponentLookup<EnvironmentTag> EnvironmentLookup
```

Field Value

ComponentLookup<EnvironmentTag>

Methods

Execute(TriggerEvent)

```
[BurstCompile]
public void Execute(TriggerEvent triggerEvent)
```

Parameters

`triggerEvent` TriggerEvent

See Also

DestroyOnEnvironmentContactSystem
DestroyOnEnvironmentContactTag
EnvironmentTag

Struct DestroyPreviousWaveEnemySystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to destroy off-screen enemies that were spawned in a previous wave.

```
[UpdateInGroup(typeof(DS_DestructionSystemGroup))]
public struct DestroyPreviousWaveEnemySystem : ISystem, ISystemCompilerGenerated
```

Remarks

This is used to ensure a fresh set of enemies are regularly spawned.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class DestroySelfOnInteractionAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add the DestroySelfOnInteractionTag component to an entity.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class DestroySelfOnInteractionAuthoring : MonoBehaviour
```

See Also

DestroySelfOnInteractionTag
DestroySelfOnInteractionSystem

Struct DestroySelfOnInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System that enables the DestroyEntityFlag on an entity tagged with the DestroySelfOnInteractionTag when an interaction is raised.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public struct DestroySelfOnInteractionSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

DestroySelfOnInteractionTag
DestroySelfOnInteractionAuthoring
EntityInteraction

Struct DestroySelfOnInteractionTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Entities with this component will self-destruct upon raising an interaction.

```
public struct DestroySelfOnInteractionTag : IComponentData, IQueryTypeParameter
```

Remarks

This is typically used for attractable items that need to get destroyed when they come in contact with the player.

See Also

DestroySelfOnInteractionSystem
DestroySelfOnInteractionAuthoring

Class DestructibleEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring component to add DestroyEntityFlag to an entity. This component will be disabled by default to ensure the entity isn't immediately destroyed after instantiation.

```
public class DestructibleEntityAuthoring : MonoBehaviour
```

See Also

DestroyEntitySystem
DestroyEntityFlag

Struct DetectCapabilityTriggerJob

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Job to detect trigger events between capabilities and damageable entities.

```
[BurstCompile]
public struct DetectCapabilityTriggerJob : ITriggerEventsJob, ITriggerEventsJobBase
```

Remarks

This is typically used to detect collisions between player attack entities and enemy entities. If true, the damageable entity will be added to the EntityInteraction buffer on the entity that will apply damage.

Fields

DamageableEntityLookup

```
[ReadOnly]
public ComponentLookup<CurrentHitPoints> DamageableEntityLookup
```

Field Value

ComponentLookup<CurrentHitPoints>

InteractionBufferLookup

```
public BufferLookup<EntityInteraction> InteractionBufferLookup
```

Field Value

BufferLookup<EntityInteraction>

Methods

Execute(TriggerEvent)

```
[BurstCompile]
public void Execute(TriggerEvent triggerEvent)
```

Parameters

`triggerEvent` TriggerEvent

Struct DetectCapabilityTriggerSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to schedule the DetectCapabilityTriggerJob.

```
[UpdateInGroup(typeof(DS_PhysicsSystemGroup))]
public struct DetectCapabilityTriggerSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Only execute this system if the player entity exists. This avoids damaging enemies in game over state to potentially inaccurately account for number of enemies defeated during the game.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct DetectItemAttractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to detect attractable items.

```
[UpdateInGroup(typeof(DS_PhysicsSystemGroup))]
public struct DetectItemAttractionSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Uses the PlayerAttractionAreaData to execute an OverlapSphere method to locate any attractable items within the attraction radius. When an item is within radius, components related to moving the item towards the player are initialized.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

AttractableItemSystem

Struct DissolveData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component data relevant to dissolve visual effect for characters upon destruction

```
public struct DissolveData : IComponentData, IQueryTypeParameter
```

Fields

Duration

```
public float Duration
```

Field Value

float

StartTimestamp

```
public float StartTimestamp
```

Field Value

float

Struct DissolveOverride

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Material property override for the dissolve property of the character. Setting this value will update the _Dissolve property in the material instance of this entity. This will make the character sprite appear to dissolve upon destruction.

```
[MaterialProperty("_Dissolve", -1)]
public struct DissolveOverride : IComponentData, IQueryTypeParameter
```

Remarks

Your IDE may gray out the Value field as this value is not used in our code. However, Unity uses it to apply this value to the material property defined in the MaterialProperty attribute. Be sure the MaterialProperty string exactly matches the reference string defined in the shader as this will silently fail if there is a typo.

Fields

Value

```
public float Value
```

Field Value

float

See Also

DissolveData
CharacterAnimationSystem

Struct DroneAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of drones into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct DroneAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

All drones are spawned at once in a circular pattern around the player under the main weapon entity. This weapon entity will rotate its transform, thus rotating the individual drones around the player. As the drone weapon entity is a child of the player, the drones will follow the player's movement for as long as they are active. Drones are affected by modifications to the player's additional attack projectiles, damage dealt, attack duration, and attack projectile speed stat modifications. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem. This system is unique in that the attack duration time is added to the cooldown time so that the effective cooldown doesn't begin until the attack ends.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class DroneEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for drone in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class DroneEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime
ConstantRotationData

Class DroneWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add DroneWeaponTag to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
public class DroneWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity.

Struct DroneWeaponTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity as the weapon entity to spawn drones into the game world.

```
public struct DroneWeaponTag : IComponentData, IQueryTypeParameter
```

Struct DropExperienceOnDestroy

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Entities with this component have a chance to drop an experience point of a specified value when the entity is destroyed via the DestroyEntitySystem.

```
public struct DropExperienceOnDestroy : IComponentData, IQueryTypeParameter
```

Remarks

This could be implemented as a cleanup component to avoid having the logic in DestroyEntitySystem, however I chose to have the logic held there so that we can also have an InstantDestroyEntitySystem which does not spawn experience gems. That implementation would get messy with this as a cleanup component. -Johnny

Fields

ChanceToDrop

Value from 0 to 100 representing the percentage chance the destroyed entity will drop an experience point gem. 0 will never drop a gem and 100 will always drop a gem.

```
public int ChanceToDrop
```

Field Value

int

ExperienceValue

The number of experience points assigned to the gem that will be dropped when this entity is destroyed.

```
public int ExperienceValue
```

Field Value

int

See Also

ExperienceGemItemData

See Also

DropExperienceOnDestroyAuthoring

DestroyEntitySystem

Class DropExperienceOnDestroyAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add the DropExperienceOnDestroy to an entity.

```
public class DropExperienceOnDestroyAuthoring : MonoBehaviour
```

Remarks

This is typically added to enemies who will drop experience point gems when they are destroyed.

Fields

ChanceToDrop

Value from 0 to 100 representing the percentage chance the destroyed entity will drop an experience point gem. 0 will never drop a gem and 100 will always drop a gem.

```
[Range(0, 100)]
public int ChanceToDrop
```

Field Value

int

ExperienceValue

The number of experience points assigned to the gem that will be dropped when this entity is destroyed.

```
public int ExperienceValue
```

Field Value

int

See Also

ExperienceGemItemData

See Also

DropExperienceOnDestroy
DestroyEntitySystem
ExperienceGemItemData

Class EnemyAnimationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for enemy character animation.

```
[RequireComponent(typeof(CharacterAnimationAuthoring))]
public class EnemyAnimationAuthoring : MonoBehaviour
```

Remarks

This authoring script should be attached to the graphics entity which is a child of the main enemy entity. CharacterAnimationAuthoring is a required component to ensure components necessary for character animation are applied to the entity.

Struct EnemyAttackCollisionJob

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Collision Event Job scheduled by EnemyAttackSystem. This job is ran on all collision events raised by the physics system for this physics step. The Execute method will first determine which of the entities in the collision event is the player entity and which is the enemy entity. If the collision event is not between exactly 1 player and 1 enemy, the job will early out. Next the job will determine if the enemy is on cooldown, if the enemy is not on cooldown, it will apply damage to the player via the DamageThisFrame Dynamic Buffer and reset the EnemyAttackCooldownExpiration for the enemy.

```
[BurstCompile]
public struct EnemyAttackCollisionJob : ICollisionEventsJob, ICollisionEventsJobBase
```

Fields

CharacterStatsLookup

Component lookup used to determine if any additional damage scaling should be added to the enemy's attack.

```
[ReadOnly]
public ComponentLookup<CharacterStatModificationState> CharacterStatsLookup
```

Field Value

ComponentLookup<CharacterStatModificationState>

CooldownExpirationLookup

Read/Write component lookup used to determine if the enemy is on cooldown. Write access is also required so the expiration timer can be updated after the enemy attacks.

```
public ComponentLookup<EnemyAttackCooldownExpiration> CooldownExpirationLookup
```

Field Value

ComponentLookup<EnemyAttackCooldownExpiration>

DamageThisFrameLookup

Buffer lookup used to apply points of damage to the player.

```
public BufferLookup<DamageThisFrame> DamageThisFrameLookup
```

Field Value

BufferLookup<DamageThisFrame>

ElapsedTime

Elapsed game time used to determine if an enemy is on cooldown.

```
public double ElapsedTime
```

Field Value

double

EnemyAttackPropertiesLookup

Component lookup used to get data for the enemy attack.

```
[ReadOnly]
public ComponentLookup<EnemyAttackData> EnemyAttackPropertiesLookup
```

Field Value

ComponentLookup<EnemyAttackData>

EnemyLookup

Component lookup used to determine the enemy entity.

```
[ReadOnly]
public ComponentLookup<EnemyTag> EnemyLookup
```

Field Value

ComponentLookup<EnemyTag>

PlayerLookup

Component lookup used to determine the player entity.

```
[ReadOnly]
public ComponentLookup<PlayerTag> PlayerLookup
```

Field Value

ComponentLookup<PlayerTag>

Methods

Execute(CollisionEvent)

```
[BurstCompile]
public void Execute(CollisionEvent collisionEvent)
```

Parameters

`collisionEvent` CollisionEvent

Struct EnemyAttackCooldownExpiration

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to store the timestamp of when the enemy attack cooldown expires. If the game time is greater than or equal to this value, the enemy is able to attack.

```
public struct EnemyAttackCooldownExpiration : IComponentData, IQueryTypeParameter
```

Fields

Value

After the enemy attacks, this value will be set to current game time + CooldownTime

```
public float Value
```

Field Value

float

See Also

EnemyAttackSystem
EnemyAttackData

Struct EnemyAttackData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component containing data related to enemy attacking.

```
public struct EnemyAttackData : IComponentData, IQueryTypeParameter
```

Fields

CooldownTime

Time in seconds defining the cooldown time of which an enemy cannot deal damage after an attack. This value will remain unchanged through the lifetime of the enemy. See the EnemyAttackCooldownExpiration component which contains the timestamp at which the enemy is able to attack again.

```
public float CooldownTime
```

Field Value

float

HitPoints

Number of points of damage to add during a single attack.

```
public int HitPoints
```

Field Value

int

See Also

EnemyAttackSystem

EnemyAttackCooldownExpiration

Struct EnemyAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System that schedules EnemyAttackCollisionJob which detects collisions between enemies and the player and deals damage to the player. Updates in the DS_PhysicsSystemGroup so collision events for this frame have been raised.

```
[UpdateInGroup(typeof(DS_PhysicsSystemGroup))]
public struct EnemyAttackSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

EnemyAttackCollisionJob

Class EnemyAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to set initial values for enemies.

```
[RequireComponent(typeof(CharacterAuthoring))]
[RequireComponent(typeof(DropExperienceOnDestroyAuthoring))]
[RequireComponent(typeof(ShowDamageNumberOnDamageAuthoring))]
public class EnemyAuthoring : MonoBehaviour
```

Fields

BaseAttackDamage

```
public int BaseAttackDamage
```

Field Value

int

EnemyAttackCooldown

```
public float EnemyAttackCooldown
```

Field Value

float

EnemyType

```
public EnemyType EnemyType
```

Field Value

EnemyType

HealthScalesWithPlayerLevel

```
public bool HealthScalesWithPlayerLevel
```

Field Value

bool

See Also

EnemyAttackData
EnemyAttackCooldownExpiration
EnemyTypeInitialization
EnemyHitPointsScaleWithPlayerLevelTag

Class EnemyBlackHoleResistAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring component to add EnemyBlackHoleResistTag to an enemy.

```
public class EnemyBlackHoleResistAuthoring : MonoBehaviour
```

See Also

EnemyBlackHoleResistTag

Struct EnemyBlackHoleResistTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enemies tagged with this component are resistant to black holes. Currently only used on ending reaper enemy.

```
public struct EnemyBlackHoleResistTag : IComponentData, IQueryTypeParameter
```

Struct EnemyHitPointsScaleWithPlayerLevelTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to identify this entity as an enemy that should scale its health with the player level. Level scaling is defined by Enemy Base Hit Points multiplied by Player Level.

```
public struct EnemyHitPointsScaleWithPlayerLevelTag : IComponentData, IQueryTypeParameter
```

Remarks

i.e. If the enemy has 10 base hit points and the player is on level 3, the enemy will spawn with 30 hit points of health.

Struct EnemyInitializationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to control initialization steps pertaining to enemy entities.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
[UpdateBefore(typeof(CharacterInitializationSystem))]
public struct EnemyInitializationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Currently the only initialization step is to perform hit point scaling on enemies tagged with the EnemyHitPointsScaleWithPlayerLevelTag. System updates before the CharacterInitializationSystem as the InitializeCharacterFlag gets disabled in that system.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct EnemyLinearMovement

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component defining the angle in radians the player will move.

```
public struct EnemyLinearMovement : IComponentData, IQueryTypeParameter
```

Fields

Angle

Angle in radians of direction of travel.

```
public float Angle
```

Field Value

float

Remarks

Degree 0 will move enemy from bottom to top of screen, degree 90 will move enemy from left of right of screen.

See Also

SetEnemyLinearMovementFlag
EnemyLinearMovementSystem
EnemyLinearMovementAuthoring

Class EnemyLinearMovementAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize the angle of travel for EnemyLinearMovement. Sets SetEnemyLinearMovementFlag to true to initialize CharacterMoveDirection.

```
public class EnemyLinearMovementAuthoring : MonoBehaviour
```

Remarks

Value authored in degrees for user-friendly editing. Stored in radians for efficient math calculations.

Fields

Angle

Angle in degrees for initial direction of travel.

```
public float Angle
```

Field Value

float

Remarks

Degree 0 will move enemy from bottom to top of screen, degree 90 will move enemy from left of right of screen.

See Also

EnemyLinearMovement
SetEnemyLinearMovementFlag
EnemyLinearMovementSystem

Struct EnemyLinearMovementSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to set CharacterMoveDirection based off angle stored in EnemyLinearMovement when SetEnemyLinearMovementFlag is enabled for an entity.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct EnemyLinearMovementSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

EnemyLinearMovement
SetEnemyLinearMovementFlag
EnemyLinearMovementAuthoring

Class EnemyMoveToPlayerAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add EnemyMoveToPlayerTag to entity.

```
public class EnemyMoveToPlayerAuthoring : MonoBehaviour
```

See Also

EnemyMoveToPlayerTag
EnemyMoveToPlayerSystem
EnemyMoveToPlayerJob
CharacterMoveSystem

Struct EnemyMoveToPlayerJob

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Job to calculate the direction from the enemy to the player and set the CharacterMoveDirection to move the enemy towards the player.

```
[BurstCompile]
[WithAll(new Type[] { typeof(EnemyTag), typeof(EnemyMoveToPlayerTag) })]
public struct EnemyMoveToPlayerJob : IJobEntity, IJobChunk
```

Fields

PlayerPosition

```
public float3 PlayerPosition
```

Field Value

float3

Methods

Execute(in ArchetypeChunk, int, bool, in v128)

```
public void Execute(in ArchetypeChunk chunk, int chunkIndexInQuery, bool useEnabledMask, in
v128 chunkEnabledMask)
```

Parameters

`chunk` ArchetypeChunk

`chunkIndexInQuery` int

`useEnabledMask` bool

`chunkEnabledMask` v128

Run()

```
public void Run()
```

Run(EntityQuery)

```
public void Run(EntityQuery query)
```

Parameters

`query` EntityQuery

RunByRef()

```
public void RunByRef()
```

RunByRef(EntityQuery)

```
public void RunByRef(EntityQuery query)
```

Parameters

`query` EntityQuery

Schedule()

```
public void Schedule()
```

Schedule(EntityQuery)

```
public void Schedule(EntityQuery query)
```

Parameters

`query` EntityQuery

Schedule(EntityQuery, JobHandle)

```
public JobHandle Schedule(EntityQuery query, JobHandle dependsOn)
```

Parameters

`query` EntityQuery

`dependsOn` JobHandle

Returns

JobHandle

Schedule(JobHandle)

```
public JobHandle Schedule(JobHandle dependsOn)
```

Parameters

`dependsOn` JobHandle

Returns

JobHandle

ScheduleByRef()

```
public void ScheduleByRef()
```

ScheduleByRef(EntityQuery)

```
public void ScheduleByRef(EntityQuery query)
```

Parameters

`query` EntityQuery

ScheduleByRef(EntityQuery, JobHandle)

```
public JobHandle ScheduleByRef(EntityQuery query, JobHandle dependsOn)
```

Parameters

`query` EntityQuery

`dependsOn` JobHandle

Returns

JobHandle

ScheduleByRef(JobHandle)

```
public JobHandle ScheduleByRef(JobHandle dependsOn)
```

Parameters

`dependsOn` JobHandle

Returns

JobHandle

ScheduleParallel()

```
public void ScheduleParallel()
```

ScheduleParallel(EntityQuery)

```
public void ScheduleParallel(EntityQuery query)
```

Parameters

`query` EntityQuery

ScheduleParallel(EntityQuery, JobHandle)

```
public JobHandle ScheduleParallel(EntityQuery query, JobHandle dependsOn)
```

Parameters

`query` EntityQuery

`dependsOn` JobHandle

Returns

JobHandle

ScheduleParallel(EntityQuery, JobHandle, NativeArray<int>)

```
public JobHandle ScheduleParallel(EntityQuery query, JobHandle dependsOn,
NativeArray<int> chunkBaseEntityIndices)
```

Parameters

`query` EntityQuery

`dependsOn` JobHandle

`chunkBaseEntityIndices` NativeArray<int>

Returns

JobHandle

ScheduleParallel(JobHandle)

```
public JobHandle ScheduleParallel(JobHandle dependsOn)
```

Parameters

`dependsOn` JobHandle

Returns

JobHandle

ScheduleParallelByRef()

```
public void ScheduleParallelByRef()
```

ScheduleParallelByRef(EntityQuery)

```
public void ScheduleParallelByRef(EntityQuery query)
```

Parameters

`query` EntityQuery

ScheduleParallelByRef(EntityQuery, JobHandle)

```
public JobHandle ScheduleParallelByRef(EntityQuery query, JobHandle dependsOn)
```

Parameters

`query` EntityQuery

`dependsOn` JobHandle

Returns

JobHandle

ScheduleParallelByRef(EntityQuery, JobHandle, NativeArray<int>)

```
public JobHandle ScheduleParallelByRef(EntityQuery query, JobHandle dependsOn,
NativeArray<int> chunkBaseEntityIndices)
```

Parameters

`query` EntityQuery

`dependsOn` JobHandle

`chunkBaseEntityIndices` NativeArray<int>

Returns

JobHandle

ScheduleParallelByRef(JobHandle)

```
public JobHandle ScheduleParallelByRef(JobHandle dependsOn)
```

Parameters

`dependsOn` JobHandle

Returns

JobHandle

See Also

EnemyMoveToPlayerSystem
EnemyMoveToPlayerTag
CharacterMoveSystem

Struct EnemyMoveToPlayerSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to schedule to EnemyMoveToPlayerJob which constantly updates the enemies' CharacterMoveDirection to move the enemy towards the player.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct EnemyMoveToPlayerSystem : ISystem, ISystemStartStop, ISystemCompilerGenerated
```

Methods

OnCreate(ref SystemState)

Only schedule the EnemyMoveToPlayerJob when the player exists.

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

Remarks

This is important for two reasons:

1. Without the player entity, the job will be unable to find the player's position
2. When the player entity is destroyed, we can trigger logic to stop enemies in the OnStopRunning(ref SystemState) method.

OnStartRunning(ref SystemState)

Empty method required for ISystemStartStop implementation.

```
public void OnStartRunning(ref SystemState state)
```

Parameters

`state` SystemState

OnStopRunning(ref SystemState)

When Player no longer exists, stop movement.

```
[BurstCompile]
public void OnStopRunning(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

EnemyMoveToPlayerJob
EnemyMoveToPlayerTag
CharacterMoveSystem

Struct EnemyMoveToPlayerTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enemies tagged with this component will constantly update their CharacterMoveDirection to move towards the player's current position.

```
public struct EnemyMoveToPlayerTag : IComponentData, IQueryTypeParameter
```

See Also

EnemyMoveToPlayerAuthoring
EnemyMoveToPlayerSystem
EnemyMoveToPlayerJob
CharacterMoveSystem

Struct EnemySineWaveMovement

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component defining the movement properties for enemy sine wave movement.

```
public struct EnemySineWaveMovement : IComponentData, IQueryTypeParameter
```

Fields

Amplitude

Amplitude of sine wave - height from bottom valley to top peak of sine wave.

```
public float Amplitude
```

Field Value

float

ConstantMoveSpeed

Speed it will move along the axis of constant movement.

```
public float ConstantMoveSpeed
```

Field Value

float

MoveDirection

Cardinal direction specifying the direction of constant movement of the enemy.

```
public SineWaveMoveDirection MoveDirection
```

Field Value

SineWaveMoveDirection

Period

Period of sine wave - units between peaks of waves. Essentially determines how fast it will move back and forth, perpendicular to direction of constant movement.

```
public float Period
```

Field Value

float

Class EnemySineWaveMovementAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize values for EnemySineWaveMovement.

```
public class EnemySineWaveMovementAuthoring : MonoBehaviour
```

Fields

Amplitude

Amplitude of sine wave - height from bottom valley to top peak of sine wave.

```
public float Amplitude
```

Field Value

float

ConstantMoveSpeed

Speed it will move along the axis of constant movement.

```
public float ConstantMoveSpeed
```

Field Value

float

MoveDirection

Cardinal direction specifying the direction of constant movement of the enemy.

```
public SineWaveMoveDirection MoveDirection
```

Field Value

SineWaveMoveDirection

Period

Period of sine wave - units between peaks of waves. Essentially determines how fast it will move back and forth, perpendicular to direction of constant movement.

```
public float Period
```

Field Value

float

Struct EnemySineWaveMovementSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to set the CharacterMoveDirection for an enemy entity based on values calculated using EnemySineWaveMovement.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct EnemySineWaveMovementSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system will also add an EnemySineWaveStartTime component with the current game time value to any entity that has an EnemySineWaveMovement component but no EnemySineWaveStartTime component.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

CharacterMoveSystem
EnemySineWaveMovement
EnemySineWaveStartTime

Struct EnemySineWaveStartTime

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component containing the start time of the sine wave so the enemy can move along the wave over time.

```
public struct EnemySineWaveStartTime : IComponentData, IQueryTypeParameter
```

Remarks

Typically this value will be initialized to the time at which the enemy was spawned and time alive can be calculated from this value.

Fields

Value

Timestamp at which the current time alive value should be calculated from.

```
public float Value
```

Field Value

float

Class EnemySpawnAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to define behavior of enemy spawning for a particular level.

```
[RequireComponent(typeof(EntityRandomAuthoring))]
public class EnemySpawnAuthoring : MonoBehaviour
```

Remarks

Requires the EntityRandomAuthoring component to add EntityRandom component to the entity for spawn randomization. It is important that we include DependsOn(authoring.StageProperties) so baking is re-ran when values on the ScriptableObject are changed.

Fields

MaxSpawnDistanceFromCameraBounds

Enemies will be spawned offscreen at a random distance from the edge of the camera bounds. This value defines the maximum distance away from the camera bounds an enemy will spawn.

```
public float MaxSpawnDistanceFromCameraBounds
```

Field Value

float

MinSpawnDistanceFromCameraBounds

Enemies will be spawned offscreen at a random distance from the edge of the camera bounds. This value defines the minimum distance away from the camera bounds an enemy will spawn.

```
public float MinSpawnDistanceFromCameraBounds
```

Field Value

float

StageProperties

ScriptableObject containing information relevant to the current stage, notably the interval between spawn waves and array of EnemySpawnWaveProperties

```
public StageProperties StageProperties
```

Field Value

StageProperties

Remarks

It is important that we include DependsOn(authoring.StageProperties) so baking is re-ran when values on the ScriptableObject are changed.

Struct EnemySpawnData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data pertaining to enemy spawning. This data is specific to a level and will remain unchanged through gameplay of a specific level.

```
public struct EnemySpawnData : IComponentData, IQueryTypeParameter
```

Fields

FinalBossType

Final boss is the boss that will be spawned after all waves complete.

```
public EnemyType FinalBossType
```

Field Value

EnemyType

Remarks

The final boss spawns with a very high health and damage value, making it essentially impossible to destroy and will immediately destroy the player on first contact. Before the final boss is spawned, all other enemies in the world will be destroyed.

MaxSpawnDistanceFromCameraBounds

Enemies will be spawned offscreen at a random distance from the edge of the camera bounds. This value defines the maximum distance away from the camera bounds an enemy will spawn.

```
public float MaxSpawnDistanceFromCameraBounds
```

Field Value

float

MinSpawnDistanceFromCameraBounds

Enemies will be spawned offscreen at a random distance from the edge of the camera bounds. This value defines the minimum distance away from the camera bounds an enemy will spawn.

```
public float MinSpawnDistanceFromCameraBounds
```

Field Value

float

WaveInterval

Time in seconds for how long the game will be spawning enemies of a particular spawn wave.

```
public float WaveInterval
```

Field Value

float

Methods

GetRandomSpawnPos(RefRW<EntityRandom>, float3, float3)

Helper function to get a random spawn position offscreen.

```
public float3 GetRandomSpawnPos(RefRW<EntityRandom> random, float3 cameraPosition,
float3 cameraHalfExtents)
```

Parameters

`random` RefRW<EntityRandom>

Used for generating the random position.

`cameraPosition` float3

Center point of the camera.

`cameraHalfExtents` float3

Half dimensions of the camera view space.

Returns

float3

See Also

EnemySpawnSystem

Class EnemySpawnEventProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring for spawn events. Spawn event are certain events that have a chance of occurrence within a spawn wave. When a spawn event happens, enemies will be spawned in a certain SpawnFormation and have unique behavior.

```
[CreateAssetMenu(fileName = "SpawnEvent-", menuName = "ScriptableObjects/Spawn Event")]
public class EnemySpawnEventProperties : ScriptableObject
```

Remarks

Certain SpawnFormations require additional data. Not all fields will be used for each formation Custom editor script displays relevant fields for the selected formation (SpawnEventPropertiesEditor.cs)

Fields

Amplitude

Optional field relevant to certain spawn formations. Defines the Amplitude for enemy sine wave movement. EnemySineWaveMovement

```
public float Amplitude
```

Field Value

float

EnemyCount

Number of enemies to be spawned during this event.

```
public int EnemyCount
```

Field Value

int

EnemySpacing

Optional field relevant to certain spawn formations. Defines the spacing between enemies.

```
public float EnemySpacing
```

Field Value

float

EnemyType

Type of enemy to be spawned for this event.

```
public EnemyType EnemyType
```

Field Value

EnemyType

MoveSpeed

Optional field relevant to certain spawn formations. Defines the movement speed for linear moving enemies (EnemyLinearMovement) and constant move speed for sine wave enemies (EnemySineWave Movement).

```
public float MoveSpeed
```

Field Value

float

Period

Optional field relevant to certain spawn formations. Defines the period for enemy sine wave movement. EnemySineWaveMovement

```
public float Period
```

Field Value

float

SpawnFormation

Arrangement the enemies will be spawned in.

```
public SpawnFormation SpawnFormation
```

Field Value

SpawnFormation

TimeToLive

Optional field relevant to certain spawn formations. Defines the time to live for a given enemy before it is destroyed. This field can also be used to store the test interval for the DestroyOffCameraData component.

```
public float TimeToLive
```

Field Value

float

See Also

EnemySpawnWaveProperties
EnemySpawnSystem
SpawnEvent

Struct EnemySpawnSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to spawn enemies as defined by spawn waves and spawn events.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup), OrderFirst = true)]
[UpdateBefore(typeof(CharacterInitializationSystem))]
public struct EnemySpawnSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System updates at the beginning of the DS_InitializationSystemGroup and before CharacterInitialization System to ensure all initialization logic is executed for new enemies.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct EnemySpawnTimer

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Timer to inform the EnemySpawnSystem when to spawn the next batch of enemies for a given wave.

```
public struct EnemySpawnTimer : IComponentData, IQueryTypeParameter
```

Fields

Value

Timer value is initialized to SpawnInterval, each frame the value is decremented by delta time, once the timer reaches 0, the next batch of enemies for the wave will be spawned.

```
public float Value
```

Field Value

float

Struct EnemySpawnWave

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer component to hold all the EnemySpawnWaveProperties ScriptableObjects for a given stage.

```
public struct EnemySpawnWave : IBufferElementData
```

Remarks

In the EnemySpawnSystem when a new wave is to be selected, the next element of the buffer is referenced and appropriate data components are populated with runtime friendly, unmanaged data. Index is tracked in EnemySpawnerState.

Fields

Value

```
public UnityObjectRef<EnemySpawnWaveProperties> Value
```

Field Value

UnityObjectRef<EnemySpawnWaveProperties>

Struct EnemySpawnWaveData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data pertaining to the currently active spawn wave. This data will be updated at the change of each spawn wave.

```
public struct EnemySpawnWaveData : IComponentData, IQueryTypeParameter
```

Fields

MinEnemyCount

Defines the minimum number of enemies that will be present while this wave is active. If there are fewer than the minimum enemies in the world for a particular spawn wave, the number of enemies required to meet this minimum value will be spawned in 1 frame.

```
public int MinEnemyCount
```

Field Value

int

SpawnInterval

If the minimum enemy count is met for this wave, additional enemies will be spawned on an interval defined by this value. One enemy of each type stored in the SpawnWaveEnemyPrefab Dynamic Buffer will be spawned once the WaveTimer reaches 0.

```
public float SpawnInterval
```

Field Value

float

See Also

EnemySpawnSystem
EnemySpawnerState

Class EnemySpawnWaveProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject used to author information about spawn waves.

```
[CreateAssetMenu(fileName = "EnemySpawnWave-", menuName = "Enemy Spawn Wave")]
public class EnemySpawnWaveProperties : ScriptableObject
```

Remarks

A spawn wave defines the behavior of enemy spawning for a certain period of time (typically 1 minute) in a level.

Fields

BossBaseHitPoints

Base hit points to be applied to the boss for this spawn wave. Boss hit points are multiplied by the player's current level for a small bit of scaling.

```
public int BossBaseHitPoints
```

Field Value

int

BossChanceToDropCrate

Percentage chance for the boss to spawn a crate once defeated. 0 will never spawn a crate and 100 will always spawn a crate.

```
[Range(0, 100)]
public int BossChanceToDropCrate
```

Field Value

int

BossExperiencePoints

Value of the experience point gem dropped by the boss for this wave.

```
public int BossExperiencePoints
```

Field Value

int

BossType

Boss enemy type for the current spawn wave.

```
public EnemyType BossType
```

Field Value

EnemyType

Remarks

Boss enemies have more health than normal, have an enhanced material effect, and have a chance to spawn a supply crate when defeated. There can only be 1 boss enemy per wave. Boss enemies always spawn at the start of a spawn wave.

ChanceOfOccurrence

Percentage chance each spawn event will occur. Each spawn event will be evaluated at its execution time to determine if it should occur or not. A value of 0 will never occur and a value of 100 will always occur.

```
[Range(0, 100)]
[Tooltip("Percent chance each event has to occur")]
```

```
public int ChanceOfOccurrence
```

Field Value

int

DelayTime

Time in seconds from the start of the wave until the first spawn event. Subsequent spawn events will be started after another interval of this delay time. Spawn events occur sequentially. It is up to the spawn event designer to ensure that all spawn events occur within the time frame of the spawn wave as spawn events that would occur after a spawn wave concludes will not occur.

```
[Tooltip("Time in seconds after wave begins until event starts.")]
public float DelayTime
```

Field Value

float

EnemyTypes

Array of enemy types that will be spawned during this wave.

```
public EnemyType[] EnemyTypes
```

Field Value

EnemyType[]

MinEnemyCount

Defines the minimum number of enemies that will be present while this wave is active. If there are fewer than the minimum enemies in the world for a particular spawn wave, the number of enemies required to meet this minimum value will be spawned in 1 frame.

```
public int MinEnemyCount
```

Field Value

int

SpawnEvents

Defines the spawn events for this spawn wave. Spawn events are groups of entities that will spawn in a certain SpawnFormation with unique behavior.

```
[Header("Spawn Event Settings")]
public EnemySpawnEventProperties[] SpawnEvents
```

Field Value

EnemySpawnEventProperties[]

SpawnInterval

```
public float SpawnInterval
```

Field Value

float

Struct EnemySpawnerState

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Holds the current state data of the enemy spawner.

```
public struct EnemySpawnerState : IComponentData, IQueryTypeParameter
```

Fields

CurWaveIndex

Index associated with the current wave as held in the Value managed array of EnemySpawnWave Properties ScriptableObjects.

```
public int CurWaveIndex
```

Field Value

int

WaveTimer

Timer for the current wave. At the start of each wave, this value is initialized to WaveInterval, each frame the value is decremented by delta time, once the timer reaches 0, the next wave should begin.

```
public float WaveTimer
```

Field Value

float

Struct EnemyTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Empty Tag component used to identify Enemy entities.

```
public struct EnemyTag : IComponentData, IQueryTypeParameter
```

Enum EnemyType

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enum used to identify unique alien types. This enum is set in the Unity Editor on the prefab for a given alien type. This enum will also serve as a key to map to the baked entity prefab. This is so EnemySpawnWaveProperties and EnemySpawnEventProperties ScriptableObjects can define which enemy types are present in the current wave or event and the EnemySpawnSystem can spawn the appropriate prefabs.

```
public enum EnemyType : byte
```

Fields

ArmorAlien_Black = 26

ArmorAlien_White = 27

BaseAlien_Blue = 2

BaseAlien_Gray = 4

BaseAlien_Green = 1

BaseAlien_Red = 3

BigAlienSpots_Blue = 33

BigAlienSpots_Green = 32

BigAlienSpots_Orange = 35

BigAlienSpots_Red = 34

BigAlienStripes_Blue = 37

BigAlienStripes_Green = 36

BigAlienStripes_Orange = 39

BigAlienStripes_Red = 38

BlobAlien_Green = 25

```
CentaurAlien_Green = 13

CentaurAlien_Purple = 14

FloatingHead_Blue = 11

FloatingHead_Gray = 12

FloatingHead_Green = 9

FloatingHead_Red = 10

GhostAlien_Blue = 28

GhostAlien_Green = 29

MechAlien_Dark = 41

MechAlien_Light = 40

MiniAlien_Blue = 22

MiniAlien_Gray = 24

MiniAlien_Green = 21

MiniAlien_Red = 23

None = 0

PlantAlien_Blue = 7

PlantAlien_Green = 6

PlantAlien_Purple = 8

PlantAlien_Red = 5

ReaperAlien = 42

SkeletonAlien_Bone = 30

SkeletonAlien_Gray = 31

SpiderAlien_Black = 15

SpiderAlien_White = 16
```

```
UFOAlien_Blue = 18

UFOAlien_Green = 17

UFOAlien_Purple = 20

UFOAlien_Red = 19
```

Struct EnemyTypeInitialization

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component containing the EnemyType value. This component is only used for initialization and will be removed from the enemy prefab before any enemies are instantiated. See GameStartSystem for usage.

```
public struct EnemyTypeInitialization : IComponentData, IQueryTypeParameter
```

Remarks

This is a regular tag component and not an enableable component as this component will be removed from the enemy prefab before any enemy instances will be spawned.

Fields

Value

```
public EnemyType Value
```

Field Value

EnemyType

Struct EnemyWaveSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to advance to the next spawn wave once the WaveTimer expires.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup), OrderFirst = true)]
[UpdateBefore(typeof(EnemySpawnSystem))]
public struct EnemyWaveSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system also spawns a new boss entity at the start of the next wave. System updates at the beginning of the DS_InitializationSystemGroup and before EnemySpawnSystem to ensure next wave is active before spawning new enemies. Also important that boss enemies spawn before the CharacterInitialization System.

If you want to add in any custom behavior to spawn something after all spawn waves have completed, you could do that in this system. Currently, a final boss entity with very high health and damage will spawn to immediately destroy the player character. The enemy spawned as the final boss is defined in FinalBossType.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct EnhanceThicknessOverride

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Material property override for the spawn time of the enemy. Setting this value will update the _SpawnTime property in the material instance of this entity. This provides an offset for the animation so all enemies can be on their own unique animation frame and will not appear to be synchronized.

```
[MaterialProperty("_EnhanceThickness", -1)]
public struct EnhanceThicknessOverride : IComponentData, IQueryTypeParameter
```

Remarks

Your IDE may gray out the Value field as this value is not used in our code. However, Unity uses it to apply this value to the material property defined in the MaterialProperty attribute. Be sure the MaterialProperty string exactly matches the reference string defined in the shader as this will silently fail if there is a typo.

Fields

Value

```
public float Value
```

Field Value

float

See Also

InitializeAnimationOffsetOverrideFlag
CharacterAnimationSystem

Struct EntityInteraction

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer component to store entity interactions.

```
public struct EntityInteraction : IBufferElementData
```

Remarks

Entity interactions are a key concept to DOTS Survivors as they facilitate interactions between multiple entities. Interactions can come in various forms - an attack projectile applying damage to an enemy entity, a healing item granting health to the player, instructing a projectile to bounce off another entity, and many more. Entity interactions can also be raised in several ways - most commonly this is through collision or time-based events, but logic-only interactions can also exist to facilitate interactions of data-only entities that do not exist physically in the game world. For example, when a damage dealing entity collides with a damage receiving entity, a new entry to the EntityInteraction buffer will be recorded. In this case the damage dealing entity will have the dynamic buffer and the entity receiving damage will be added as the TargetEntity for this record. Note that there is nothing stopping the damage receiving entity to have its own EntityInteraction buffer and adding the damage dealing entity as a record during this same collision event. When an interaction between two entities has been recorded, various interaction systems can trigger additional behavior based on the component makeup of the entity that holds the interaction buffer. In the example of the damage dealing entity, a system could execute to deal damage to the entity stored in the TargetEntity field. However other interaction systems could trigger behavior that don't use the target entity field - there could be a system to play a sound effect for any new entity interaction, or just outright self-destruct on any interaction raised. The inclusion in these interaction systems and their behaviors are determined by the other data components added to the entity with the EntityInteraction buffer. At the end of the of the DS_InteractionSystemGroup the IsHandled field for every element in the EntityInteraction buffer is set to true. This is so that when the interaction systems run in the next frame, they can check the buffer and only trigger behavior for new interactions that have not yet been handled. Rather than clearing the buffer, all previous interactions remain in the buffer. This is because most systems that raise interactions should scan the buffer and see if the target entity is already in the buffer so it doesn't get added multiple times. This is especially useful for trigger events where one entity may pass through another entity for many frames, but the intended behavior is to only trigger behavior on the first frame the entities overlap. Note that depending on the desired behavior of systems that raise interactions and systems that invoke behavior based on interactions, these systems can simply ignore checking the buffer for the target entity or if interactions are already handled. The vast majority of systems in this game do implement these checks, but you will

find examples of ones that do not. BounceOnEnvironmentJob is an example of a trigger events job that does not scan the interaction buffer for the existence of previously added environment entities.

Fields

IsHandled

Boolean field to denote that this interaction has already been handled.

```
public bool IsHandled
```

Field Value

bool

Remarks

This field is checked by systems that trigger behavior from interactions to ensure the behavior is only executed against new, unhandled interactions.

TargetEntity

Entity field to store the target entity of the interaction.

```
public Entity TargetEntity
```

Field Value

Entity

Remarks

This can be used when one entity needs to interact with another in a certain way. For example if one entity should apply damage to another, the entity receiving damage will exist in this field. This field can also be used to ensure a particular entity hasn't already been added to the buffer to avoid situations where interactions are being raised more frequently than they should.

Class EntityInteractionAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring component to add the EntityInteraction buffer to an entity.

```
public class EntityInteractionAuthoring : MonoBehaviour
```

Remarks

This authoring component is commonly added as a "RequireComponent" on other authoring scripts as many different types of entities have the ability to interact with other entities.

Struct EntityRandom

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to store Unity.Mathematics.Random type used for random number generation to be held by a specific entity.

```
public struct EntityRandom : IComponentData, IQueryTypeParameter
```

Remarks

An entity having its own random number generator can be helpful for instances where unique random numbers need to be generated on separate threads in parallel.

Fields

Value

```
public Random Value
```

Field Value

Random

Class EntityRandomAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add EntityRandom component to an entity. This can also add the InitializeEntityRandom component to trigger initialization of random seed that cannot be set during baking i.e. using milliseconds of the system when the entity spawns.

```
public class EntityRandomAuthoring : MonoBehaviour
```

Fields

ManualOverrideValue

If RandomInitializationType is set to ManualOverride, this is the value that will be set as the initial seed during baking.

```
public uint ManualOverrideValue
```

Field Value

uint

RandomInitializationType

Sets the initialization type to be used when setting the initial seed of EntityRandom.

```
public EntityRandomInitializationType RandomInitializationType
```

Field Value

EntityRandomInitializationType

Enum EntityRandomInitializationType

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enum denoting the type of initialization that should be used when determining the initial seed of Entity Random.

```
public enum EntityRandomInitializationType : byte
```

Fields

`ManualOverride = 1`

Manual override value will initialize EntityRandom using index ManualOverrideValue during baking in Bake(EntityRandomAuthoring). This can be helpful for debugging to get consistent random results.

`None = 0`

None value generally should not be used. Will initialize EntityRandom using index 0 during baking in Bake(EntityRandomAuthoring).

`SystemMilliseconds = 2`

System milliseconds value will initialize EntityRandom using index from the system's millisecond time value at the time of initialization in the InitializeEntityRandomSystem

Class EnvironmentAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to apply the EnvironmentTag to an entity.

```
public class EnvironmentAuthoring : MonoBehaviour
```

Struct EnvironmentTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity is an environment entity.

```
public struct EnvironmentTag : IComponentData, IQueryTypeParameter
```

Remarks

Environment entities are similar to decoration entities (DecorationTag) however these do have colliders which can stop movement of player and their attacks. For design decisions, enemies can still pass through environment entities. This is determined by the collision filters on both enemies and environment entities.

Class ExperienceGemItemAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to set initial experience point value for an experience point gem.

```
[RequireComponent(typeof(ItemAuthoring))]
[RequireComponent(typeof(GraphicsEntityAuthoring))]
[RequireComponent(typeof(DestroySelfOnInteractionAuthoring))]
public class ExperienceGemItemAuthoring : MonoBehaviour
```

Fields

ExperiencePointValue

Number of experience points to grant the player when the gem is collected.

```
public int ExperiencePointValue
```

Field Value

int

Struct ExperienceGemItemData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data related to experience gems that have a chance to be dropped when an enemy is destroyed.

```
public struct ExperienceGemItemData : IComponentData, IQueryTypeParameter
```

Fields

Value

Number of experience points to grant the player when the player picks up the gem.

```
public int Value
```

Field Value

int

Struct ExperiencePointColor

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data struct used to assist with authoring experience point gem colors in the Unity editor.

```
[Serializable]
public struct ExperiencePointColor
```

Fields

Color

Color of the experience gem if the experience point value is greater than or equal to the specified ExperiencePointValue.

```
public Color Color
```

Field Value

Color

ExperiencePointValue

Minimum experience point value for the associated color.

```
public int ExperiencePointValue
```

Field Value

int

Struct ExperiencePointTable

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store information related to the amount of experience points required for the player to level up to the next level.

```
public struct ExperiencePointTable : IComponentData, IQueryTypeParameter
```

Remarks

The amount of experience to reach the next level is equal to the amount of experience to reach the current level plus a fixed value. Once the player reaches a certain level, this fixed value between levels increases. This increase happens multiple times throughout gameplay.

Fields

InitialExperiencePoints

Experience points required to level up for the first time.

```
public int InitialExperiencePoints
```

Field Value

int

LevelAtExperienceIncrease

Collection of integers storing the level indices at which the differential between experience points required to reach the next level increases.

```
public int3 LevelAtExperienceIncrease
```

Field Value

int3

Remarks

Using int3 type like an array as only 3 values need to be stored. Can access x, y, z values with 0, 1, 2 indices.

LevelExperienceIncreases

Collection of integers storing the differential between experience points required to reach the next level.

```
public int3 LevelExperienceIncreases
```

Field Value

int3

Remarks

Using int3 type like an array as only 3 values need to be stored. Can access x, y, z values with 0, 1, 2 indices.

Properties

this[int]

Custom indexer to easily access the experience points required to reach a certain level.

```
public int this[int i] { get; }
```

Parameters

`i` int

Index of the level to retrieve experience points required to reach.

Property Value

int

Class ExplodeOnTimerAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for explosion to an entity.

```
[RequireComponent(typeof(DestroyAfterTimeAuthoring))]
[RequireComponent(typeof(GraphicsEntityAuthoring))]
public class ExplodeOnTimerAuthoring : MonoBehaviour
```

Remarks

Requires DestroyAfterTimeAuthoring and GraphicsEntityAuthoring to ensure other necessary components are present.

Fields

Area

The scale of the explosion entity.

```
public float Area
```

Field Value

float

Damage

Amount of damage the explosion entity will deal to opposing enemies.

```
public int Damage
```

Field Value

int

ExplosionPrefab

GameObject prefab that will be spawned via SpawnOnDestroy component in the DestroyEntitySystem.

```
public GameObject ExplosionPrefab
```

Field Value

GameObject

ExplosionTimeToLive

Duration in seconds for how long the explosion entity will exist in the game world.

```
public float ExplosionTimeToLive
```

Field Value

float

FlashColor

Color tint that will be applied to the graphics entity when flashing.

```
public Color FlashColor
```

Field Value

Color

FlashFrequency

Frequency at which the graphics entity will flash - meaning the graphics entity will be alternate between no color tint and being tinted with defined color value each for this amount of time.

```
public float FlashFrequency
```

Field Value

float

FlashTime

Time at which the graphics entity will begin to flash. When the parent's Value is less than this number, the graphics entity will begin to flash.

```
public float FlashTime
```

Field Value

float

Struct ExplodeOnTimerData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data relevant to entities that can explode.

```
public struct ExplodeOnTimerData : IComponentData, IQueryTypeParameter
```

Remarks

During explosion, entities will spawn an explosion entity via a SpawnOnDestroy component in the DestroyEntitySystem. Data in this component will be applied to the explosion entity.

Fields

Area

The scale of the explosion entity.

```
public float Area
```

Field Value

float

Damage

Amount of damage the explosion entity will deal to opposing enemies.

```
public int Damage
```

Field Value

int

ExplosionTimeToLive

Duration in seconds for how long the explosion entity will exist in the game world.

```
public float ExplosionTimeToLive
```

Field Value

float

Struct ExplosionGraphicsData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to store data relevant to the flashing of a GraphicsEntity for exploding entities that have the ExplodeOnTimerData component.

```
public struct ExplosionGraphicsData : IComponentData, IQueryTypeParameter
```

Fields

Color

Color tint that will be applied to the graphics entity when flashing.

```
public float4 Color
```

Field Value

float4

FlashFrequency

Frequency at which the graphics entity will flash - meaning the graphics entity will be alternate between no color tint and being tinted with defined color value each for this amount of time.

```
public float FlashFrequency
```

Field Value

float

FlashTime

Time at which the graphics entity will begin to flash. When the parent's Value is less than this number, the graphics entity will begin to flash.

```
public float FlashTime
```

Field Value

float

Struct ExplosionGraphicsInitializationData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data used to initialize the ExplosionGraphicsData.

```
public struct ExplosionGraphicsInitializationData : IComponentData,
IQueryTypeParameter, IEnableableComponent
```

Remarks

This is required as the ExplosionGraphicsData component will be added to the GraphicsEntity for this explosion entity, so these values cannot be baked onto the other entity and must be set at runtime. Enableable component will be disabled after initialization.

Fields

Color

Color tint that will be applied to the graphics entity when flashing.

```
public float4 Color
```

Field Value

float4

FlashFrequency

Frequency at which the graphics entity will flash - meaning the graphics entity will be alternate between no color tint and being tinted with defined color value each for this amount of time.

```
public float FlashFrequency
```

Field Value

float

FlashTime

Time at which the graphics entity will begin to flash. When the parent's Value is less than this number, the graphics entity will begin to flash.

```
public float FlashTime
```

Field Value

float

Struct FacingDirectionOverride

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Material property override for the facing direction of the character. Setting this value will update the _FacingDirection property in the material instance of this entity. This will flip the character animation so the character faces to the left or right depending on the direction of movement.

```
[MaterialProperty("_FacingDirection", -1)]
public struct FacingDirectionOverride : IComponentData, IQueryTypeParameter
```

Remarks

A value of 1 makes the character face to the right and a value of -1 makes the character face to the left. Your IDE may gray out the Value field as this value is not used in our code. However, Unity uses it to apply this value to the material property defined in the MaterialProperty attribute. Be sure the MaterialProperty string exactly matches the reference string defined in the shader as this will silently fail if there is a typo.

Fields

Value

```
public float Value
```

Field Value

float

See Also

CharacterMoveSystem

Class FadeAttackInOutAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components required for fading the attack visual in and out.

```
public class FadeAttackInOutAuthoring : MonoBehaviour
```

Remarks

Note: this authoring component will be added to the graphics entity which should be the direct child of the main attack prefab. This also assumes that the main attack prefab has a DestroyAfterTime component.

Struct FadeAttackInOutSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to process fading the attack visuals in and out.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct FadeAttackInOutSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System gets the current value of the parent's DestroyAfterTime component, which stores the time in seconds remaining until the entity will be destroyed. Uses a quadratic function to calculate the current alpha value which is applied via the Unity built-in material override - URPMaterialPropertyBaseColor. This system updates in the DS_EffectsSystemGroup which updates towards the end of the frame.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct FadeAttackInOutTotalTimeToLive

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to hold the total time this entity will exist in the game world. Used to calculate the alpha value for fading the attack visual in and out.

```
public struct FadeAttackInOutTotalTimeToLive : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public float Value
```

Field Value

float

Struct FadeCompanionTrailRenderer

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Cleanup component to call the EndTrailRenderer() method when the main jetpack entity is destroyed. This provides a nice visual fade out of the trail renderer on jetpacks.

```
public struct FadeCompanionTrailRenderer : ICleanupComponentData,
IComponentData, IQueryTypeParameter
```

Fields

Value

```
public UnityObjectRef<JetpackTrailRendererController> Value
```

Field Value

UnityObjectRef<JetpackTrailRendererController>

Struct FadeCompanionTrailRendererSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize the FadeCompanionTrailRenderer cleanup component. Also upon entity destruction, this system will call the EndTrailRenderer() method which provides a nice fade out effect of the trail renderer.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct FadeCompanionTrailRendererSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system updates in the DS_EffectsSystemGroup which updates towards the end of the frame.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class FadeCompanionTrailRendererTagAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add the InitializeFadeCompanionTrailRenderer component to the entity.

```
[RequireComponent(typeof(CompanionGameObjectAuthoring))]
public class FadeCompanionTrailRendererTagAuthoring : MonoBehaviour
```

Remarks

Requires the CompanionGameObjectAuthoring component to ensure all companion elements are added to the entity. This authoring script should be added to the attack graphics entity which is a direct child of the the main attack prefab.

See Also

FadeCompanionTrailRenderer
FadeCompanionTrailRendererSystem

Struct FixCharacterPositionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

This system fixes the character's position to 0 in the y-axis as this game takes place on the x and z 2D axes.

```
[UpdateInGroup(typeof(DS_PhysicsSystemGroup), OrderLast = true)]
public struct FixCharacterPositionSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Although no game systems move characters along the y-axis, characters can slightly fall off the y-axis during collisions. This can lead to behavior where entities overlap and move over/under one another.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class FlashColorOnDamageAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize values for the FlashColorOnDamageData component and add other necessary components.

```
public class FlashColorOnDamageAuthoring : MonoBehaviour
```

Remarks

Note that this authoring script should be attached to the graphics entity which is a child of the character entity.

Fields

Color

Color the entity will be tinted when taking damage.

```
public Color Color
```

Field Value

Color

Remarks

This is actually the 2nd color that will flash. For the first half of the effect the entity will be all black.

FlashTime

Total time of the flash effect in seconds.

```
public float FlashTime
```

Field Value

float

Remarks

The first half of this time, the entity will be all black. The second half the entity will be tinted with the color defined in the Color field.

Struct FlashColorOnDamageData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to hold data related to visual effect that flashes the graphics entity of a character when taking damage.

```
public struct FlashColorOnDamageData : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

This component should go on the graphics entity of the character. Enableable component so the Flash ColorOnDamageSystem will only run on entities with this component enabled.

Fields

Color

Color the entity will be tinted when taking damage.

```
public float4 Color
```

Field Value

float4

Remarks

This is actually the 2nd color that will flash. For the first half of the effect the entity will be all black.

FlashTime

Total time of the flash effect in seconds.

```
public float FlashTime
```

Field Value

float

Remarks

The first half of this time, the entity will be all black. The second half the entity will be tinted with the color defined in the Color field.

See Also

FlashColorOnDamageTimer
FlashColorOnDamageSystem

Struct FlashColorOnDamageSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

This system will flash the graphics entity black and then a color defined in Color when the parent character entity takes damage. This provides a nice visual feedback for the player when an entity takes damage.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct FlashColorOnDamageSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system updates in the DS_EffectsSystemGroup which updates towards the end of the frame.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct FlashColorOnDamageTimer

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Timer used to display the flash effect for a certain period of time. This timer will count down from Flash Time to zero.

```
public struct FlashColorOnDamageTimer : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public float Value
```

Field Value

float

See Also

FlashColorOnDamageSystem

Struct FlashOnTimerSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle initialization of ExplosionGraphicsData on the GraphicsEntity and flash the graphics entity during the flash phase.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct FlashOnTimerSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class GameAudioController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

MonoBehaviour for controlling audio in game.

```
public class GameAudioController : MonoBehaviour
```

Fields

Instance

Public singleton access to play audio in game.

```
public static GameAudioController Instance
```

Field Value

GameAudioController

Methods

GetNormalizedMusicLevel()

Method to return the normalized music volume level.

```
public float GetNormalizedMusicLevel()
```

Returns

float

Normalized music volume level from 0 and 1

Remarks

If volume level is lower than the defined minimum value, 0 will be returned as the music volume can be considered muted. This is used to set the music volume slider in UI configuration based on its current values.

GetNormalizedSfxLevel()

Method to return the normalized sound effects volume level.

```
public float GetNormalizedSfxLevel()
```

Returns

float

Normalized sound effects volume level from 0 and 1

Remarks

If volume level is lower than the defined minimum value, 0 will be returned as the sound effects volume can be considered muted. This is used to set the sound effects volume slider in UI configuration based on its current values.

PlayLoopingAudioClip(AudioClip, int)

Method for playing a looping sound effects clip.

```
public int PlayLoopingAudioClip(AudioClip audioClip, int priority = 64)
```

Parameters

`audioClip` AudioClip

Audio clip to play.

`priority` int

Priority of the sound effect. AudioPriority

Returns

int

Returns the index in the TMG.DOTSSurvivors.GameAudioController._loopingAudioSources of the audio source this effect is playing out of. This is used so LoopingAudioSourceIndex can stop the looping audio source when it should no longer play.

PlayPauseBackgroundMusic()

Method for toggling the background music between play and pause state.

```
public void PlayPauseBackgroundMusic()
```

Remarks

Certain events such as picking up a crate should pause background music. It should resume once the crate UI is closed.

PlayPauseResistantAudioClip(AudioClip, int)

Method for playing a pause resistant sound effects clip.

```
public void PlayPauseResistantAudioClip(AudioClip audioClip, int priority = 32)
```

Parameters

`audioClip` AudioClip

Audio clip to play.

`priority` int

Priority of the sound effect. AudioPriority

Remarks

This is typically used for sound effects that must play when the game is in a paused state. i.e. level up sound effect.

PlaySfxAudioClip(AudioClip, int)

Method for playing a normal sound effects clip.

```
public void PlaySfxAudioClip(AudioClip audioClip, int priority = 128)
```

Parameters

`audioClip` AudioClip

Audio clip to play.

`priority` int

Priority of the sound effect. AudioPriority

SetMusicVolume(float)

Sets the music volume based on normalized value from UI slider.

```
public void SetMusicVolume(float normalizedLevel)
```

Parameters

`normalizedLevel` float

Value from 0 to 1 for the music volume.

Remarks

If normalized value is below the TMG.DOTSSurvivors.GameAudioController.MIN_CUTOFF_LEVEL, the music volume will be set to -80dB, effectively muting the channel.

SetSfxVolume(float)

Sets the sound effects volume based on normalized value from UI slider.

```
public void SetSfxVolume(float normalizedLevel)
```

Parameters

`normalizedLevel` float

Value from 0 to 1 for the sound effects volume.

Remarks

If normalized value is below the TMG.DOTSSurvivors.GameAudioController.MIN_CUTOFF_LEVEL, the sound effects volume will be set to -80dB, effectively muting the channel.

StopLoopingAudioClip(int)

Method for stopping a looping audio source.

```
public void StopLoopingAudioClip(int index)
```

Parameters

`index` int

Index in the TMG.DOTSSurvivors.GameAudioController._loopingAudioSources of the audio source to stop playing.

See Also

LoopingAudioSourceIndex

See Also

MainMenuAudioController

Class GameAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for controlling the ECS side of the game.

```
[RequireComponent(typeof(EntityRandomAuthoring))]
public class GameAuthoring : MonoBehaviour
```

Remarks

Requires the EntityRandomAuthoring script to add EntityRandom component used for random number generation.

Fields

CratePrefab

GameObject prefab of the supply crate which will be converted to an entity.

```
public GameObject CratePrefab
```

Field Value

GameObject

ExperienceGemColorLookup

List to store the mapping of experience point values to the color of gems they will spawn.

```
public List<ExperiencePointColor> ExperienceGemColorLookup
```

Field Value

List <ExperiencePointColor>

ExperienceGemPrefab

GameObject prefab of the experience point gem which will be converted to an entity.

```
public GameObject ExperienceGemPrefab
```

Field Value

GameObject

HealthBonusItem

BonusItemProperties for granting the player health.

```
public BonusItemProperties HealthBonusItem
```

Field Value

BonusItemProperties

InitialExperiencePoints

Experience points required to level up for the first time.

```
[Header("Experience Point Data")]
public int InitialExperiencePoints
```

Field Value

int

LevelAtExperienceIncrease

Collection of integers storing the level indices at which the differential between experience points required to reach the next level increases.

```
public int[] LevelAtExperienceIncrease
```

Field Value

int[]

Remarks

Array should be size of 3 as they are stored in an int3 value in the data component.

LevelExperienceIncrease

Collection of integers storing the differential between experience points required to reach the next level.

```
public int[] LevelExperienceIncrease
```

Field Value

int[]

Remarks

Array should be size of 3 as they are stored in an int3 value in the data component.

MaxPassiveCount

Maximum number of passive items the player is allowed to have.

```
public int MaxPassiveCount
```

Field Value

int

MaxWeaponCount

Maximum number of weapons the player is allowed to have.

```
[Header("Player Capability Data")]
public int MaxWeaponCount
```

Field Value

int

MoneyBonusItem

BonusItemProperties for granting the player money.

```
public BonusItemProperties MoneyBonusItem
```

Field Value

BonusItemProperties

PassivePrefab

GameObject prefab of the passive item which will be converted to an entity.

```
public GameObject PassivePrefab
```

Field Value

GameObject

PlayerPrefab

GameObject prefab of the player which will be converted to an entity.

```
[Header("Entity Prefabs")]
public GameObject PlayerPrefab
```

Field Value

GameObject

PlayerWorldUICanvasPrefab

GameObject prefab of the world UI canvas which will follow the player and display their health bar.

```
[Header("Managed Elements")]
public GameObject PlayerWorldUICanvasPrefab
```

Field Value

GameObject

StartingPlayerProperties

Debug-only setting - when entering playmode while in a level, this character will be spawned into the world. Otherwise, if entering the game through the title screen, the selected character will be used.

```
public CharacterProperties StartingPlayerProperties
```

Field Value

CharacterProperties

Struct GameEntityPrefabs

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to entity prefabs utilized throughout the game.

```
public struct GameEntityPrefabs : IComponentData, IQueryTypeParameter
```

Fields

CratePrefab

Prefab of a crate dropped by boss enemies.

```
public Entity CratePrefab
```

Field Value

Entity

ExperienceGemPrefab

Prefab of an experience gem dropped by enemies.

```
public Entity ExperienceGemPrefab
```

Field Value

Entity

PassivePrefab

Prefab of a passive item the player can obtain through leveling up.

```
public Entity PassivePrefab
```

Field Value

Entity

PlayerEntityPrefab

Prefab of the player entity.

```
public Entity PlayerEntityPrefab
```

Field Value

Entity

Struct GameEntityTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity as the entity containing data related to gameplay.

```
public struct GameEntityTag : IComponentData, IQueryTypeParameter
```

Remarks

This is often used when attempting to access managed data components on this entity. First a reference to the entity must be obtained (using SystemAPI.GetSingletonEntity or otherwise) then the managed component can be grabbed from the entity.

Class GameOverUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the game over screen that is displayed after the player character is destroyed.

```
public class GameOverUIController : MonoBehaviour
```

Struct GameStartSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize things for the start of the game.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
public struct GameStartSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Updates in the DS_InitializationSystemGroup which executes towards the beginning of the frame to ensure the game is initialized before core gameplay systems run.

Methods

OnCreate(ref SystemState)

Requiring the InitializeGameTag means this system should only run the OnUpdate method once as this component will be removed from the game entity immediately. The OnUpdate method is where the game initialization logic exists.

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct GameStartSystem.Singleton

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Singleton component associated with this system to store the game index.

```
public struct GameStartSystem.Singleton : IComponentData, IQueryTypeParameter
```

Fields

GameIndex

Game index is incremented each time the game is played.

```
public int GameIndex
```

Field Value

int

Remarks

This game index is used in the InitializeSpawnRandomOnDestroySystem to ensure a unique hash for blob assets are created each time the game is entered. This is important because if blob assets from previous games are used, they will not map to valid prefabs and thus nothing will spawn.

Struct GameTime

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store the current game time in seconds.

```
public struct GameTime : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public float Value
```

Field Value

float

Class GetPlayerInputSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System for reading input from the input system package and assigning it to the player's CharacterMoveDirection.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
public class GetPlayerInputSystem : SystemBase
```

Remarks

System is a SystemBase type so it can have a managed member variable for the input actions. Updates in the DS_InitializationSystemGroup so input is read early in the frame.

Methods

OnCreate()

```
protected override void OnCreate()
```

OnStartRunning()

```
protected override void OnStartRunning()
```

OnStopRunning()

```
protected override void OnStopRunning()
```

OnUpdate()

```
protected override void OnUpdate()
```

Class GlobalTimeUpdateSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to update the global time shader property. This is required so that when the game pauses, any animated shaders that rely on this property will also pause as this system is part of a group that will be disabled when the game is paused due to level up or pressing the ESC key (PauseManager).

```
[UpdateInGroup(typeof(InitializationSystemGroup))]
[UpdateAfter(typeof(UpdateWorldTimeSystem))]
public class GlobalTimeUpdateSystem : SystemBase
```

Remarks

System updates in Unity's InitializationSystemGroup and after their UpdateWorldTimeSystem to ensure that SystemAPI.Time.ElapsedTime is accurate for the current frame.

Methods

OnCreate()

```
protected override void OnCreate()
```

OnUpdate()

```
protected override void OnUpdate()
```

Struct GrantMoneyOnInteraction

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component defining the value of a coin item.

```
public struct GrantMoneyOnInteraction : IComponentData, IQueryTypeParameter
```

Fields

Value

Amount of money granted when the coin item is collected by the player.

```
public int Value
```

Field Value

int

See Also

GrantMoneyOnInteractionSystem

Class GrantMoneyOnInteractionAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize the value of a coin item.

```
[RequireComponent(typeof(ItemAuthoring))]
[RequireComponent(typeof(DestroySelfOnInteractionAuthoring))]
public class GrantMoneyOnInteractionAuthoring : MonoBehaviour
```

Remarks

Additional authoring scripts required to add components necessary for desired behavior.

Fields

CoinValue

```
public int CoinValue
```

Field Value

int

Class GrantMoneyOnInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to grant money to the player when the coin item is collected.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public class GrantMoneyOnInteractionSystem : SystemBase
```

Remarks

System is a SystemBase type as it has a System.Action event type as a member variable that is invoked when the player collects a coin.

Fields

OnUpdateCoinCount

Event invoked when the player collects a coin. Used to update the HUD UI with an up to date coin count.

```
public Action<int> OnUpdateCoinCount
```

Field Value

Action<int>

Methods

OnUpdate()

```
protected override void OnUpdate()
```

See Also

GrantMoneyOnInteraction
EntityInteraction

Struct GraphicsEntity

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store a reference to the graphics entity for this entity.

```
public struct GraphicsEntity : IComponentData, IQueryTypeParameter
```

Remarks

A common pattern in this game is to have the visual representation of an entity exist as a child of the primary entity. This is useful to reduce the memory footprint of the main entity which is often a part of more complex systems. This also allows for independent effects to animate via the transform of the graphics entity without messing with the transform of the main entity. Often the player entity will need to reference its graphics entity to play effects and this component provides reliable access to that entity without needing to iterate the child buffer.

Fields

Value

```
public Entity Value
```

Field Value

Entity

Class GraphicsEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add GraphicsEntity component to an entity.

```
public class GraphicsEntityAuthoring : MonoBehaviour
```

Fields

GraphicsEntity

```
public GameObject GraphicsEntity
```

Field Value

GameObject

PlayDestructionAnimation

```
public bool PlayDestructionAnimation
```

Field Value

bool

Struct GraphicsEntityPlayDestroyEffectTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to inform the DestroyEntitySystem this character has a destruction animation that should play when the character is destroyed.

```
public struct GraphicsEntityPlayDestroyEffectTag : IComponentData, IQueryTypeParameter
```

Class HUDUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the in-game HUD UI.

```
public class HUDUIController : MonoBehaviour
```

Remarks

Controls all UI elements the player can reference when in the game.

Struct HandleBounceOnEnvironmentSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle EntityInteractions between environment entities (EnvironmentTag) and entities tagged with BounceOnEnvironmentTag)

```
[UpdateInGroup(typeof(DS_PhysicsSystemGroup))]
[UpdateAfter(typeof(BounceOnEnvironmentSystem))]
public struct HandleBounceOnEnvironmentSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System updates in the DS_PhysicsSystemGroup to ensure the bounce is handled at the appropriate time. This system iterates over entity interactions on the bounceable entity. Skip over handled interactions and interactions with entities that do not have the environment tag. Next the physics collider of the environment entity is used to do a collider cast at the environment entity's position to find the collision with the bounceable entity. The purpose of this is to find the surface normal of the environment entity to perform the bounce. As the bounce is initially recorded as a trigger event in BounceOnEnvironmentJob, there is no way to get information regarding the collision point or surface normal in the trigger event. Finally, using the surface normal a reflection can be applied, also the bounceable entity is moved slightly away from the environment entity so multiple bounces are not incurred.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class HandleCrateItemInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System containing logic for when the player collects a crate.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public class HandleCrateItemInteractionSystem : SystemBase
```

Remarks

System is a SystemBase type as it has a System.Action event type as a member variable that is invoked when the player collects a crate. This system first determines which upgrade will be granted to the player. If all the player's current weapons and passive abilities are already at the max level, additional money will be granted to the player.

Fields

OnBeginOpenCrate

Event invoked when the player collects a crate. Informs CrateUIController to begin the crate opening sequence.

```
public Action<CapabilityUpgradeLevel> OnBeginOpenCrate
```

Field Value

Action <CapabilityUpgradeLevel>

Methods

OnCreate()

```
protected override void OnCreate()
```

OnUpdate()

```
protected override void OnUpdate()
```

See Also

CrateUIController

Struct HandleEntityInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to mark the IsHandled field to true.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup), OrderLast = true)]
public struct HandleEntityInteractionSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system runs at the end of the DS_InteractionSystemGroup to ensure that all systems that trigger behavior based on interactions have already executed before setting this value. See documentation on the EntityInteraction dynamic buffer component for more information on the purpose and use-case for the IsHandled field.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct HandleExperienceGemInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to add player experience points when the player collects an experience point gem.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public struct HandleExperienceGemInteractionSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class HandlePlayerExperienceThisFrameSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to add player experience from the player's PlayerExperienceThisFrame dynamic buffer to the player's TotalPoints. This system also determines if the player should level up. If the player should level up, then this system will also determine a weapon or passive ability to upgrade.

```
[UpdateAfter(typeof(DS_InteractionSystemGroup))]
public class HandlePlayerExperienceThisFrameSystem : SystemBase
```

Remarks

System is a SystemBase system so it can have System.Action events as member variables that can be invoked when UI elements need to be updated. System checks to ensure IsOpeningCrateFlag is set to false to avoid the possibility of showing level up UI and crate opening UI on the same frame.

Fields

OnBeginLevelUp

Event to be invoked when the player levels up. CapabilityUpgradeLevel array passed containing the selected UpgradeProperties and next level indices for the selected upgrades.

```
public Action<CapabilityUpgradeLevel[]> OnBeginLevelUp
```

Field Value

Action <CapabilityUpgradeLevel[]>

See Also

ShowLevelUpUI(CapabilityUpgradeLevel[])

OnUpdatePlayerExperience

Event to be invoked when the player gains experience points. Used for UI to update the HUD experience slider at the top of the screen. Argument passed:

1. float - Player's new total experience point value.

```
public Action<float> OnUpdatePlayerExperience
```

Field Value

Action<float>

OnUpdatePlayerLevel

Event to be invoked when the player levels up. Arguments passed:

1. int - level the player is upgrading to
2. int - minimum experience points for the level the player is upgrading to
3. int - maximum experience points for the level the player is upgrading to

```
public Action<int, int, int> OnUpdatePlayerLevel
```

Field Value

Action<int, int, int>

See Also
UpdatePlayerLevel(int, int, int)

Methods

OnCreate()

```
protected override void OnCreate()
```

OnUpdate()

```
protected override void OnUpdate()
```

Struct HandleTemporaryStatModifierInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to create temporary stat modifier entities that will last for a certain period of time as defined by Value. Created as an interaction system as the TemporaryStatModifierActiveTime is applied to items that are picked up by the player.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public struct HandleTemporaryStatModifierInteractionSystem :
ISystem, ISystemCompilerGenerated
```

Remarks

This system creates a new stat modifier entity with a DestroyAfterTime component.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct HealOnInteraction

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Hit points to be granted if player collects a health item.

```
public struct HealOnInteraction : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public int Value
```

Field Value

int

See Also

HealOnInteractionSystem

Class HealOnInteractionAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize value of health item.

```
[RequireComponent(typeof(ItemAuthoring))]
[RequireComponent(typeof(DestroySelfOnInteractionAuthoring))]
public class HealOnInteractionAuthoring : MonoBehaviour
```

Remarks

Additional authoring scripts required to add components necessary for desired behavior.

Fields

HealthValue

```
public int HealthValue
```

Field Value

int

Struct HealOnInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to grant health to the player when a health item is collected.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public struct HealOnInteractionSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

HealOnInteraction
EntityInteraction

Struct InitializeAnimationOffsetOverrideFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag component to signify this entity needs its AnimationOffsetOverride component initialized.

```
public struct InitializeAnimationOffsetOverrideFlag : IComponentData,
IQueryTypeParameter, IEnableableComponent
```

See Also

CharacterAnimationSystem

Struct InitializeArtTestSceneSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize data for the secret art test scene.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
public struct InitializeArtTestSceneSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

ArtTestSceneData

Struct InitializeArtTestSceneTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to determine if the secret art test scene should be initialized

```
public struct InitializeArtTestSceneTag : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

Component will be removed from associated singleton entity after initialization has completed.

Struct InitializeCameraTarget

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component used to initialize the CameraTarget component attached to the player entity.

```
public struct InitializeCameraTarget : IComponentData, IQueryTypeParameter
```

Remarks

In the InitializeCameraTargetSystem this component will be used to locate the CameraTargetObject and assign values in the CameraTarget component attached to the player entity.

Fields

YOffset

Units along the Y-axis the camera will be offset to ensure all entities are in view.

```
public float YOffset
```

Field Value

float

Struct InitializeCameraTargetSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize data for the CameraTarget by locating the CameraTargetObject and using data from InitializeCameraTarget.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
public struct InitializeCameraTargetSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct InitializeCharacterFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag component to signify this character entity needs to be initialized via the CharacterInitialization System.

```
public struct InitializeCharacterFlag : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

Characters spawn with this flag enabled, once initialization for the character is complete, this component will be disabled. Initialization should happen very shortly after the character is spawned.

Struct InitializeEnhancementMaterialFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag component to signify the EnhanceThicknessOverride component should be enabled.

```
public struct InitializeEnhancementMaterialFlag : IComponentData,
IQueryTypeParameter, IEnableableComponent
```

Remarks

Note that this component will be added to the main enemy entity when spawning an enhanced entity. However, the EnhanceThicknessOverride component will be added to the child graphics entity in Enemy AnimationAuthoring.

See Also

CharacterAnimationSystem

Struct InitializeEntityRandom

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Helper data component to signify this entity needs its EntityRandom component initialized in the InitializeEntityRandomSystem.

```
public struct InitializeEntityRandom : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

Enableable component will be disabled once EntityRandom is initialized.

Fields

InitializationType

```
public EntityRandomInitializationType InitializationType
```

Field Value

EntityRandomInitializationType

Struct InitializeEntityRandomSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize EntityRandom based on the EntityRandomInitializationType stored in InitializeEntityRandom.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup), OrderFirst = true)]
[UpdateBefore(typeof(EnemyWaveSystem))]
public struct InitializeEntityRandomSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system only sets initial seed for initialization types of SystemMilliseconds. For other initialization types, it is assumed EntityRandom is already initialized during baking or instantiation. An additional index is added to the system's milliseconds value so that multiple entities that are initialized on the same frame do not start with the same random seed. System updates in the DS_InitializationSystemGroup before the EnemyWaveSystem to ensure EntityRandom is initialized before other systems need access to it.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct InitializeExperienceGemFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag to initialize the experience gem.

```
public struct InitializeExperienceGemFlag : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

Initialization sets the color of the gem based on its value. Value to color mapping is authored in ExperienceGemColorLookup and stored in ExperienceGemColorLookup

Struct InitializeExperienceGemSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize the color of the experience point gem based on its value.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
public struct InitializeExperienceGemSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct InitializeFadeAttackInOutFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag component to signify this entity needs its FadeAttackInOutTotalTimeToLive component initialized.

```
public struct InitializeFadeAttackInOutFlag : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Struct InitializeFadeCompanionTrailRenderer

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to signify the associated entity needs its FadeCompanionTrailRenderer component initialized.

```
public struct InitializeFadeCompanionTrailRenderer : IComponentData, IQueryTypeParameter
```

Remarks

This initialization step is required as FadeCompanionTrailRenderer is an enableable component which cannot be baked onto an entity.

Struct InitializeGameTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to signify the game needs to be initialized.

```
public struct InitializeGameTag : IComponentData, IQueryTypeParameter
```

Remarks

The reason this is a tag component and not an enableable flag component is because the GameStart System will initialize the game in its OnUpdate() method. As such this tag component is a required component for updating the system - enableable components cannot be added to the RequireForUpdate of a system. This tag component will be removed after initialization so that this system will not run again throughout the duration of the game.

Struct InitializePlayAudioClipOnDestroyData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to initialize PlayAudioClipOnDestroyData.

```
public struct InitializePlayAudioClipOnDestroyData : IComponentData, IQueryTypeParameter
```

Remarks

As PlayAudioClipOnDestroyData is a cleanup component, it cannot be added to the entity through baking so this initialization component is used to pass these values to the instance.

Fields

AudioClip

Audio clip to play when this entity is destroyed.

```
public UnityObjectRef<AudioClip> AudioClip
```

Field Value

UnityObjectRef<AudioClip>

Priority

Priority of the audio clip to play. Lower values have a higher priority.

```
public byte Priority
```

Field Value

byte

Struct InitializeSpawnRandomOnDestroySystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

This system is responsible for creating a blob asset containing a blob array of RandomItemToSpawn to be stored in a RandomItemToSpawnBlob component.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
public struct InitializeSpawnRandomOnDestroySystem : ISystem, ISystemCompilerGenerated
```

Remarks

Due to the nature of entity remapping from prefabs created during baking to prefabs available at runtime, a RandomItemToSpawnBlob blob array must be populated with the required prefabs at runtime. To facilitate with this, prefabs that can be spawned are first baked into entities in the SpawnRandomOnDestroyAuthoring.Baker into a DynamicBuffer of SpawnRandomPrefabHelper. The InitializeSpawnRandomOnDestroySystem will populate this blob array with the appropriate data. This system uses the GameIndex to ensure a unique hash for blob assets are created each time the game is entered. This is important because if blob assets from previous games are used, they will not map to valid prefabs and thus nothing will spawn.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

RandomItemToSpawn
RandomItemDropProperties
DestroyEntitySystem
SpawnRandomOnDestroyAuthoring
SpawnRandomPrefabHelper

Struct InstantDestroyEntitySystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System responsible for immediately destroying entities with the InstantDestroyTag. This system will execute at the end of the DS_DestructionSystemGroup to ensure all systems that may add this tag have executed but before the DestroyEntitySystem to avoid the possibility of executing any additional destruction logic in that system.

```
[UpdateInGroup(typeof(DS_DestructionSystemGroup), OrderLast = true)]
[UpdateBefore(typeof(DestroyEntitySystem))]
public struct InstantDestroyEntitySystem : ISystem, ISystemCompilerGenerated
```

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

InstantDestroyTag

DestroyEntitySystem

Class InstantDestroyOnInteractionAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add InstantDestroyOnInteractionTag component to entity.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
public class InstantDestroyOnInteractionAuthoring : MonoBehaviour
```

See Also

InstantDestroyOnInteractionSystem
InstantDestroyOnInteractionTag
EntityInteraction

Struct InstantDestroyOnInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to add the InstantDestroyTag to an entity tagged with the InstantDestroyOnInteractionTag when an EntityInteraction is raised.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public struct InstantDestroyOnInteractionSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

InstantDestroyOnInteractionTag
EntityInteraction
InstantDestroyTag
InstantDestroyEntitySystem

Struct InstantDestroyOnInteractionTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Entities with this component wll be instantly destroyed when an interaction is raised.

```
public struct InstantDestroyOnInteractionTag : IComponentData, IQueryTypeParameter
```

See Also

InstantDestroyOnInteractionSystem
InstantDestroyOnInteractionAuthoring
EntityInteraction

Struct InstantDestroyTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

When this component is added to an entity, it will be destroyed at the end of the frame, ignoring any additional destruction logic that would normally be triggered in the DestroyEntitySystem

```
public struct InstantDestroyTag : IComponentData, IQueryTypeParameter
```

See Also

InstantDestroyEntitySystem
DestroyEntitySystem

Struct InvincibilityExpirationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to take the player off invincibility once the invincibility expiration timestamp is reached. As InvincibilityExpirationTimestamp is an enableable component, entities are only considered invincible when the component is enabled; foreach loop will also only iterate over entities with the component enabled.

```
public struct InvincibilityExpirationSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct InvincibilityExpirationTimestamp

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Timestamp for when the associated entity is no longer invincible to incoming damage. This component will be enabled when the entity is invincible to incoming damage and disabled for normal behavior where the entity can receive damage.

```
public struct InvincibilityExpirationTimestamp : IComponentData,
IQueryTypeParameter, IEnableableComponent
```

Remarks

Should the entity be considered invincible, incoming damage buffered in a DamageThisFrame dynamic buffer will simply be ignored in the ProcessDamageThisFrameSystem.

Fields

Value

```
public double Value
```

Field Value

double

Struct IsAttractedToPlayerData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enableable component to signify when the item is attracted to the player.

```
public struct IsAttractedToPlayerData : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

This component also contains data related to the movement of the item from its start position to the player. A component becomes attracted to the player when the item is within attractable radius of the player (Radius).

Fields

StartPosition

Start position of the item. Required for calculating movement towards the player.

```
public float3 StartPosition
```

Field Value

float3

StartTime

Start time in game time that the item became attracted to the player.

```
public double StartTime
```

Field Value

double

Struct IsOpeningCrateFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag component to signify if the player is currently opening a loot crate.

```
public struct IsOpeningCrateFlag : IComponentData, IQueryTypeParameter, IEnableableComponent
```

Remarks

Enableable component will be enabled when the player is currently opening a loot crate and the associated UI is being displayed. Component will be disabled when the player is not actively opening a loot crate - i.e. during normal gameplay. The purpose of this component is to stop the HandlePlayerExperienceThisFrameSystem from executing to avoid a situation where the player level up UI and loot crate UI are shown on the same frame.

See Also

CrateItemAuthoring

Class ItemAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring component to add ItemTag to the baked entity.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
public class ItemAuthoring : MonoBehaviour
```

Remarks

Additional authoring scripts required to add components necessary for desired behavior.

Struct ItemCollectionJob

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Job that checks for collision events between the player and items in the game world. Valid collisions will add an interaction to the EntityInteraction dyanmic buffer so subsequent systems can apply additional logic pertaining to the specific item.

```
[BurstCompile]
public struct ItemCollectionJob : ICollisionEventsJob, ICollisionEventsJobBase
```

Fields

InteractionBufferLookup

Buffer lookup used to add the player entity to the interaction buffer for the item in the collision event.

```
public BufferLookup<EntityInteraction> InteractionBufferLookup
```

Field Value

BufferLookup<EntityInteraction>

ItemLookup

ReadOnly component lookup used to identity the item entity in the collision event.

```
[ReadOnly]
public ComponentLookup<ItemTag> ItemLookup
```

Field Value

ComponentLookup<ItemTag>

PlayerLookup

ReadOnly component lookup used to identity the player entity in the collision event.

```
[ReadOnly]
public ComponentLookup<PlayerTag> PlayerLookup
```

Field Value

ComponentLookup<PlayerTag>

Methods

Execute(CollisionEvent)

```
[BurstCompile]
public void Execute(CollisionEvent collisionEvent)
```

Parameters

`collisionEvent` CollisionEvent

Struct ItemCollectionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to schedule the ItemCollectionJob. Job is a collision event job so it is scheduled from the DS_PhysicsSystemGroup.

```
[UpdateInGroup(typeof(DS_PhysicsSystemGroup))]
public struct ItemCollectionSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct ItemMoveSpeed

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component that defines the move speed of attractable items from their starting position to the player.

```
public struct ItemMoveSpeed : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public float Value
```

Field Value

float

Struct ItemTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to identify this as an item entity.

```
public struct ItemTag : IComponentData, IQueryTypeParameter
```

Struct JetpackAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of jetpacks into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct JetpackAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Jetpacks are affected by modifications to the player's additional attack projectiles, damage dealt, attack duration, and attack projectile speed stat modifications. The initial jetpack attack will spawn in the direction towards a random enemy on screen. Subsequent jetpacks will be offset from that initial angle by the angle defined in RadianAngleBetweenAttacks. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

```
state SystemState
```

Class JetpackEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for jetpack in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class JetpackEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime
LinearMovementSpeed
ScreenEdgeBounceTag
DestroyAfterNumberHits

Class JetpackTrailRendererController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

MonoBehaviour component attached to companion trail renderers of jetpack attack entities. Provides some nice visual effects to fade the trail renderer in and out.

```
public class JetpackTrailRendererController : MonoBehaviour
```

Methods

EndTrailRenderer()

```
public void EndTrailRenderer()
```

Class JetpackWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add JetpackWeaponData to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
[RequireComponent(typeof(EntityRandomAuthoring))]
public class JetpackWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity. Requires the EntityRandomAuthoring script for random number generation in JetpackAttackSystem.

Fields

DegreeAngleBetweenAttacks

The initial jetpack attack will spawn in the direction towards a random enemy on screen. Subsequent jetpacks will be offset from that initial angle by the angle defined in this field.

```
public float DegreeAngleBetweenAttacks
```

Field Value

float

Remarks

Authored in degrees for ease of use, stored in radians as that is what unity expects for the quaternion.Euler() method.

Struct JetpackWeaponData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data relevant to jetpack weapon.

```
public struct JetpackWeaponData : IComponentData, IQueryTypeParameter
```

Fields

RadianAngleBetweenAttacks

An offset to apply to the angle at which a jetpack attack spawns in a group. Value will be multiplied by its spawn index in the attack group.

```
public float RadianAngleBetweenAttacks
```

Field Value

float

Remarks

Authored in degrees for ease of use, stored in radians as that is what unity expects for the quaternion.Euler() method.

RadianAngleToFirstEnemy

Stores the angle towards the first enemy the weapon will be firing a jetpack towards. Subsequent jetpacks will fire at an offset from this angle.

```
public float RadianAngleToFirstEnemy
```

Field Value

float

Remarks

As this value will change with each attack group, normally I would store something like this in a separate "state" component. However, as this component is only used in the JetpackAttackSystem I don't have any concern over potential data dependency issues so there is no problem having this field in here too.

See Also

JetpackWeaponAuthoring
JetpackAttackSystem

Class KnockbackAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components required for knockback

```
public class KnockbackAuthoring : MonoBehaviour
```

Remarks

Add this to an entity that will receive knockback from an entity with KnockbackOnInteractionData.

Fields

KnockbackTakenMultiplier

Multiplier to change the strength of knockback received.

```
public float KnockbackTakenMultiplier
```

Field Value

float

See Also

KnockbackState
KnockbackTakenMultiplier
KnockbackSystem
KnockbackOnInteractionSystem

Class KnockbackOnInteractionAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add KnockbackOnInteractionData to an entity.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
public class KnockbackOnInteractionAuthoring : MonoBehaviour
```

Remarks

Add this to an entity that will give knockback force to an entity with KnockbackState and KnockbackTakenMultiplier.

Fields

Strength

```
public float Strength
```

Field Value

float

Struct KnockbackOnInteractionData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data related to knockback force to give.

```
public struct KnockbackOnInteractionData : IComponentData, IQueryTypeParameter
```

Fields

Strength

Strength of the knockback.

```
public float Strength
```

Field Value

float

See Also

KnockbackState
KnockbackTakenMultiplier
KnockbackSystem
KnockbackOnInteractionSystem

Struct KnockbackOnInteractionSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to apply knockback force to an entity.

```
[UpdateInGroup(typeof(DS_InteractionSystemGroup))]
public struct KnockbackOnInteractionSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

KnockbackState
KnockbackTakenMultiplier
KnockbackSystem
KnockbackOnInteractionData

Struct KnockbackState

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to hold current state to knockback.

```
public struct KnockbackState : IComponentData, IQueryTypeParameter, IEnableableComponent
```

Remarks

Enableable component - when enabled, the entity is currently being moved by knockback.

Fields

Direction

Direction of the knockback.

```
public float2 Direction
```

Field Value

float2

EndTimestamp

Timestamp at which the knockback will expire.

```
public float EndTimestamp
```

Field Value

float

Strength

Strength of the knockback.

```
public float Strength
```

Field Value

float

Struct KnockbackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to apply knockback force to an entity.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct KnockbackSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

KnockbackState
KnockbackTakenMultiplier
KnockbackOnInteractionSystem

Struct KnockbackTakenMultiplier

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Multiplier to change the strength of knockback received.

```
public struct KnockbackTakenMultiplier : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public float Value
```

Field Value

float

Struct LaserStrikeAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of laser strikes into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct LaserStrikeAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Laser strikes are affected by modifications to the player's additional attack projectiles, damage dealt, and attack area stat modifications. Laser strikes have an area of effect damaging radius indicated by the circular shaped strike at the bottom of the attack. The vertical laser column is for visual purposes only. System will target a random entity on screen and strike it with a laser. If the randomly selected enemy has already been hit with a laser during a group of laser strikes, a new random enemy will be selected. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem. The idiomatic foreach in this system is unique as it has a large number of component types in the query - it has 7 which is the limit for these types of queries. As such, rather than using EnabledRefRW with type WeaponActiveFlag as I do in many other attack systems, I've moved the WeaponActiveFlag to the WithAll portion of the query, included WithEntityAccess, then disable the component using SystemAPI at the end of the foreach loop.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class LaserStrikeEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for laser strike in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class LaserStrikeEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime

Class LaserStrikeWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for executing laser strike attacks.

```
[RequireComponent(typeof(WeaponAuthoring))]
[RequireComponent(typeof(EntityRandomAuthoring))]
public class LaserStrikeWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity. Requires the EntityRandomAuthoring script for random number generation in JetpackAttackSystem.

Fields

InnerPadding

This value is used when determining a random enemy to strike in the LaserStrikeAttackSystem as a way to select targets inside the camera bounds plus some additional inner padding. This is so laser strikes occur fully on screen and not cut off by screen edges.

```
public float InnerPadding
```

Field Value

float

Struct LaserStrikeWeaponData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component containing data related to the laser strike weapon attack.

```
public struct LaserStrikeWeaponData : IComponentData, IQueryTypeParameter
```

Fields

InnerPadding

This value is used when determining a random enemy to strike in the LaserStrikeAttackSystem as a way to select targets inside the camera bounds plus some additional inner padding. This is so laser strikes occur fully on screen and not cut off by screen edges.

```
public float InnerPadding
```

Field Value

float

Struct LevelElementData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer data for level elements. Level elements are decoration and environment entities that will spawn in the level.

```
public struct LevelElementData : IBufferElementData
```

Fields

MaxScale

Maximum scale value to set this entity at when spawning.

```
public float MaxScale
```

Field Value

float

Remarks

During spawning, a random scale value will be selected between min and max to add some additional randomness to spawning.

MinScale

Minimum scale value to set this entity at when spawning.

```
public float MinScale
```

Field Value

float

Remarks

During spawning, a random scale value will be selected between min and max to add some additional randomness to spawning.

Rarity

Rarity value for how frequently this item will be spawned.

```
public int Rarity
```

Field Value

int

Remarks

A rarity value of 0 will never be selected for spawning while a rarity value of 100 will have the highest likelihood of being selected to be spawned.

Value

Entity prefab that can be spawned via level generation.

```
public Entity Value
```

Field Value

Entity

See Also

DecorationTag
EnvironmentTag

Struct LevelElementInfo

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data struct used to author a collection of level elements in the Unity editor.

```
[Serializable]
public struct LevelElementInfo
```

Fields

MaxScale

Maximum scale value to set this entity at when spawning.

```
public float MaxScale
```

Field Value

float

Remarks

During spawning, a random scale value will be selected between min and max to add some additional randomness to spawning.

MinScale

Minimum scale value to set this entity at when spawning.

```
public float MinScale
```

Field Value

float

Remarks

During spawning, a random scale value will be selected between min and max to add some additional randomness to spawning.

Prefab

GameObject prefab for the entity that will be spawned by level generation.

```
public GameObject Prefab
```

Field Value

GameObject

Rarity

Rarity value for how frequently this item will be spawned.

```
[Range(0, 100)]
public int Rarity
```

Field Value

int

Remarks

A rarity value of 0 will never be selected for spawning while a rarity value of 100 will have the highest likelihood of being selected to be spawned.

Class LevelGenerationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for level generation.

```
public class LevelGenerationAuthoring : MonoBehaviour
```

Remarks

This script uses the OnValidate method to determine when the seed value has changed in the editor, which will trigger a regeneration of the level. This regeneration occurs both in and out of playmode as the LevelGenerationSystem is set to update in both Editor and Default worlds.

Fields

BackgroundPrefab

GameObject prefab of the background entity that will be spawned.

```
public GameObject BackgroundPrefab
```

Field Value

GameObject

CautionPrefab

GameObject prefab of the caution tape entity that will be spawned around the edge of the play area.

```
public GameObject CautionPrefab
```

Field Value

GameObject

ItemCount

Number of decoration and environment items to spawn in the play area.

```
public int ItemCount
```

Field Value

int

LevelElements

Collection of level elements that store data about entities to be spawned during level generation.

```
public LevelElementInfo[] LevelElements
```

Field Value

LevelElementInfo[]

See Also
LevelElementData

LevelSize

Level is generated in a square centered at the origin with length and height defined by this level size.

```
public float LevelSize
```

Field Value

float

Seed

Random seed to generate level data from.

```
public uint Seed
```

Field Value

uint

UnitsPerBackgroundTile

Scaling factor used to determine the number of units per background tile on the background prefab.

```
public float UnitsPerBackgroundTile
```

Field Value

float

Remarks

Note that the background entity is a single quad mesh for the entire play area and tiling is set in the shader via the TileScaleOverride component.

Struct LevelGenerationData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data related to level generation.

```
public struct LevelGenerationData : IComponentData, IQueryTypeParameter
```

Fields

BackgroundPrefab

Entity prefab for the background entity - ground surface.

```
public Entity BackgroundPrefab
```

Field Value

Entity

CautionPrefab

Entity prefab for the caution tape graphics entity that is displayed around the far edges of the play area.

```
public Entity CautionPrefab
```

Field Value

Entity

ItemCount

Number of decoration and environment items to spawn in the play area.

```
public int ItemCount
```

Field Value

int

LevelSize

Level is generated in a square centered at the origin with length and height defined by this level size.

```
public float LevelSize
```

Field Value

float

MaxRarity

Each level element has a rarity component to influence how often it will be spawned in the world. This value is the sum of all rarity values on all level elements used for random number generation to determine what level element should spawn next.

```
public int MaxRarity
```

Field Value

int

UnitsPerBackgroundTile

Scaling factor used to determine the number of units per background tile on the background prefab.

```
public float UnitsPerBackgroundTile
```

Field Value

float

Remarks

Note that the background entity is a single quad mesh for the entire play area and tiling is set in the shader via the TileScaleOverride component.

See Also

LevelGenerationSystem

Struct LevelGenerationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to randomly generate a level with decoration and environment entities.

```
[WorldSystemFilter(WorldSystemFilterFlags.Default|WorldSystemFilterFlags.Editor,
WorldSystemFilterFlags.Default)]
[UpdateInGroup(typeof(InitializationSystemGroup))]
public struct LevelGenerationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system utilizes WorldSystemFilter flags to determine which worlds this system should run in. This system will run in the editor world so changes to level generation data can be seen and experimented with outside playmode. This system will also run during playmode to ensure the level is created as these entities will not be baked into the subscene. This system will first destroy any entities previously generated by this level generator. To do this, a native array of entities is first generated from the query, this is to effectively "capture" the entities to be destroyed and will not destroy any entities spawned later in the system as the default behavior of EntityCommandBuffer operations on EntityQueries is to evaluate the query at playback time. Next it will spawn the background entity and set the size and graphics tiling as needed. It will next do something similar with the caution tape around the bounds of the play area. Finally, all the level elements will be randomly selected and placed randomly around the map.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct LevelSafeBounds

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component used to store the safe bounds of the play area.

```
public struct LevelSafeBounds : IComponentData, IQueryTypeParameter
```

Remarks

Used by the DamagePlayerOutsideSafeBoundsSystem to apply damage to the player when they have left the play area.

Fields

MaxPosition

```
public float2 MaxPosition
```

Field Value

float2

MinPosition

```
public float2 MinPosition
```

Field Value

float2

Methods

InsideSafeBounds(float2)

```
public bool InsideSafeBounds(float2 testPosition)
```

Parameters

`testPosition` float2

Returns

bool

Class LevelUpUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the level up UI that is displayed when the player levels up.

```
public class LevelUpUIController : MonoBehaviour
```

Remarks

Will spawn instances of UI elements that display info about the capability to be leveled up. Capability UpgradeUIController helps display this info and invoke the proper methods when upgrade is selected.

Fields

Instance

```
public static LevelUpUIController Instance
```

Field Value

LevelUpUIController

Methods

HideLevelUpUI(bool)

```
public void HideLevelUpUI(bool resumeGame = true)
```

Parameters

`resumeGame` bool

Class LinearMovementAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add LinearMovementSpeed component to an entity, so it will constantly move along its forward axis in the LinearMovementSystem

```
public class LinearMovementAuthoring : MonoBehaviour
```

Fields

MoveSpeed

```
public float MoveSpeed
```

Field Value

float

Struct LinearMovementSpeed

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store speed of linear movement. Linear movement is when an entity will move along its forward axis constantly at the speed defined here.

```
public struct LinearMovementSpeed : IComponentData, IQueryTypeParameter
```

Remarks

Used for lots of player attack projectiles to move them along their forward facing axis.

Fields

Value

```
public float Value
```

Field Value

float

See Also

LinearMovementAuthoring
LinearMovementSystem

Struct LinearMovementSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to move an entity along its forward axis, with data defined in LinearMovementSpeed.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct LinearMovementSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class LinearScaleTransformationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add LinearScaleTransformationData to an entity.

```
public class LinearScaleTransformationAuthoring : MonoBehaviour
```

Remarks

Note that this does not add the LinearScaleTransformationEndTimestamp component which is also required to perform the transformation. This will be initialized and added in the LinearScaleTransformationSystem

Fields

Duration

Duration the scale transformation will take to complete in seconds.

```
public float Duration
```

Field Value

float

EndScale

End uniform scale value.

```
public float EndScale
```

Field Value

float

StartScale

Starting uniform scale value.

```
public float StartScale
```

Field Value

float

Struct LinearScaleTransformationData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data related to linear scale transformation. Linear scale transformation is when the scale of an entity changes from the start scale to the end scale in the duration listed.

```
public struct LinearScaleTransformationData : IComponentData, IQueryTypeParameter
```

Fields

Duration

Duration the scale transformation will take to complete in seconds.

```
public float Duration
```

Field Value

float

EndScale

End uniform scale value.

```
public float EndScale
```

Field Value

float

StartScale

Starting uniform scale value.

```
public float StartScale
```

Field Value

float

See Also

LinearScaleTransformationEndTimestamp
LinearScaleTransformationSystem
LinearScaleTransformationAuthoring

Struct LinearScaleTransformationEndTimestamp

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to hold the end timestamp of when the linear scale transformation will end.

```
public struct LinearScaleTransformationEndTimestamp : IComponentData,
IQueryTypeParameter, IEnableableComponent
```

Fields

Value

```
public float Value
```

Field Value

float

See Also

LinearScaleTransformationData
LinearScaleTransformationSystem

Struct LinearScaleTransformationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to perform the linear transformation from the data defined in LinearScaleTransformationData.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct LinearScaleTransformationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system will first initialize and add the LinearScaleTransformationEndTimestamp component as that is required to calculate the scale transformation.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

LinearScaleTransformationEndTimestamp
LinearScaleTransformationAuthoring

Class LoadingScreenUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller for the loading screen that is displayed when switching between title scene and gameplay scenes.

```
public class LoadingScreenUIController : MonoBehaviour
```

Remarks

Marked as don't destroy on load so it can be shown/hidden across scene bounds.

Fields

Instance

```
public static LoadingScreenUIController Instance
```

Field Value

LoadingScreenUIController

Methods

HideLoadingScreen()

```
public void HideLoadingScreen()
```

ShowLoadingScreen()

```
public void ShowLoadingScreen()
```

Struct LoopingAudioSourceIndex

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to store the index of the looping audio source in TMG.DOTSSurvivors.GameAudioController._loopingAudioSources.

```
public struct LoopingAudioSourceIndex : ICleanupComponentData,
IComponentData, IQueryTypeParameter
```

Remarks

Cleanup component will call StopLoopingAudioClip(int) when the associated entity is destroyed to stop the looping sound effect.

Fields

AudioClipIndex

```
public int AudioClipIndex
```

Field Value

int

Class MainMenuAudioController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

MonoBehaviour for controlling audio in main menu.

```
public class MainMenuAudioController : MonoBehaviour
```

Fields

Instance

Public singleton access to play audio in main menu.

```
public static MainMenuAudioController Instance
```

Field Value

MainMenuAudioController

Methods

GetNormalizedMusicLevel()

Method to return the normalized music volume level.

```
public float GetNormalizedMusicLevel()
```

Returns

float

Normalized music volume level from 0 and 1

Remarks

If volume level is lower than the defined minimum value, 0 will be returned as the music volume can be considered muted. This is used to set the music volume slider in UI configuration based on its current values.

GetNormalizedSfxLevel()

Method to return the normalized sound effects volume level.

```
public float GetNormalizedSfxLevel()
```

Returns

float

Normalized sound effects volume level from 0 and 1

Remarks

If volume level is lower than the defined minimum value, 0 will be returned as the sound effects volume can be considered muted. This is used to set the sound effects volume slider in UI configuration based on its current values.

PlaySfxAudioClip(AudioClip, int)

Method for playing a normal sound effects clip.

```
public void PlaySfxAudioClip(AudioClip audioClip, int priority = 128)
```

Parameters

`audioClip` AudioClip

Audio clip to play.

`priority` int

Priority of the sound effect. AudioPriority

SetMusicVolume(float)

Sets the music volume based on normalized value from UI slider.

```
public void SetMusicVolume(float normalizedLevel)
```

Parameters

`normalizedLevel` float

Value from 0 to 1 for the music volume.

Remarks

If normalized value is below the TMG.DOTSSurvivors.MainMenuAudioController.MIN_CUTOFF_LEVEL, the music volume will be set to -80dB, effectively muting the channel.

SetSfxVolume(float)

Sets the sound effects volume based on normalized value from UI slider.

```
public void SetSfxVolume(float normalizedLevel)
```

Parameters

`normalizedLevel` float

Value from 0 to 1 for the sound effects volume.

Remarks

If normalized value is below the TMG.DOTSSurvivors.MainMenuAudioController.MIN_CUTOFF_LEVEL, the sound effects volume will be set to -80dB, effectively muting the channel.

See Also

GameAudioController

Class ManagedGameData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Managed data component to store managed types required for gameplay.

```
public class ManagedGameData : IComponentData, IQueryTypeParameter
```

Fields

EnemyEntityPrefabLookup

Dictionary lookup to retrieve the entity prefab of an enemy by its EnemyType.

```
public Dictionary<EnemyType, Entity> EnemyEntityPrefabLookup
```

Field Value

Dictionary<EnemyType, Entity>

ExperienceGemColorLookup

List of (int, float4) to look up the color of an experience gem at a given experience point value.

```
public List<(int, float4)> ExperienceGemColorLookup
```

Field Value

List<(int, float4)>

Remarks

Similar to ExperiencePointColor, int is the minimum experience point value for the associated color, and float4 is the color value.

HealthBonusItem

BonusItemProperties for granting the player health.

```
public BonusItemProperties HealthBonusItem
```

Field Value

BonusItemProperties

MoneyBonusItem

BonusItemProperties for granting the player money.

```
public BonusItemProperties MoneyBonusItem
```

Field Value

BonusItemProperties

PlayerWorldUICanvasPrefab

GameObject prefab of the world space UI canvas that should follow the player to display their health bar.

```
public GameObject PlayerWorldUICanvasPrefab
```

Field Value

GameObject

WeaponEntityPrefabLookup

Dictionary lookup to retrieve the entity prefab of a weapon by its WeaponType enum.

```
public Dictionary<WeaponType, Entity> WeaponEntityPrefabLookup
```

Field Value

Dictionary<WeaponType, Entity>

Struct MoveCameraSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

This system updates the position of the GameObject camera target to match that of the player's position plus an additional offset along the y-axis.

```
[UpdateAfter(typeof(TransformSystemGroup))]
public struct MoveCameraSystem : ISystem, ISystemCompilerGenerated
```

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct OnDamageParticleSystemPrefab

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

GameObject prefab for the particle system that should play when this entity takes damage.

```
public struct OnDamageParticleSystemPrefab : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public UnityObjectRef<GameObject> Value
```

Field Value

UnityObjectRef<GameObject>

See Also

PlayParticleSystemOnDamage

Struct OxygenHoseAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of oxygen hoses into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct OxygenHoseAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Oxygen hoses are affected by modifications to the player's additional attack projectiles, damage dealt, and attack area stat modifications. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class OxygenHoseEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for oxygen hose in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class OxygenHoseEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime

Class OxygenHoseWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add OxygenHoseWeaponData to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
public class OxygenHoseWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity.

Fields

YOffset

An offset to apply along the y-axis for oxygen hose attack spawns in a group. Value will be multiplied by its spawn index in the attack group.

```
public float YOffset
```

Field Value

float

Struct OxygenHoseWeaponData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data relevant to spawning oxygen hose attacks into the game world.

```
public struct OxygenHoseWeaponData : IComponentData, IQueryTypeParameter
```

Fields

YOffset

An offset to apply along the y-axis for oxygen hose attack spawns in a group. Value will be multiplied by its spawn index in the attack group.

```
public float YOffset
```

Field Value

float

Class ParabolicMovementAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to define initial values for ParabolicMovementState.

```
public class ParabolicMovementAuthoring : MonoBehaviour
```

Remarks

Note that this script does not initialize StartTime as this value will not be known at bake time, so it is important that this value is set during instantiation of the entity.

Fields

BaseForce

Additional force to be applied to the initial velocity.

```
public float BaseForce
```

Field Value

float

StartingAngle

Starting angle in degrees of the parabolic movement. From this a float2 value for initial velocity will be calculated.

```
public float StartingAngle
```

Field Value

float

Struct ParabolicMovementState

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to hold current state of parabolic movement.

```
public struct ParabolicMovementState : IComponentData, IQueryTypeParameter
```

Fields

StartTime

Holds the start timestamp of when the entity was spawned into the world. Used to calculate the velocity parabola the entity will follow.

```
public double StartTime
```

Field Value

double

Velocity

Holds the velocity of the object. Value along the x-axis will remain constant through the lifetime of the entity. Value along the y-axis will update with the current velocity so velocity for the next frame can be calculated.

```
public float2 Velocity
```

Field Value

float2

Struct ParabolicMovementSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to apply parabolic movement to entities. Parabolic movement gives the effect that an entity will move towards the bottom of the screen as if screen-space gravity is being applied.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct ParabolicMovementSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System update in the DS_TranslationSystemGroup which updates before Unity's TransformSystemGroup so the LocalTransform component can be safely modified.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class PassiveEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add required components to passive entity.

```
public class PassiveEntityAuthoring : MonoBehaviour
```

Remarks

PassiveLevelIndex is initialized to a value of -1 as passive will be upgraded after instantiation as UpgradePassiveFlag will be enabled by default.

See Also

UpgradePassiveSystem

Struct PassiveLevelIndex

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Current level index of the passive ability. Level 1 (lowest level) is index 0.

```
public struct PassiveLevelIndex : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public int Value
```

Field Value

int

Struct PassiveLevelInfo

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data struct used to store StatModifier array and description of a passive item at a single level of the item's upgrade path.

```
[Serializable]
public struct PassiveLevelInfo
```

Remarks

Has the System.Serializable attribute so these values can be initialized in the editor via PassiveUpgrade Properties.

Fields

Description

Description of the passive item at a specific level.

```
public string Description
```

Field Value

string

StatModifiers

Array of StatModifiers that will be added to the player's CharacterStatModificationState when the level of this passive item is selected.

```
public StatModifier[] StatModifiers
```

Field Value

StatModifier[]

Remarks

Gets added indirectly via ActiveStatModifierEntity in RecalculateStatsSystem.

Class PassiveUpgradeProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject to define the properties for a passive item and its upgrade path.

```
[CreateAssetMenu(fileName = "PassiveUpgradeProperties", menuName =
"ScriptableObjects/Passive Upgrade Properties")]
public class PassiveUpgradeProperties : UpgradeProperties
```

Remarks

Inherits from UpgradeProperties.

Fields

UpgradeProperties

Array to store StatModifiers and descriptions for each level in the passive item's upgrade path.

```
public PassiveLevelInfo[] UpgradeProperties
```

Field Value

PassiveLevelInfo[]

Properties

MaxLevelIndex

Property to return the maximum level index for the passive item.

```
public override int MaxLevelIndex { get; }
```

Property Value

int

Methods

GetDescription(int)

Gets the description of the passive item upgrade at a given level.

```
public override string GetDescription(int levelIndex)
```

Parameters

`levelIndex` int

Index of the level to get the description from. Index 0 for level 1, index 1 for level 2, and so on.

Returns

string

Description of the attack upgrade at the given level.

Struct PassiveUpgradePropertiesReference

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to reference the associated PassiveUpgradeProperties via UnityObjectRef.

```
public struct PassiveUpgradePropertiesReference : IComponentData, IQueryTypeParameter
```

Fields

Value

UnityObjectRef to the PassiveUpgradeProperties for this passive ability.

```
public UnityObjectRef<PassiveUpgradeProperties> Value
```

Field Value

UnityObjectRef<PassiveUpgradeProperties>

Properties

this[int]

Custom indexer to retrieve the PassiveLevelInfo for a given level. Contains information about stats to modify.

```
public PassiveLevelInfo this[int i] { get; }
```

Parameters

`i` int

Index of the level to get PassiveLevelInfo of. Level 1 = index 0

Property Value

PassiveLevelInfo

Class PauseManager

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Singleton MonoBehaviour to control the pause state of the game.

```
public class PauseManager : MonoBehaviour
```

Fields

Instance

Public singleton access to this MonoBehaviour.

```
public static PauseManager Instance
```

Field Value

PauseManager

OnPauseGame

Event to be invoked when the game enters the paused state.

```
public Action OnPauseGame
```

Field Value

Action

Remarks

This triggers certain effects (i.e. sound effects) to enter a paused state as well.

OnResumeGame

Event to be invoked when the game exits the paused state and resumes the game.

```
public Action OnResumeGame
```

Field Value

Action

Remarks

This triggers certain effects (i.e. sound effects) to exit the paused state and resume their effects.

Properties

IsPaused

Boolean to determine if the game is currently in a paused state.

```
public bool IsPaused { get; }
```

Property Value

bool

Methods

PauseGame()

Pauses the game.

```
public void PauseGame()
```

Remarks

Disabled Unity's top-level system groups of InitializationSystemGroup and SimulationSystemGroup that all gameplay systems exist under, effectively pausing the game.

ResumeGame()

Resumes the game.

```
public void ResumeGame()
```

Remarks

Enables Unity's top-level system groups of InitializationSystemGroup and SimulationSystemGroup that all gameplay systems exist under, effectively resuming the game.

Class PauseMenuUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the pause menu that is displayed when the player pauses the game.

```
public class PauseMenuUIController : MonoBehaviour
```

Remarks

Displays more detailed information about the player's current game run, has some basic settings and ability to quit the game.

Class PersistentDataManager

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Singleton MonoBehaviour to handle saving, loading, and processing, persistent data relevant to gameplay.

```
public class PersistentDataManager : MonoBehaviour
```

Fields

Instance

Public singleton access to this MonoBehaviour.

```
public static PersistentDataManager Instance
```

Field Value

PersistentDataManager

OnUpdateCoinCount

Event to be invoked when the player's coin count is updated. This will trigger UI elements to update with the appropriate coin count.

```
public Action<int> OnUpdateCoinCount
```

Field Value

Action<int>

OnUpdateEnemiesDefeated

Event to be invoked when the player's alien defeated count is updated. This will trigger UI elements to update with the appropriate coin count.

```
public Action<int> OnUpdateEnemiesDefeated
```

Field Value

Action <int>

OnUpdateTimeSurvived

Event to be invoked when the player's time survived is updated. This will trigger UI elements to update with the appropriate coin count.

```
public Action<float> OnUpdateTimeSurvived
```

Field Value

Action <float>

Properties

CurrentCoinCount

Running count of coins the player has access to. Can gain coins through gameplay and spend coins on characters and stages.

```
public int CurrentCoinCount { get; }
```

Property Value

int

EnemiesDefeatedCount

Property to access the total number of enemies the player has defeated.

```
public int EnemiesDefeatedCount { get; }
```

Property Value

int

MusicVolumeLevel

Property to access the current music volume level in dB.

```
public float MusicVolumeLevel { get; }
```

Property Value

float

SfxVolumeLevel

Property to access the current sfx volume level in dB.

```
public float SfxVolumeLevel { get; }
```

Property Value

float

TotalTimeSurvived

Property to access the total time the player has survived in game in seconds.

```
public float TotalTimeSurvived { get; }
```

Property Value

float

Methods

AddEnemiesDefeated(int)

Adds to the player's TotalEnemiesDefeated count.

```
public void AddEnemiesDefeated(int additionalEnemiesDefeated)
```

Parameters

`additionalEnemiesDefeated` int

Additional number of enemies to add to the running total.

AddMoney(int)

Adds money to the player's CurrentCoinCount

```
public void AddMoney(int moneyToAdd)
```

Parameters

`moneyToAdd` int

Amount of money to add.

AddTimeSurvived(float)

Adds to the player's TotalTimeSurvived variable.

```
public void AddTimeSurvived(float additionalTimeSurvived)
```

Parameters

`additionalTimeSurvived` float

Time in seconds the player has survived to add to the running total.

DeleteData()

Deletes all persistent data and restores the game to the point as if the player just opened the game for the first time.

```
public void DeleteData()
```

Remarks

This is a destructive, irreversible action and the player is warned in the UI of such.

IsCharacterUnlocked(CharacterID)

Method to determine if a character is unlocked.

```
public bool IsCharacterUnlocked(CharacterID characterID)
```

Parameters

`characterID` CharacterID

ID of the character used to look at the bitflags in UnlockedCharacterFlags to determine if the character is unlocked.

Returns

bool

True if the character is unlocked, false if the character has not yet been unlocked by the player.

IsStageUnlocked(StageID)

Method to determine if a stage is unlocked.

```
public bool IsStageUnlocked(StageID stageID)
```

Parameters

`stageID` StageID

ID of the stage used to look at the bitflags in UnlockedStageFlags to determine if the stage is unlocked.

Returns

bool

True if the stage is unlocked, false if the stage has not yet been unlocked by the player.

SaveMusicVolume(float)

Saves the music volume level, stored in dB.

```
public void SaveMusicVolume(float volumeLevel)
```

Parameters

`volumeLevel` float

Volume level to save in dB.

SaveSfxVolume(float)

Saves the sound effects volume level, stored in dB.

```
public void SaveSfxVolume(float volumeLevel)
```

Parameters

`volumeLevel` float

Volume level to save in dB.

TryBuyCharacter(int, CharacterID)

Method to attempt to purchase access to a character.

```
public bool TryBuyCharacter(int cost, CharacterID characterID)
```

Parameters

`cost` int

Cost of the character.

`characterID` CharacterID

ID of the character, used to set the bitflag for the character on UnlockedCharacterFlags if the character becomes unlocked.

Returns

bool

True if the character was successfully purchased, false if the player doesn't have enough money to purchase the character.

Remarks

If the character was successfully purchased, persistent fields for CurrentCoinCount and UnlockedCharacterFlags will be updated accordingly.

TryBuyStage(int, StageID)

Method to attempt to purchase access to a stage.

```
public bool TryBuyStage(int cost, StageID stageID)
```

Parameters

`cost` int

Cost of the stage.

`stageID` StageID

ID of the stage, used to set the bitflag for the stage on UnlockedStageFlags if the stage becomes unlocked.

Returns

bool

True if the stage was successfully purchased, false if the player doesn't have enough money to purchase the stage.

Remarks

If the stage was successfully purchased, persistent fields for CurrentCoinCount and UnlockedStageFlags will be updated accordingly.

UnlockArtTestScene()

```
public void UnlockArtTestScene()
```

Struct PersistentGameData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Persistent game data that will be saved to disk.

```
[Serializable]
public struct PersistentGameData
```

Fields

CurrentCoinCount

Running count of coins the player has access to. Can gain coins through gameplay and spend coins on characters and stages.

```
public int CurrentCoinCount
```

Field Value

int

Remarks

Value is shown to the player in the title screen.

MusicVolumeLevel

Current music volume level stored in dB.

```
public float MusicVolumeLevel
```

Field Value

float

PersistentGameDataVersion

Version number of the save file.

```
public int PersistentGameDataVersion
```

Field Value

int

Remarks

The PersistentDataManager can use this version number to upgrade the save file to the current version.

SfxVolumeLevel

Current SFX volume level stored in dB.

```
public float SfxVolumeLevel
```

Field Value

float

TotalEnemiesDefeated

Total number of enemies defeated.

```
public int TotalEnemiesDefeated
```

Field Value

int

Remarks

Value is shown to the player in the title screen.

TotalTimeSurvived

Total time in seconds the player has survived across all gameplay sessions.

```
public float TotalTimeSurvived
```

Field Value

float

Remarks

Value is shown to the player in the title screen.

UnlockedCharacterFlags

Byte flags to track which characters have been unlocked.

```
public byte UnlockedCharacterFlags
```

Field Value

byte

Remarks

Each character is assigned a bit that is assigned in CharacterID.

UnlockedStageFlags

Byte flags to track which stages have been unlocked.

```
public byte UnlockedStageFlags
```

Field Value

byte

Remarks

Each stage is assigned a bit that is assigned in StageID.

Class PhysicsHelper

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Static helper class used to implement helpers not present in Unity's API.

```
public static class PhysicsHelper
```

Methods

GetCollisionMaskForLayer(int)

Static method to return the 32-bit collision layer bitmask that will have bits set to 1 for the indices of layers it can collide with.

```
public static int GetCollisionMaskForLayer(int layer)
```

Parameters

`layer` int

Index of the layer to get the collision bitmask for.

Returns

int

Returns the 32-bit collision layer bitmask for all the layers this layer collides with.

Remarks

Typically used to create a CollisionFilter type which can be used in raycasts and spatial queries.

Struct PlasmaBlastAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of plasma blasts into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct PlasmaBlastAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Plasma blasts are affected by modifications to the player's additional attack projectiles, damage dealt, attack speed, and attack duration stat modifications. Plasma blasts shoot towards the closest enemy to the player. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class PlasmaBlastEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for plasma blast in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class PlasmaBlastEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime
LinearMovementSpeed
DestroyAfterNumberHits

Class PlasmaBlastWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add PlasmaBlastWeaponTag to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
public class PlasmaBlastWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity.

Struct PlasmaBlastWeaponTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity as the weapon entity to spawn plasma blasts into the game world.

```
public struct PlasmaBlastWeaponTag : IComponentData, IQueryTypeParameter
```

Class PlayAudioClipOnDamageAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components to entity required for playing audio clips when taking damage.

```
public class PlayAudioClipOnDamageAuthoring : MonoBehaviour
```

Fields

AudioClip

Audio clip to play when this entity takes damage.

```
public AudioClip AudioClip
```

Field Value

AudioClip

Priority

Priority of the audio clip to play. Lower values have a higher priority.

```
public AudioPriority Priority
```

Field Value

AudioPriority

Struct PlayAudioClipOnDamageData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component for defining the audio clip to play when the associated entity takes damage.

```
public struct PlayAudioClipOnDamageData : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

Enableable component - when enabled, the audio clip will play.

Fields

AudioClip

Audio clip to play when this entity takes damage.

```
public UnityObjectRef<AudioClip> AudioClip
```

Field Value

UnityObjectRef<AudioClip>

Priority

Priority of the audio clip to play. Lower values have a higher priority

```
public byte Priority
```

Field Value

byte

See Also

PlayAudioClipOnDamageSystem

Struct PlayAudioClipOnDamageSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System for playing audio clips when an entity takes damage.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct PlayAudioClipOnDamageSystem : ISystem, ISystemCompilerGenerated
```

Remarks

ProcessDamageThisFrameSystem will enable the PlayAudioClipOnDamageData component if the entity takes damage. System updates in the DS_EffectsSystemGroup which executes towards the end of the frame after ProcessDamageThisFrameSystem has ran for the frame.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class PlayAudioClipOnDestroyAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize PlayAudioClipOnDestroyData.

```
public class PlayAudioClipOnDestroyAuthoring : MonoBehaviour
```

Fields

AudioClip

Audio clip to play when this entity is destroyed.

```
public AudioClip AudioClip
```

Field Value

AudioClip

Priority

Priority of the audio clip to play. Lower values have a higher priority.

```
public AudioPriority Priority
```

Field Value

AudioPriority

See Also

DestroyEntitySystem

Struct PlayAudioClipOnDestroyData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component holding the audio clip that will be played when the associated entity is destroyed.

```
public struct PlayAudioClipOnDestroyData : ICleanupComponentData,
IComponentData, IQueryTypeParameter
```

Remarks

As this is a cleanup component, it cannot be added to the entity through baking, so the InitializePlayAudioClipOnDestroyData is used to set this up.

Fields

AudioClip

Audio clip to play when this entity is destroyed.

```
public UnityObjectRef<AudioClip> AudioClip
```

Field Value

UnityObjectRef<AudioClip>

Priority

Priority of the audio clip to play. Lower values have a higher priority.

```
public byte Priority
```

Field Value

byte

See Also

DestroyEntitySystem

Struct PlayAudioClipOnDestroySystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle playing audio clip when entity is destroyed.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct PlayAudioClipOnDestroySystem : ISystem, ISystemCompilerGenerated
```

Remarks

As PlayAudioClipOnDestroyData is a cleanup component, it cannot be baked into a prefab, so we also need to initialize this component with InitializePlayAudioClipOnDestroyData shortly after the entity is spawned.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class PlayAudioClipOnSpawnAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add necessary components for playing audio clips when the entity is spawned.

```
public class PlayAudioClipOnSpawnAuthoring : MonoBehaviour
```

Fields

AudioClip

Audio clip to play when this entity is spawned.

```
public AudioClip AudioClip
```

Field Value

AudioClip

Priority

Priority of the audio clip to play. Lower values have a higher priority.

```
public AudioPriority Priority
```

Field Value

AudioPriority

ShouldLoop

Boolean to denote this entity as an entity that should loop its audio for the duration of its lifecycle.

```
public bool ShouldLoop
```

Field Value

bool

Struct PlayAudioClipOnSpawnData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component holding the audio clip that will be played when the associated entity is spawned.

```
public struct PlayAudioClipOnSpawnData : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Fields

AudioClip

Audio clip to play when this entity is spawned.

```
public UnityObjectRef<AudioClip> AudioClip
```

Field Value

UnityObjectRef<AudioClip>

Priority

Priority of the audio clip to play. Lower values have a higher priority

```
public byte Priority
```

Field Value

byte

Struct PlayAudioClipOnSpawnSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to play audio clips when an entity is spawned.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct PlayAudioClipOnSpawnSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Entities can be tagged with a ShouldLoopAudioClipTag which will loop the audio clip for the duration of its lifecycle.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

PlayAudioClipOnSpawnData
ShouldLoopAudioClipTag
LoopingAudioSourceIndex
GameAudioController
DestroyEntitySystem

Struct PlayParticleSystemOnDamage

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Cleanup component to hold a reference to the particle system that will play when this entity takes damage.

```
public struct PlayParticleSystemOnDamage : ICleanupComponentData,
IComponentData, IQueryTypeParameter
```

Remarks

As this is a cleanup component, this value will be initialized via the OnDamageParticleSystemPrefab as a GameObject instance will not be able to be baked into this component.

Fields

Value

```
public UnityObjectRef<ParticleSystem> Value
```

Field Value

UnityObjectRef<ParticleSystem>

Class PlayParticleSystemOnDamageAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for playing a particle system when an entity takes damage.

```
public class PlayParticleSystemOnDamageAuthoring : MonoBehaviour
```

Remarks

Authoring script should be attached to the root character entity.

Fields

ParticleSystemPrefab

GameObject prefab for the particle system that should play when this entity takes damage.

```
public GameObject ParticleSystemPrefab
```

Field Value

GameObject

Struct PlayParticleSystemOnDamageFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enableable component to signify this entity should begin playing the particle system as it is taking damage.

```
public struct PlayParticleSystemOnDamageFlag : IComponentData,
IQueryTypeParameter, IEnableableComponent
```

Struct PlayParticleSystemOnDamageSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to manage the lifecycle of particle systems that play when an entity takes damage. Has 3 foreach loops that do the following:

1. Instantiate an instance of the particle system prefab from OnDamageParticleSystemPrefab and store a reference to it in PlayParticleSystemOnDamage.
2. Sync the transform of the particle system to the entity's transform. If PlayParticleSystemOnDamage is enabled, it will play the particle system.
3. When the entity is destroyed, destroy the particle system GameObject.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct PlayParticleSystemOnDamageSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system updates in the DS_EffectsSystemGroup which updates towards the end of the frame. As such, transform is synchronized via the entity's LocalToWorld component as this will have been updated by this point in the frame.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class PlayerAnimationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize and add components required for player character animation.

```
[RequireComponent(typeof(CharacterAnimationAuthoring))]
public class PlayerAnimationAuthoring : MonoBehaviour
```

Remarks

This authoring script should be attached to the graphics entity which is a child of the main player entity. CharacterAnimationAuthoring is a required component to ensure components necessary for character animation are applied to the entity.

See Also

AnimationIndexOverride

Enum PlayerAnimationIndex

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enum to define the indices for player animations.

```
public enum PlayerAnimationIndex : byte
```

Fields

`Idle = 1`

Animation to player when character is not moving - i.e. idle animation

`Movement = 0`

Animation to play when player is moving - i.e. walk cycle

`None = 255`

Invalid animation

Class PlayerAttractionAreaAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize data for the player's attraction area.

```
public class PlayerAttractionAreaAuthoring : MonoBehaviour
```

Fields

PlayerEntity

Reference to the player entity. Used in systems such as AttractableItemSystem to determine any modifications to the attraction radius as defined by ItemAttractionRadius.

```
public GameObject PlayerEntity
```

Field Value

GameObject

Radius

Defines the radius of the attraction area.

```
public float Radius
```

Field Value

float

Struct PlayerAttractionAreaData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component containing the data related to the player's attraction area.

```
public struct PlayerAttractionAreaData : IComponentData, IQueryTypeParameter
```

Remarks

Attraction area is the area at which the player can pick up attractable items and experience gems.

Fields

CollisionFilter

Collision filter determines the types of colliders that trigger events will be raised with. i.e. this should raise trigger events with items and experience gems, but not enemies or environment elements.

```
public CollisionFilter CollisionFilter
```

Field Value

CollisionFilter

Radius

Defines the radius of the attraction area.

```
public float Radius
```

Field Value

float

Class PlayerAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components specific to the player entity.

```
[RequireComponent(typeof(CharacterAuthoring))]
public class PlayerAuthoring : MonoBehaviour
```

Struct PlayerExperienceState

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Current state of the player's experience.

```
public struct PlayerExperienceState : IComponentData, IQueryTypeParameter
```

Remarks

Experience points are granted in integers and the number of points required to reach the next level will always be an integer. Total experience points are stored as a float so that fractions of experience points can be accounted for. Player can achieve fractions of experience points through experience point modification (see ExperiencePointGain

Fields

Level

Level the player is on.

```
public int Level
```

Field Value

int

Remarks

Player starts the game at level 1 and level is incremented once TotalPoints is greater than or equal to PointsToNextLevel.

PointsToNextLevel

Total experience points required for the player to level up.

```
public int PointsToNextLevel
```

Field Value

int

TotalPoints

Total experience points the player has achieved in the current game run.

```
public float TotalPoints
```

Field Value

float

Struct PlayerExperienceThisFrame

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer to queue experience points to add to the player in a given frame.

```
[InternalBufferCapacity(1)]
public struct PlayerExperienceThisFrame : IBufferElementData
```

Remarks

Purpose of the dynamic buffer is so that experience points can potentially be granted from multiple threads and all be properly accounted for. This also leads to a single system (in this case HandlePlayerExperienceThisFrameSystem) for handling the addition of player experience to the player and performing any additional logic i.e. leveling up.

Fields

Value

```
public int Value
```

Field Value

int

See Also

HandlePlayerExperienceThisFrameSystem

Class PlayerInitializationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize values pertaining to player entities.

```
[UpdateInGroup(typeof(DS_InitializationSystemGroup))]
[UpdateBefore(typeof(CharacterInitializationSystem))]
public class PlayerInitializationSystem : SystemBase
```

Remarks

System is a SystemBase system so it can have System.Action events as member variables that can be invoked when UI elements need to be updated.

Fields

OnInitializePlayerLevel

Event to be invoked to initialize UI elements corresponding to the player's starting level. Arguments passed:

1. int - level the player is upgrading to
2. int - minimum experience points for the level the player is upgrading to
3. int - maximum experience points for the level the player is upgrading to

```
public Action<int, int, int> OnInitializePlayerLevel
```

Field Value

Action<int, int, int>

See Also

UpdatePlayerLevel(int, int, int)

Methods

OnCreate()

```
protected override void OnCreate()
```

OnUpdate()

```
protected override void OnUpdate()
```

Class PlayerStatStatusUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with individual stat elements that display the player's current modifiable stats.

```
public class PlayerStatStatusUIController : MonoBehaviour
```

Remarks

These stats account for modifications the character spawns with and ones modified via passive abilities. Depending on the stat type it will display as an absolute number, absolute difference, or percentage difference.

Methods

ShowUI(StatModifierProperties, float, float)

Displays the UI element associated with the current stat to be displayed.

```
public void ShowUI(StatModifierProperties modifierProperties, float defaultValue,
float currentValue)
```

Parameters

`modifierProperties` StatModifierProperties

ScriptableObject containing the data and assets associated with this stat.

`defaultValue` float

Default value of the stat. Used to compare with the current value to determine how the value should be displayed.

`currentValue` float

Current value of the stat. Compared with the default value for times when a difference from the default should be shown.

See Also

CurrentStatsUIController

Struct PlayerTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to identify this as the player entity.

```
public struct PlayerTag : IComponentData, IQueryTypeParameter
```

Remarks

Several systems and MonoBehaviours assume there is only one player entity in the game world at a time.

Class PlayerUpgradeData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Managed data component to store data related to player upgrades.

```
public class PlayerUpgradeData : IComponentData, IQueryTypeParameter
```

Fields

ActivePassiveEntityLookup

Dictionary to lookup active passive item entities by PassiveUpgradeProperties.

```
public Dictionary<PassiveUpgradeProperties, Entity> ActivePassiveEntityLookup
```

Field Value

Dictionary<PassiveUpgradeProperties, Entity>

Remarks

This is the passive item entity that is persistent in the world. Used to determine if the player already has a passive item, and if so what level it is on.

ActiveWeaponEntityLookup

Dictionary to lookup active weapon entities by WeaponUpgradeProperties.

```
public Dictionary<WeaponUpgradeProperties, Entity> ActiveWeaponEntityLookup
```

Field Value

Dictionary<WeaponUpgradeProperties, Entity>

Remarks

This is the weapon entity that is persistent in the world and spawns AttackPrefab instances. Used to determine if the player already has a weapon, and if so what level it is on.

MaxPassiveCount

Maximum number of passive items the player is allowed to have.

```
public int MaxPassiveCount
```

Field Value

int

MaxWeaponCount

Maximum number of weapons the player is allowed to have.

```
public int MaxWeaponCount
```

Field Value

int

TotalRandomPoints

Total random points allocated for random selection of upgrades.

```
public int TotalRandomPoints
```

Field Value

int

Remarks

This is effectively the highest WeightedRandomValue and could also be calculated by summing all Rarity values.

WeightedUpgradeProperties

Collection of WeightedUpgradeProperties. Used for selecting a random upgrade during level up and chest opening events.

```
public WeightedUpgradeProperties[] WeightedUpgradeProperties
```

Field Value

WeightedUpgradeProperties[]

Properties

HasFreePassiveSlots

Helper method to determine if the player has any free passive item slots.

```
public bool HasFreePassiveSlots { get; }
```

Property Value

bool

Remarks

If true, the player can obtain new passive items. If false, the player cannot obtain any new passive items as all the slots are full.

HasFreeWeaponSlots

Helper property to determine if the player has any free weapon slots.

```
public bool HasFreeWeaponSlots { get; }
```

Property Value

bool

Remarks

If true, the player can obtain new weapons. If false, the player cannot obtain any new weapons as all the slots are full.

Methods

GetRandomUpgradeProperties(ref Random)

Helper method to select a random UpgradeProperties in the WeightedUpgradeProperties array.

```
public UpgradeProperties GetRandomUpgradeProperties(ref Random random)
```

Parameters

`random` Random

Takes in a Unity.Mathematics.Random type used for random number generation. Passed in by reference so internal state can be updated to continually generate random values.

Returns

UpgradeProperties

The selected random upgrade property.

Remarks

First selects a random value between 0 and TotalRandomPoints. Next, each element of WeightedUpgrade Properties is iterated. The WeightedRandomValue is compared against the random value generated. If the random value is lower than current weighted value, the current UpgradeProperties will be returned as the random upgrade.

Struct PlayerWorldUI

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to hold Unity object references to the transform and slider for the player's health bar which exists on a world space canvas.

```
public struct PlayerWorldUI : ICleanupComponentData, IComponentData, IQueryTypeParameter
```

Fields

HealthBarSlider

UnityEngine.UI.Slider component of the player's health bar.

```
public UnityObjectRef<Slider> HealthBarSlider
```

Field Value

UnityObjectRef<Slider>

UICanvas

Transform of the world UI canvas that contains the player health bar.

```
public UnityObjectRef<Transform> UICanvas
```

Field Value

UnityObjectRef<Transform>

Struct PlayerWorldUISystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to initialize, update, and cleanup the player UI - used for player's in-world health bar.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct PlayerWorldUISystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system contains multiple foreach that do the following:

1. Instantiates an instance of the player world UI and sets up UnityObjectRef references in the Player WorldUI component.
2. Updates the position of the world UI instance to match that of the player, so the health bar appears to follow the player entity.
3. When the UpdatePlayerHealthUIFlag is enabled, update the current and max health values on the health bar slider UI element.
4. When the player is destroyed, destroy the player world UI instance so it no longer exists in the game world.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

PlayerWorldUI

Struct PreviousPlayerInput

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store the player's previous input.

```
public struct PreviousPlayerInput : IComponentData, IQueryTypeParameter
```

Fields

LastFacingDirection

Sign of player's last non-zero input on the x-axis.

```
public float LastFacingDirection
```

Field Value

float

Remarks

Used for attack and animation systems that need to know the player's facing direction. Only updates when x-input is non-zero. Will be -1 for left and +1 for right.

LastPositiveInput

Player's last non-zero input.

```
public float2 LastPositiveInput
```

Field Value

float2

Remarks

Used for firing attacks in the direction of the player's most recent input when not moving.

See Also

ScrewdriverAttackSystem

PreviousInput

Player's input from the previous frame.

```
public float2 PreviousInput
```

Field Value

float2

Remarks

This is required for the game to know when the player switches between moving and not moving states to change player's animation.

Struct ProcessDamageThisFrameSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to process damage accumulated in the DamageThisFrame dynamic buffer and apply that damage towards the entity's current hit points.

```
[UpdateAfter(typeof(DS_InteractionSystemGroup))]
public struct ProcessDamageThisFrameSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system accumulates total damage to be applied to the entity. Entities can receive negative damage hit points to heal. Value can be modified if an entity has any stat modifications (i.e. CharacterStat ModificationState. Current hit points will be clamped between 0 and max hit points (BaseHitPoints plus any modifiers to health). If the entity is out of hit points, this system will enable the character's Destroy EntityFlag to clean up the entity at the end of the frame. This system can also trigger VFX and SFX related to taking damage or healing. This system uses enableable components to trigger VFX and SFX so that this system can be fully burst compiled - it is important in this case as this system will be executing across all damageable entities (i.e. player, enemies, and damageable items in world).

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct RadioactiveWasteAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of radioactive waste drops into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct RadioactiveWasteAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Radioactive waste drops are affected by modifications to the player's additional attack projectiles, damage dealt, attack area, and attack duration stat modifications. The first radioactive waste drop will drop from the top of the screen towards the closest enemy to the player. Subsequent drops will spawn at a random location within a certain distance from the player. Radioactive waste drops do not deal damage themselves, however they will spawn a radioactive waste spill which has an area of effect damage and will stay in the game world for a certain period of time. As such certain data will be set on the drop entity and certain data will be set on the spill prefab entity to be spawned on destruction. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class RadioactiveWasteDropEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for radioactive waste drop in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class RadioactiveWasteDropEntityAuthoring : MonoBehaviour
```

Remarks

The radioactive waste drop entity is the initial entity that spawns from the top of the screen and "falls" to a determined position in the game world. This attack does not deal any damage, however once it reaches its target position, it will self-destruct and spawn a radioactive waste spill entity (see RadioactiveWasteSpillEntityAuthoring). Requires EntityInteractionAuthoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

Fields

RadioactiveWasteSpillPrefab

```
public GameObject RadioactiveWasteSpillPrefab
```

Field Value

GameObject

See Also

DestroyAtPosition
LinearMovementSpeed
SpawnOnDestroy

Class RadioactiveWasteSpillEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for oxygen hose in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class RadioactiveWasteSpillEntityAuthoring : MonoBehaviour
```

Remarks

Radioactive waste spill entity is spawned by the radioactive waste drop entity via its SpawnOnDestroy component. See RadioactiveWasteDropEntityAuthoring for more information. Requires EntityInteraction Authoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime

Class RadioactiveWasteWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add RadioactiveWasteWeaponData to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
[RequireComponent(typeof(EntityRandomAuthoring))]
public class RadioactiveWasteWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity. Requires the EntityRandomAuthoring script for random number generation in RadioactiveWasteAttack System.

Fields

MaxSpawnOffset

Radioactive waste drops after the first will spawn at a random position within a range of the player. This value denotes the maximum offset from the player the random position will be selected in.

```
public float3 MaxSpawnOffset
```

Field Value

float3

MinSpawnOffset

Radioactive waste drops after the first will spawn at a random position within a range of the player. This value denotes the minimum offset from the player the random position will be selected in.

```
public float3 MinSpawnOffset
```

Field Value

float3

Struct RadioactiveWasteWeaponData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data relevant to spawning radioactive waste drops attacks.

```
public struct RadioactiveWasteWeaponData : IComponentData, IQueryTypeParameter
```

Fields

MaxSpawnOffset

Radioactive waste drops after the first will spawn at a random position within a range of the player. This value denotes the maximum offset from the player the random position will be selected in.

```
public float3 MaxSpawnOffset
```

Field Value

float3

MinSpawnOffset

Radioactive waste drops after the first will spawn at a random position within a range of the player. This value denotes the minimum offset from the player the random position will be selected in.

```
public float3 MinSpawnOffset
```

Field Value

float3

Struct RandomItemDropInfo

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Serializable helper struct to facilitate Unity editor authoring of many of these elements contained in the ItemDrops List.

```
[Serializable]
public struct RandomItemDropInfo
```

Fields

Prefab

GameObject prefab to be dropped. In the SpawnRandomOnDestroyAuthoring.Baker this GameObject will be baked into an entity prefab.

```
public GameObject Prefab
```

Field Value

GameObject

Rarity

Determines how frequently an item will be dropped. Value of 100 gives the highest chance while value of 0 will never be chosen.

```
[Range(0, 100)]
[Tooltip("Determines how frequently an item will be dropped. Value of 100 gives the highest
chance while value of 0 will never be chosen.")]
public int Rarity
```

Field Value

int

Remarks

This is not a percentage chance, this is a relative chance, so if 4 elements each have a Rarity value of 100, they will each have a 25% chance of being dropped.

Class RandomItemDropProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject to define which items can be dropped when an entity destroyed.

```
[CreateAssetMenu(fileName = "RandomItemDropProperties", menuName = "ScriptableObjects/Random
Item Drop Properties")]
public class RandomItemDropProperties : ScriptableObject
```

Fields

ItemDrops

List of RandomItemDropInfo elements that are used to author which items can be dropped with their associated rarities via the Unity editor.

```
public List<RandomItemDropInfo> ItemDrops
```

Field Value

List<RandomItemDropInfo>

See Also

RandomItemToSpawnBlob
DestroyEntitySystem

Struct RandomItemToSpawn

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Struct to store information related to items that can be spawned when this entity is destroyed. Used as the base element of the RandomItemToSpawnBlob blob array.

```
public struct RandomItemToSpawn
```

Fields

Entity

```
public Entity Entity
```

Field Value

Entity

RarityKey

```
public int RarityKey
```

Field Value

int

See Also

RandomItemToSpawnBlob

Struct RandomItemToSpawnBlob

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component containing the blob array of RandomItemToSpawn elements. Used in the DestroyEntitySystem to determine which entity prefab to instantiate when the entity with this component is destroyed.

```
public struct RandomItemToSpawnBlob : IComponentData, IQueryTypeParameter
```

Remarks

Due to the nature of entity remapping from prefabs created during baking to prefabs available at runtime, this blob array must be populated with the required prefabs at runtime. To facilitate with this, prefabs that can be spawned are first baked into entities in the SpawnRandomOnDestroyAuthoring.Baker into a DynamicBuffer of SpawnRandomPrefabHelper. The InitializeSpawnRandomOnDestroySystem will populate this blob array with the appropriate data. This could be implemented as a cleanup component to avoid having the logic in DestroyEntitySystem, however I chose to have the logic held there so that we can also have an InstantDestroyEntitySystem which does not spawn additional entities. With this as a cleanup component, we'd need to specifically remove the component in InstantDestroyEntitySystem.

Fields

Value

```
public BlobAssetReference<BlobArray<RandomItemToSpawn>> Value
```

Field Value

BlobAssetReference<BlobArray<RandomItemToSpawn>>

See Also

RandomItemToSpawn
RandomItemDropProperties
DestroyEntitySystem

Struct RecalculateStatsFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enableable component to signify this entity needs their CharacterStatModificationState need to be recalculated in the RecalculateStatsSystem.

```
public struct RecalculateStatsFlag : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

See Also

UpgradePassive(PassiveUpgradeProperties)

Struct RecalculateStatsSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to recalculate the character's current stat modifications stored in CharacterStatModificationState.

```
[UpdateAfter(typeof(DS_InteractionSystemGroup))]
public struct RecalculateStatsSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Stat modification is implemented by iterating stat modification entities associated with a given character stored in the ActiveStatModifierEntity dynamic buffer on each character. Each active stat modifier entity has a StatModifier dynamic buffer that contains the individual stat modifications; this buffer will be iterated in an inner foreach loop and stats will be added to default stat modification values stored in CharacterDefaultModificationValues using the helper method AddStat(StatModifier). Updates after the DS_InteractionSystemGroup to all ensure stat modifications are accounted for before recalculating stats.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct RegenerateLevelFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enableable component to inform the LevelGenerationSystem that the level should be regenerated.

```
public struct RegenerateLevelFlag : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Class ResultsPanelUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the results panel that is displayed after the game over screen.

```
public class ResultsPanelUIController : MonoBehaviour
```

Methods

ShowResultsUI()

```
public void ShowResultsUI()
```

Struct SatelliteAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of satellites into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct SatelliteAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Satellites are affected by modifications to the player's additional attack projectiles, damage dealt, attack area, and attack projectile speed stat modifications. Satellites will spawn at an increasing angle towards the player's facing direction. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class SatelliteEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for satellite in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
[RequireComponent(typeof(DestroyOffCameraAuthoring))]
public class SatelliteEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring, DestructibleEntityAuthoring, and DestroyOffCameraAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
ParabolicMovementState
DestroyAfterNumberHits

Class SatelliteWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add SatelliteWeaponData to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
public class SatelliteWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity.

Fields

DegreeAngleBetweenAttacks

An offset to apply to the angle at which a satellite attack spawns in a group. Value will be multiplied by its spawn index in the attack group.

```
public float DegreeAngleBetweenAttacks
```

Field Value

float

Remarks

Authored in degrees for ease of use, stored in radians as that is what unity expects for the quaternion.Euler() method

Struct SatelliteWeaponData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data relevant to spawning satellite attacks.

```
public struct SatelliteWeaponData : IComponentData, IQueryTypeParameter
```

Fields

RadianAngleBetweenAttacks

An offset to apply to the angle at which a satellite attack spawns in a group. Value will be multiplied by its spawn index in the attack group.

```
public float RadianAngleBetweenAttacks
```

Field Value

float

Remarks

Authored in degrees for ease of use, stored in radians as that is what unity expects for the quaternion.Euler() method

Struct SawBladeAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of satellites into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct SawBladeAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Saw blades are affected by modifications to the player's additional attack projectiles, damage dealt, attack area, attack duration, and attack projectile speed stat modifications. System will target a random entity on screen and fire an arc volley of saw blades towards it. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

```
state SystemState
```

Class SawBladeEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for saw blade in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class SawBladeEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime
LinearMovementSpeed
DestroyAfterNumberHits

Class SawBladeWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add SawBladeWeaponData to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
[RequireComponent(typeof(EntityRandomAuthoring))]
public class SawBladeWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity. Requires the EntityRandomAuthoring script for random number generation in SawBladeAttackSystem.

Fields

DegreeAngleBetweenAttacks

An angle offset will be applied to the spawn direction of a saw blade so that they all spawn in an arc pattern.

```
public float DegreeAngleBetweenAttacks
```

Field Value

float

Remarks

Authored in degrees for ease of use, stored in radians as that is what unity expects for the quaternion.Euler() method

Struct SawBladeWeaponData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data relevant to spawning saw blade attacks.

```
public struct SawBladeWeaponData : IComponentData, IQueryTypeParameter
```

Fields

RadianAngleBetweenAttacks

An angle offset will be applied to the spawn direction of a saw blade so that they all spawn in an arc pattern.

```
public float RadianAngleBetweenAttacks
```

Field Value

float

Remarks

Authored in degrees for ease of use, stored in radians as that is what unity expects for the quaternion.Euler() method

Class ScaleOscillationAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize values on the ScaleOscillationData of an entity.

```
public class ScaleOscillationAuthoring : MonoBehaviour
```

Remarks

Authoring script uses user-friendly data values for authoring and bakes them into runtime friendly data.

Fields

MaxScale

Maximum uniform scale value the entity will be oscillating between.

```
public float MaxScale
```

Field Value

float

MinScale

Minimum uniform scale value the entity will be oscillating between.

```
public float MinScale
```

Field Value

float

Period

Period (frequency) of the sine wave used to calculate scale.

```
public float Period
```

Field Value

float

Struct ScaleOscillationData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data related to scale oscillation. Scale oscillation is when the scale of an entity's LocalTransform is scaled up and down in an sine wave pattern.

```
public struct ScaleOscillationData : IComponentData, IQueryTypeParameter
```

Fields

Amplitude

Amplitude (height) of the sine wave used to calculate scale.

```
public float Amplitude
```

Field Value

float

Period

Period (frequency) of the sine wave used to calculate scale.

```
public float Period
```

Field Value

float

YOffset

Offset applied along the y-axis to control the midpoint of the sine wave used to calculate scale.

```
public float YOffset
```

Field Value

float

Struct ScaleOscillationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to update the uniform scale value of an entity based on data in ScaleOscillationData.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct ScaleOscillationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System update in the DS_TranslationSystemGroup which updates before Unity's TransformSystemGroup so the LocalTransform component can be safely modified.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

See Also

ScaleOscillationTimer
ScaleOscillationAuthoring

Struct ScaleOscillationTimer

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Timer used to evaluate the sine wave to determine scale at a given time.

```
public struct ScaleOscillationTimer : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public float Value
```

Field Value

float

Class ScreenEdgeBounceAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add ScreenEdgeBounceTag to entity.

```
public class ScreenEdgeBounceAuthoring : MonoBehaviour
```

Struct ScreenEdgeBounceSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to evaluate the position of entities tagged with a ScreenEdgeBounceTag and bounce them in a similar effect to a DVD screensaver.

```
[UpdateInGroup(typeof(DS_TranslationSystemGroup))]
public struct ScreenEdgeBounceSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System update in the DS_TranslationSystemGroup which updates before Unity's TransformSystemGroup so the LocalTransform component can be safely modified.

Methods

OnCreate(ref SystemState)

As a slight optimization, this system will only update when an entity with the ScreenEdgeBounceTag actually exists in the game world.

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

```
state SystemState
```

Struct ScreenEdgeBounceTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity should bounce once it reaches the edge of the screen.

```
public struct ScreenEdgeBounceTag : IComponentData, IQueryTypeParameter
```

Struct ScrewdriverAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct ScrewdriverAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Screwdrivers are affected by modifications to the player's additional attack projectiles, damage dealt, attack speed, and attack duration stat modifications. Screwdrivers shoot in the most recent, non-zero directional input from the player. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivation System.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class ScrewdriverEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for screwdriver in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
public class ScrewdriverEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring and DestructibleEntityAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
DestroyAfterTime
LinearMovementSpeed
DestroyAfterNumberHits

Class ScrewdriverWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add ScrewdriverWeaponTag to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
public class ScrewdriverWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity.

Struct ScrewdriverWeaponTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity as the weapon entity to spawn screwdrivers into the game world.

```
public struct ScrewdriverWeaponTag : IComponentData, IQueryTypeParameter
```

Struct SelectedCharacterReference

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store a reference to the CharacterProperties of the player to be spawned.

```
public struct SelectedCharacterReference : IComponentData, IQueryTypeParameter
```

Remarks

This reference can be assigned in two ways - 1. through selecting a character in the title screen or 2. by assigning a value to the StartingPlayerProperties. Through normal gameplay, this will always be set by the character selected in the title screen. However during debugging in the editor, if the developer enters play mode in a level scene, the value stored in the GameAuthoring will be used as the player.

Fields

Value

```
public UnityObjectRef<CharacterProperties> Value
```

Field Value

UnityObjectRef<CharacterProperties>

Class SelectionIconUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the selection icons that display on either side of the current highlighted element.

```
public class SelectionIconUIController : MonoBehaviour
```

Remarks

Primarily implemented for the ability to navigate UI menus using a controller or the keyboard with no mouse.

Fields

Instance

```
public static SelectionIconUIController Instance
```

Field Value

SelectionIconUIController

Methods

SetPositionOffscreen()

```
public void SetPositionOffscreen()
```

Struct SetEnemyLinearMovementFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enableable component used to inform the EnemyLinearMovementSystem to update the CharacterMoveDirection to a new value. Used to improve efficiency so CharacterMoveDirection is only set when needed.

```
public struct SetEnemyLinearMovementFlag : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

See Also

EnemyLinearMovement
EnemyLinearMovementSystem
EnemyLinearMovementAuthoring

Struct ShouldLoopAudioClipTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to signify this entity should loop their associated audio clip for its full lifecycle.

```
public struct ShouldLoopAudioClipTag : IComponentData, IQueryTypeParameter
```

See Also

PlayAudioClipOnSpawnData

Struct ShowDamageNumberOnDamage

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to hold data related to displaying damage numbers above characters when taking damage.

```
public struct ShowDamageNumberOnDamage : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

Enableable component - when enabled it signifies that this entity should display a damage number above it.

Fields

BaseOffset

The initial offset of the damage number so it properly displays above the character.

```
public float3 BaseOffset
```

Field Value

float3

DamageAmount

Damage amount to be shown in world UI.

```
public int DamageAmount
```

Field Value

int

Class ShowDamageNumberOnDamageAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize components necessary for displaying damage numbers above the entity.

```
public class ShowDamageNumberOnDamageAuthoring : MonoBehaviour
```

Remarks

Authoring script should be attached to the base character entity.

Fields

BaseOffset

The initial offset of the damage number so it properly displays above the character.

```
public Vector3 BaseOffset
```

Field Value

Vector3

See Also

ShowDamageNumberOnDamage
ShowDamageNumberOnDamageSystem

Struct ShowDamageNumberOnDamageSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to display damage numbers above entities when taking damage. Only will run on entities with their ShowDamageNumberOnDamage component enabled.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct ShowDamageNumberOnDamageSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system updates in the DS_EffectsSystemGroup which updates towards the end of the frame. As such base transform position comes from the entity's LocalToWorld component which will have been updated by this point in the frame.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct ShowDamageNumberOnHeal

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to hold data related to displaying green "damage" number above characters when healing.

```
public struct ShowDamageNumberOnHeal : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

Enableable component will be enabled on the frame that the "damage" number should be displayed and disabled subsequently after.

Fields

BaseOffset

The initial offset of the damage number so it properly displays above the character.

```
public float3 BaseOffset
```

Field Value

float3

HealAmount

Integer number of the number of hit points currently being healed to be shown in the world space UI.

```
public int HealAmount
```

Field Value

int

Class ShowDamageNumberOnHealAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize components necessary for displaying "damage" numbers above the entity when healing.

```
public class ShowDamageNumberOnHealAuthoring : MonoBehaviour
```

Remarks

Authoring script should be attached to the base character entity.

Fields

BaseOffset

The initial offset of the damage number so it properly displays above the character.

```
public Vector3 BaseOffset
```

Field Value

Vector3

See Also

ShowDamageNumberOnHeal
ShowDamageNumberOnHealSystem

Struct ShowDamageNumberOnHealSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to display "damage" numbers above enemies when healing. Only will run on entities with the ShowDamageNumberOnHeal component enabled.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct ShowDamageNumberOnHealSystem : ISystem, ISystemCompilerGenerated
```

Remarks

This system updates in the DS_EffectsSystemGroup which updates towards the end of the frame. As such base transform position comes from the entity's LocalToWorld component which will have been updated by this point in the frame.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Enum SineWaveMoveDirection

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enum type to define the direction of travel the enemy will move in as it also moves in a sine wave pattern.

```
[Serializable]
public enum SineWaveMoveDirection : byte
```

Fields

`Down = 4`

`Left = 1`

`None = 0`

`Right = 2`

`Up = 3`

Remarks

For example, left means the character is moving from right to left at a constant move speed while moving up and down in a sine wave pattern.

Class SpaceAgencyFlagController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Singleton MonoBehaviour to control setting of the space agency flag in each level with the flag of the nation associated with the current character.

```
public class SpaceAgencyFlagController : MonoBehaviour
```

Remarks

This controller will exist on the 'Flag' GameObject that is a child of the 'Flagpole' GameObject in the root of the scene.

Fields

Instance

Public singleton instance.

```
public static SpaceAgencyFlagController Instance
```

Field Value

SpaceAgencyFlagController

Methods

SetFlag(Sprite)

Sets the flag to the desired country flag.

```
public void SetFlag(Sprite flagSprite)
```

Parameters

`flagSprite` Sprite

Country flag associated with the current character.

Remarks

Called from OnUpdate(ref SystemState) that executes once at the start of the game.

Class SpaceAgencyProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject to define properties related to an individual space agency.

```
[CreateAssetMenu(fileName = "-SpaceAgencyProperties", menuName = "ScriptableObjects/Space
Agency Properties")]
public class SpaceAgencyProperties : ScriptableObject
```

Fields

DisplayName

Name of the space agency to be displayed in the results UI.

```
public string DisplayName
```

Field Value

string

FlagIcon

Flag icon of the space agency to be displayed in multiple places in the UI and gameplay.

```
public Sprite FlagIcon
```

Field Value

Sprite

Struct SpawnEvent

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer containing information pertaining to the spawn events for the current wave.

```
[InternalBufferCapacity(4)]
public struct SpawnEvent : IBufferElementData
```

Remarks

At the start of each wave, this dynamic buffer is cleared out and populated with runtime data for each EnemySpawnEventProperties element in the SpawnEvents array. This dynamic buffer contains all data pertaining to any spawn event that could occur, so not all fields are relevant to all spawn events.

Constructors

SpawnEvent(EnemySpawnEventProperties, EnemySpawnWaveProperties, int, Entity)

Constructor to assist with creation of SpawnEvent elements.

```
public SpawnEvent(EnemySpawnEventProperties properties, EnemySpawnWaveProperties
wavePropertiesProperties, int spawnEventIndex, Entity enemyPrefab)
```

Parameters

`properties` EnemySpawnEventProperties

Authoring data for the spawn event.

`wavePropertiesProperties` EnemySpawnWaveProperties

Authoring data for the spawn wave.

`spawnEventIndex` int

Index of this spawn event to be added. Used to calculate delay time.

`enemyPrefab` Entity

Entity prefab of the enemy to be spawned for this event.

Fields

Amplitude

Optional field relevant to certain spawn formations. Defines the Amplitude for enemy sine wave movement. EnemySineWaveMovement

```
public float Amplitude
```

Field Value

float

ChanceOfOccurrence

Percentage chance from 0 to 100 that the event will occur. A value of 0 means the spawn event will never occur and a value of 100 means the event will always occur.

```
public int ChanceOfOccurrence
```

Field Value

int

DelayTime

Delay time is the time in seconds from the start of the wave until the spawn event occurs. It is up to the event designer to ensure that all events occur within the duration of the spawn event as events that take place after the conclusion of the spawn event will not occur.

```
public float DelayTime
```

Field Value

float

EnemyCount

Number of enemies that will be spawned during this spawn event.

```
public int EnemyCount
```

Field Value

int

EnemyPrefab

Entity prefab for the enemy type that will be spawned in this spawn event.

```
public Entity EnemyPrefab
```

Field Value

Entity

EnemySpacing

Optional field relevant to certain spawn formations. Defines the spacing between enemies.

```
public float EnemySpacing
```

Field Value

float

HasOccurred

Flag to determine if this spawn event has already occurred.

```
public bool HasOccurred
```

Field Value

bool

MoveSpeed

Optional field relevant to certain spawn formations. Defines the movement speed for linear moving enemies (EnemyLinearMovement) and constant move speed for sine wave enemies (EnemySineWave Movement).

```
public float MoveSpeed
```

Field Value

float

Period

Optional field relevant to certain spawn formations. Defines the period for enemy sine wave movement. EnemySineWaveMovement

```
public float Period
```

Field Value

float

SpawnFormation

Arrangement the enemies will be spawned in.

```
public SpawnFormation SpawnFormation
```

Field Value

SpawnFormation

TimeToLive

Optional field relevant to certain spawn formations. Defines the time to live for a given enemy before it is destroyed.

```
public float TimeToLive
```

Field Value

float

Enum SpawnFormation

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Defines the formation enemies for a SpawnEvent will spawn in. The formation will also determine certain aspects of their behavior in the game.

```
public enum SpawnFormation : byte
```

Fields

`EllipseAroundView = 2`

`LinearMoveGroup = 1`

`None = 0`

`SineMoveHorizontalLine = 4`

`SineMoveVerticalLine = 3`

Struct SpawnOnDestroy

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Component to store the entity prefab that will be instantiated when the entity with this component is destroyed.

```
public struct SpawnOnDestroy : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public Entity Value
```

Field Value

Entity

See Also

DestroyEntitySystem
SpawnOnDestroyAuthoring

Class SpawnOnDestroyAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize the entity prefab stored in the SpawnOnDestroy component.

```
public class SpawnOnDestroyAuthoring : MonoBehaviour
```

Fields

EntityPrefabToSpawn

GameObject prefab to be converted into an entity in this class's baker which gets assigned to the SpawnOnDestroy component added to this entity.

```
public GameObject EntityPrefabToSpawn
```

Field Value

GameObject

See Also

DestroyEntitySystem
SpawnOnDestroy

Class SpawnRandomOnDestroyAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring component to associate a RandomItemDropProperties ScriptableObject with an entity. This authoring component will also convert GameObject prefabs set in the RandomItemDropProperties and store them in a temporary SpawnRandomPrefabHelper DynamicBuffer.

```
[RequireComponent(typeof(DamageableEntityAuthoring))]
public class SpawnRandomOnDestroyAuthoring : MonoBehaviour
```

Remarks

Due to the nature of entity remapping from prefabs created during baking to prefabs available at runtime, a RandomItemToSpawnBlob blob array must be populated with the required prefabs at runtime. To facilitate with this, prefabs that can be spawned are first baked into entities in the SpawnRandomOnDestroyAuthoring.Baker into a DynamicBuffer of SpawnRandomPrefabHelper. The InitializeSpawnRandomOnDestroySystem will populate this blob array with the appropriate data. It is important that we include DependsOn(authoring.RandomItemDropProperties) so baking is re-ran when values on the ScriptableObject are changed.

Fields

RandomItemDropProperties

ScriptableObject defining the items and rarity this entity can drop upon destruction.

```
public RandomItemDropProperties RandomItemDropProperties
```

Field Value

RandomItemDropProperties

Remarks

It is important that we include DependsOn(authoring.RandomItemDropProperties) so baking is re-ran when values on the ScriptableObject are changed.

See Also

RandomItemToSpawn
RandomItemDropProperties
DestroyEntitySystem
InitializeSpawnRandomOnDestroySystem
SpawnRandomPrefabHelper

Struct SpawnRandomPrefabHelper

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic Buffer component which is required because there is no way to "covert" an GameObject into an Entity at runtime. As such the entity conversion will take place in the SpawnRandomOnDestroyAuthoring.Baker and those prefabs will be stored in this Dynamic Buffer. At runtime, a blob asset will be generated storing these entity prefabs.

```
public struct SpawnRandomPrefabHelper : IBufferElementData
```

Remarks

Due to Unity's entity remapping from the "bake-time" entity to the runtime entity, the entity index will not be the same. Thus the prefabs need to be created and available at runtime before a reference to them can be stored in a blob asset.

Fields

Prefab

```
public Entity Prefab
```

Field Value

Entity

Rarity

```
public int Rarity
```

Field Value

int

Struct SpawnRandomPropertiesHashCode

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store the hash code of the RandomItemDropProperties ScriptableObject associated with this entity.

```
public struct SpawnRandomPropertiesHashCode : IComponentData, IQueryTypeParameter
```

Remarks

This is used in the InitializeSpawnRandomOnDestroySystem to determine if a RandomItemToSpawnBlob blob asset has already been created for this ScriptableObject. If it has, that blob asset will be assigned, if it hasn't been created, it will create one and associate with this hash value.

Fields

Value

```
public uint Value
```

Field Value

uint

See Also

RandomItemDropProperties
RandomItemToSpawnBlob
DestroyEntitySystem

Struct SpawnWaveEnemyPrefab

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic Buffer to hold all the enemy prefabs that will be spawned in the current wave.

```
public struct SpawnWaveEnemyPrefab : IBufferElementData
```

Remarks

Note that this buffer does not store the prefabs for boss enemies or enemies belonging to a spawn event. Boss enemies are instantiated at the wave start and thus there is no need to hold onto a prefab reference. Spawn events currently spawn enemies of a single type and their prefab is held in the Spawn Event Dynamic Buffer.

Fields

Value

```
public Entity Value
```

Field Value

Entity

Enum StageID

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ID of the stage, used for serialization.

```
public enum StageID : byte
```

Fields

`ArtTestScene = 16`

`Mars = 4`

`Moon = 2`

`None = 0`

`Venus = 8`

Remarks

Each stage is assigned a bit value so they can be used as bitflags in UnlockedStageFlags.

Class StageProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject to store properties related to stages.

```
[CreateAssetMenu(fileName = "-StageProperties", menuName = "ScriptableObjects/Stage
Properties")]
public class StageProperties : ScriptableObject
```

Fields

EnemySpawnWaves

Collection of EnemySpawnWaveProperties to define which waves will be spawned in a given level.

```
public EnemySpawnWaveProperties[] EnemySpawnWaves
```

Field Value

EnemySpawnWaveProperties[]

FinalBossType

Final boss is the boss that will be spawned after all waves complete.

```
public EnemyType FinalBossType
```

Field Value

EnemyType

Remarks

The final boss spawns with a very high health and damage value, making it essentially impossible to destroy and will immediately destroy the player on first contact. Before the final boss is spawned, all other enemies in the world will be destroyed.

PreviewSprite

Preview sprite shown in the stage selection UI.

```
public Sprite PreviewSprite
```

Field Value

Sprite

SceneIndex

Build index of the scene associated with this level.

```
public int SceneIndex
```

Field Value

int

StageDescription

Description of the stage that will be displayed in the stage selection UI.

```
[TextArea]
public string StageDescription
```

Field Value

string

StageID

ID of the stage, used for serialization.

```
public StageID StageID
```

Field Value

StageID

StageName

Name of the stage that will be displayed in the stage selection UI.

```
public string StageName
```

Field Value

string

UnlockCost

Cost to unlock the stage for playing.

```
public int UnlockCost
```

Field Value

int

WaveInterval

Interval at which the next wave of entities will be selected.

```
public int WaveInterval
```

Field Value

int

Properties

TimeLimit

Time it will take to reach the final spawn wave in the level, shown in the stage selection UI.

```
public float TimeLimit { get; }
```

Property Value

float

Remarks

EnemySpawnWaves.Length - 1 is used as the final stage in the game is the reaper stage that will effectively destroy the entity immediately, so this stage shouldn't be factored into the time calculation.

Struct StagePropertiesReference

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store Unity Object reference to StageProperties ScriptableObject associated with the current stage.

```
public struct StagePropertiesReference : IComponentData, IQueryTypeParameter
```

Fields

Value

```
public UnityObjectRef<StageProperties> Value
```

Field Value

UnityObjectRef<StageProperties>

Class StageSelectButtonUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the individual UI elements to display a stage available to be chosen.

```
public class StageSelectButtonUIController : MonoBehaviour
```

Properties

SelectionButton

```
public Button SelectionButton { get; }
```

Property Value

Button

Methods

SetUI(StageProperties, bool, UnityAction)

Sets UI elements associated with the stage available to be selected.

```
public void SetUI(StageProperties stageProperties, bool isUnlocked,
UnityAction onSelectStage)
```

Parameters

`stageProperties` StageProperties

ScriptableObject containing information and assets pertaining to this stage.

`isUnlocked` bool

Flag to denote if this stage is unlocked. If the stage is locked, a lock icon will be shown with the price to unlock.

`onSelectStage` UnityAction

Method that will be called when the main button associated with this UI element is clicked.

Class StageSelectPanelUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the stage select panel displayed in the main menu of the title scene.

```
public class StageSelectPanelUIController : MonoBehaviour
```

Properties

SelectedStageSceneIndex

```
public int SelectedStageSceneIndex { get; }
```

Property Value

int

Methods

HideStageSelectPanel()

```
public void HideStageSelectPanel()
```

ShowStageSelectPanel(int)

```
public void ShowStageSelectPanel(int stageIndexToSelect = 0)
```

Parameters

`stageIndexToSelect` int

Struct StatModifier

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Dynamic buffer to hold stat modifier types and values.

```
[Serializable]
[InternalBufferCapacity(12)]
public struct StatModifier : IBufferElementData
```

Remarks

This dynamic buffer is attached to the active stat entity for a character, references are stored in ActiveStatModifierEntity. Internal buffer capacity is set to worst case of 12, matching the number of unique stat modifier types. Memory space is not critical for these entities so we can store up to 12 elements inside the chunk.

Fields

Type

```
public StatModifierType Type
```

Field Value

StatModifierType

Value

```
public float Value
```

Field Value

float

See Also

RecalculateStatsSystem

Enum StatModifierCalculationType

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Defines the calculation type of the stat modifier.

```
public enum StatModifierCalculationType : byte
```

Extension Methods
StatModifierHelper.GetDefaultModificationValue(StatModifierCalculationType)

Fields

`AbsoluteDecimal = 2`

Stat modification type is an absolute decimal. This means this value will be added to the base value for the stat modifications and the full value is displayed as a decimal in the UI, not just the differential value.

`AbsoluteInteger = 1`

Stat modification type is an absolute integer. This means this value will be added to the base value for stat modifications and the full value is displayed as an integer in the UI, not just the differential value.

`IncreasingInteger = 3`

Stat modification type is an increasing integer. This means this value will be added to the base value for the stat modification and just the differential value from the default value (0) will be displayed in the UI.

`None = 0`

None - invalid stat modifier calculation type.

`Percentage = 4`

Stat modification type is a percentage. This means this value will be multiplied by the base stat modification value for the type and just the differential value from the default value (1) will be displayed in the UI.

Class StatModifierController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Stat modifier controller singleton. Objects with this component will be marked as Don't Destroy on Load as all scenes will need access to StatModifierProperties ScriptableObjects loaded from this controller.

```
public class StatModifierController : MonoBehaviour
```

Fields

Instance

Public static singleton access.

```
public static StatModifierController Instance
```

Field Value

StatModifierController

Properties

StatModifierProperties

Array of StatModifierProperties loaded from the Resources/ScriptableObjects/StatModifierProperties folder.

```
public static StatModifierProperties[] StatModifierProperties { get; }
```

Property Value

StatModifierProperties[]

Methods

GetMaximumModificationValue(StatModifierType)

Gets the maximum modification value as defined by the associated StatModifierProperties.

```
public float GetMaximumModificationValue(StatModifierType statModifierType)
```

Parameters

`statModifierType` StatModifierType

Stat modification type to get maximum modification value of

Returns

float

Maximum modification value for desired stat modification type

GetMinimumModificationValue(StatModifierType)

Gets the minimum modification value as defined by the associated StatModifierProperties.

```
public float GetMinimumModificationValue(StatModifierType statModifierType)
```

Parameters

`statModifierType` StatModifierType

Stat modification type to get minimum modification value of

Returns

float

Minimum modification value for desired stat modification type

Struct StatModifierEntityTag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Tag component to identify a stat modifier entity.

```
public struct StatModifierEntityTag : IComponentData, IQueryTypeParameter
```

Class StatModifierHelper

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Helper class to add extension methods to StatModifierType and StatModifierCalculationType enums.

```
public static class StatModifierHelper
```

Methods

ClampModificationValue(StatModifierType, ref float)

Extension method to clamp modification value within minimum and maximum values defined in the StatModifierProperties ScriptableObject.

```
public static void ClampModificationValue(this StatModifierType statModifierType, ref
float value)
```

Parameters

`statModifierType` StatModifierType

Stat modifier type, used to lookup the minimum and maximum values for the type.

`value` float

Value to clamp. Variable is a reference type so variable passed in when calling this method will be updated to be inside the minimum and maximum values.

GetDefaultModificationValue(StatModifierCalculationType)

Extension method to get the default modification value of a stat modification type. Uses the StatModifierCalculationType as each of these types will have the same default value.

```
public static float GetDefaultModificationValue(this StatModifierCalculationType
calculationType)
```

Parameters

`calculationType` StatModifierCalculationType

Value type of the stat modifier.

Returns

float

Default modification value.

Remarks

Default value for percentage types is 1f as when it is multiplied by the base value, the base vale will remain unchanged. For other types where stat modifications are added to the base value, the value is initialized to a default value of 0f.

Struct StatModifierInfo

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Helper struct to author stat modifications in the editor.

```
[Serializable]
public struct StatModifierInfo
```

Fields

Type

Type of stat modification.

```
public StatModifierType Type
```

Field Value

StatModifierType

Value

Value of the stat modification.

```
public float Value
```

Field Value

float

Class StatModifierProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject to author properties related to stat modifiers. Certain properties are used for runtime data and others are used for displaying information in UI elements.

```
[CreateAssetMenu(fileName = "-Properties", menuName = "ScriptableObjects/Stat
Modifier Properties")]
public class StatModifierProperties : ScriptableObject
```

Remarks

These properties should be stored in the Resources/ScriptableObjects/StatModifierProperties directory. Only one StatModifierProperties ScriptableObject should exist in the above directory else the StatModifierController will throw an error when trying to load these ScriptableObjects.

Fields

CalculationType

Calculation type for the stat modifier.

```
public StatModifierCalculationType CalculationType
```

Field Value

StatModifierCalculationType

Icon

Sprite icon of the stat modifier to be displayed in the UI.

```
public Sprite Icon
```

Field Value

Sprite

MaximumModificationValue

Minimum value for the stat modification. Used when clamping the value to ensure it stays within a range to facilitate game balance.

```
public float MaximumModificationValue
```

Field Value

float

MinimumModificationValue

Minimum value for the stat modification. Used when clamping the value to ensure it stays within a range to facilitate game balance.

```
public float MinimumModificationValue
```

Field Value

float

ModifierType

Type of stat modifier.

```
public StatModifierType ModifierType
```

Field Value

StatModifierType

Title

Title of the stat modifier to be displayed in the UI.

```
public string Title
```

Field Value

string

Enum StatModifierType

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enum containing all stat modifier types. Used as a way to author stat modifications in the editor and used as a key in several places in code to access information for a given stat modifier type via the associated StatModifierProperties

```
[Serializable]
public enum StatModifierType : byte
```

Extension Methods

StatModifierHelper.ClampModificationValue(StatModifierType, ref float)

Fields

`AdditionalAttackProjectiles = 10`

Number of additional attack projectiles to spawn for each applicable attack.

`AdditionalHitPoints = 3`

Additional hit points to increase the character's maximum hit points.

`AttackArea = 6`

Item attraction radius modifier. This value is multiplied by the player's Radius in the DetectItem AttractionSystem to calculate the effective attraction area for the player.

`AttackCooldown = 5`

Attack cooldown modification. This value is multiplied by the base cooldown for an attack in various attack systems to calculate the current effective cooldown for that attack.

`AttackDuration = 9`

Attack duration modification. This value is multiplied by the base attack duration for an attack in various attack systems to calculate the current effective duration for how long the attack entity exists in the world.

`AttackProjectileSpeed = 7`

Attack duration modification. This value is multiplied by the base attack duration for an attack in various attack systems to calculate the current effective duration for how long the attack entity exists in the world.

`DamageDealt = 1`

Damage dealing modification. This value is multiplied by the base damage value for an attack in various attack systems to calculate the current effective damage to be assigned to that attack entity.

`DamageReceived = 2`

Reduce incoming damage points by this integer value.

`ExperiencePointGain = 12`

Experience point gain modifier. This value is multiplied by the value of experience points collected by the player in the HandlePlayerExperienceThisFrameSystem to calculate the effective experience point gain for the collected gem.

`HealthRegeneration = 4`

Health regeneration modification. This value represents how many hit points per second are restored in the CharacterHealthRegenerationSystem.

`ItemAttractionRadius = 11`

Item attraction radius modifier. This value is multiplied by the player's Radius in the DetectItem AttractionSystem to calculate the effective attraction area for the player.

`MoveSpeed = 8`

Move speed modification. This value is multiplied by the character's CharacterBaseMoveSpeed in the CharacterMoveSystem to calculate the current effective move speed.

`None = 0`

None - unused, invalid value.

Struct TemporaryStatModifierActiveTime

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component defining the time in seconds this temporary stat modifier entity should remain active.

```
public struct TemporaryStatModifierActiveTime : IComponentData, IQueryTypeParameter
```

Remarks

In the HandleTemporaryStatModifierInteractionSystem a new stat modifier entity is created with a DestroyAfterTime component.

Fields

Value

```
public float Value
```

Field Value

float

Class TemporaryStatModifierAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to initialize data components on temporary stat modifier entities.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
public class TemporaryStatModifierAuthoring : MonoBehaviour
```

Fields

StatModifiers

Array of stat modifications to be applied when this stat modifier is active.

```
public StatModifierInfo[] StatModifiers
```

Field Value

StatModifierInfo[]

TimeActive

Time in seconds this stat modifier should be activel

```
public float TimeActive
```

Field Value

float

See Also

TemporaryStatModifierActiveTime
HandleTemporaryStatModifierInteractionSystem

StatModifier

Struct TileScaleOverride

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Material property override to set the tile scale factor.

```
[MaterialProperty("_TileScale", -1)]
public struct TileScaleOverride : IComponentData, IQueryTypeParameter
```

Remarks

Your IDE may gray out the Value field as this value is not used in our code. However, Unity uses it to apply this value to the material property defined in the MaterialProperty attribute. Be sure the MaterialProperty string exactly matches the reference string defined in the shader as this will silently fail if there is a typo.

Fields

Value

```
public float Value
```

Field Value

float

Class TitleScreenUIController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the main menu in the title screen.

```
public class TitleScreenUIController : MonoBehaviour
```

Remarks

Handles showing and hiding UI panels when appropriate.

Struct UpdateCompanionGameObjectSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle the lifecycle of companion GameObjects. Companion GameObjects are GameObjects that will match the position and rotation of their associated entity. This system has 3 foreach loops that do the following:

1. Instantiate the companion GameObject from Prefab assign values to a CompanionGameObjectData and add it to the entity.
2. Update the position and rotation of the companion GameObject from values in the entity's LocalToWorld component.
3. When the entity is destroyed, destroy the companion GameObject. Depending on settings, this may happen after a delay set in DelayCleanupTime.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public struct UpdateCompanionGameObjectSystem : ISystem, ISystemCompilerGenerated
```

Remarks

System updates in the DS_EffectsSystemGroup which updates after the Unity's TransformSystemGroup. To ensure accuracy of position and rotation, these values are sourced from the entity's LocalToWorld component.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class UpdateGameTimeSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Updates the in-game time during gameplay.

```
[UpdateInGroup(typeof(InitializationSystemGroup))]
[UpdateAfter(typeof(GlobalTimeUpdateSystem))]
public class UpdateGameTimeSystem : SystemBase
```

Remarks

System updates in Unity's InitializationSystemGroup and after the GlobalTimeUpdateSystem to ensure that SystemAPI.Time.ElapsedTime is accurate for the current frame.

Fields

OnUpdateGameTime

Event to update in-game UI with the current time.

```
public Action<float> OnUpdateGameTime
```

Field Value

Action <float>

Methods

OnCreate()

```
protected override void OnCreate()
```

OnUpdate()

```
protected override void OnUpdate()
```

Struct UpdatePlayerHealthUIFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag component to inform the PlayerWorldUISystem to update the health slider.

```
public struct UpdatePlayerHealthUIFlag : IComponentData, IQueryTypeParameter,
IEnableableComponent
```

Remarks

This is set to enabled when the player current or max health is changed due to being damaged or picking up an item.

Class UpdateUISystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to update UI elements on screen.

```
[UpdateInGroup(typeof(DS_EffectsSystemGroup))]
public class UpdateUISystem : SystemBase
```

Remarks

System invokes the event every frame, in this case that is fine as I know there is just a single listener to the event and the alien defeated count it likely to change very frequently so there is no point in adding complexity to manage when to fire these events.

Fields

OnUpdateEnemiesDefeatedCount

Event invoked to update HUD UI controller with the number of enemies the player has defeated in the current run.

```
public Action<int> OnUpdateEnemiesDefeatedCount
```

Field Value

Action<int>

Methods

OnUpdate()

```
protected override void OnUpdate()
```

Struct UpgradePassiveFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Enableable component to signify this passive ability should be upgraded to the next level in the UpgradePassiveSystem.

```
public struct UpgradePassiveFlag : IComponentData, IQueryTypeParameter, IEnableableComponent
```

Struct UpgradePassiveSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to upgrade passive to the next level.

```
public struct UpgradePassiveSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Stat modifier upgrades do not stack. The StatModifier dynamic buffer is cleared on each upgrade so when authoring the stat modifications in PassiveUpgradeProperties only the stat modifications for the current level will be applied.

Methods

OnUpdate(ref SystemState)

```
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class UpgradeProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Abstract class of a ScriptableObject to define the upgradable properties for weapons and passive items that can be upgraded during gameplay.

```
public abstract class UpgradeProperties : ScriptableObject
```

Derived
PassiveUpgradeProperties, WeaponUpgradeProperties

Fields

Icon

Icon of the upgradeable item to be displayed in the UI.

```
public Sprite Icon
```

Field Value

Sprite

Name

Name of the upgradeable item to be displayed in the UI.

```
public string Name
```

Field Value

string

Properties

MaxLevelIndex

Virtual property to return the maximum level index for the upgradable item.

```
public virtual int MaxLevelIndex { get; }
```

Property Value

int

Rarity

Rarity property for builds to return the TMG.DOTSSurvivors.UpgradeProperties._rarity field.

```
public int Rarity { get; }
```

Property Value

int

Methods

GetDescription(int)

Virtual method to get the description of an upgrade at a given level index.

```
public virtual string GetDescription(int levelIndex)
```

Parameters

`levelIndex` int

Level index to get the description of. Index 0 for level 1, index 1 for level 2, and so on.

Returns

string

Description of changes to the item granted by this upgrade shown to the player in the level up UI.

Remarks

This method will be overloaded in child classes to return the description at a specific index in the upgrade properties array.

Struct UpgradeWeaponFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag component to signify this weapon should be upgraded to the next WeaponLevelData level.

```
public struct UpgradeWeaponFlag : IComponentData, IQueryTypeParameter, IEnableableComponent
```

Remarks

Enableable component will be marked as active when the weapon should be upgraded.

Struct UpgradeWeaponSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to upgrade runtime weapon data when the UpgradeWeaponFlag is set to true.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct UpgradeWeaponSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Note that the WeaponActiveFlag is not an EnabledRefRW in the foreach query. This is because this isn't guaranteed to be active during upgrading. However, it is set to false to effectively reset its state if it is in the middle of being active. Note that we also set CooldownTimer to a very low number so the weapon will become active almost immediately after upgrading.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct WaveIndexOfSpawn

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Shared component to store the index of the wave the enemy was spawned in.

```
public struct WaveIndexOfSpawn : ISharedComponentData, IQueryTypeParameter
```

Remarks

Shared component is used so enemies in a specific wave can be queried for. This is notably used in the DestroyPreviousWaveEnemySystem system that cleans up offscreen enemies from previous waves.

Fields

Value

```
public int Value
```

Field Value

int

Struct WeaponActivationSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to determine when a weapon should become active.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup), OrderFirst = true)]
public struct WeaponActivationSystem : ISystem, ISystemCompilerGenerated
```

Remarks

A weapon is active when its WeaponActiveFlag is enabled; at this time it will begin spawning instances of AttackPrefabs into the game world. Spawning of attack instances take place in systems unique to the weapon type.

Methods

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Struct WeaponActiveFlag

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Flag component to signify this weapon is currently active.

```
public struct WeaponActiveFlag : IComponentData, IQueryTypeParameter, IEnableableComponent
```

Remarks

Enableable component will be marked as active when CooldownTimer is expired and the weapon should perform its attack behavior (typically spawning AttackPrefab entities).

Class WeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Base authoring script to add components all weapon entities need.

```
public class WeaponAuthoring : MonoBehaviour
```

Remarks

Note that this component will be attached to the base weapon entity which exists from instantiation through the remainder of a game run. This should not be attached to AttackPrefabs that spawn as visible objects in the game world. The baker of this authoring script will allocate a blob array of WeaponLevelData and populate it with data from the WeaponUpgradeProperties field of this authoring script. It is important that we include DependsOn(authoring.WeaponUpgradeProperties) so baking is re-ran when values on the ScriptableObject are changed.

Fields

StartingLevel

Debug field to set the level the weapon will start on when added to the player's weapon inventory. Used for testing higher levels of a weapon without needing to upgrade in normal gameplay.

```
public int StartingLevel
```

Field Value

int

Remarks

This field is for debugging purposes only and should be set to 0 before shipping.

WeaponUpgradeProperties

Reference to the ScriptableObject defining properties of a weapon.

```
public WeaponUpgradeProperties WeaponUpgradeProperties
```

Field Value

WeaponUpgradeProperties

Remarks

Unmanaged data from this ScriptableObject get loaded into a blob asset during baking. It is important that we include DependsOn(authoring.WeaponUpgradeProperties) so baking is re-ran when values on the ScriptableObject are changed.

Struct WeaponLevelData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data related to player weapons.

```
[Serializable]
public struct WeaponLevelData : IComponentData, IQueryTypeParameter
```

Remarks

Data in this component represents a single level for a weapon. A collection of WeaponLevelData components are stored in the WeaponUpgradeData to represent the all WeaponLevelData components in a weapon's upgrade path. Not all fields are used on all weapon types. Has the System.Serializable attribute so these values can be initialized in the editor via WeaponUpgradeProperties.

Fields

Area

Scale modifier of an attack entity.

```
public float Area
```

Field Value

float

AttackCount

Number of attack entities that will be spawned in a single attack group.

```
public int AttackCount
```

Field Value

int

BaseHitPoints

Base hit points that will be assigned to an attack entity to deal damage to an enemy entity via Entity Interaction

```
public int BaseHitPoints
```

Field Value

int

CollisionFilter

Collision filter to be applied to attack entities.

```
public CollisionFilter CollisionFilter
```

Field Value

CollisionFilter

Cooldown

Cooldown for the weapon in seconds. This is the amount of time between when the final attack entity of an attack group is spawned and the first attack entity will be spawned for the next attack group.

```
public float Cooldown
```

Field Value

float

IntervalBetweenAttacks

Interval between attack entity spawns in an attack group in seconds.

```
public float IntervalBetweenAttacks
```

Field Value

float

MaxEnemyHitCount

Maximum number of enemy entities an attack entity can collide with before self-destructing.

```
public int MaxEnemyHitCount
```

Field Value

int

MovementSpeed

Movement speed in units per second of a moving attack entity.

```
public float MovementSpeed
```

Field Value

float

RandomChance

Random chance used to determine if certain random behaviors should or should not happen.

```
public int RandomChance
```

Field Value

int

Remarks

Value will be from 0 to 100 where 0 will never occur and 100 will always occur.

TimeToLive

Time an attack entity will exist in the game world before self-destructing.

```
public float TimeToLive
```

Field Value

float

Struct WeaponLevelInfo

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data struct used to store weapon data and a description string for a single level of a weapon's upgrade path.

```
[Serializable]
public struct WeaponLevelInfo
```

Remarks

Has the System.Serializable attribute so these values can be initialized in the editor via WeaponUpgrade Properties.

Fields

Description

Description of changes to WeaponLevelData to be shown to the player in the level up UI.

```
public string Description
```

Field Value

string

WeaponLevelData

WeaponLevelData associated with a specific level of a weapon.

```
public WeaponLevelData WeaponLevelData
```

Field Value

[WeaponLevelData]

Struct WeaponState

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to hold the current state of a weapon.

```
public struct WeaponState : IComponentData, IQueryTypeParameter
```

Remarks

Not all fields are relevant to all weapon types.

Fields

AttackCount

Value to keep track of the number of attack entities spawned in the current attack group.

```
public int AttackCount
```

Field Value

int

CooldownTimer

Timer to keep track of the time until the next attack group begins in seconds.

```
public float CooldownTimer
```

Field Value

float

LevelIndex

Value to store the current level index of a weapon. Level 1 (lowest level) is stored at 0 in this field.

```
public int LevelIndex
```

Field Value

int

NextAttackTimer

Timer to keep track of time until the next attack entity should spawn.

```
public float NextAttackTimer
```

Field Value

float

Enum WeaponType

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Defines the type of the weapon. Often used as a key to obtain types relevant to the weapon in managed collections.

```
public enum WeaponType : byte
```

Fields

BlackHole = 1

CO2Cloud = 2

DroneSwarm = 3

Jetpack = 4

LaserStrike = 5

None = 0

OxygenHose = 6

PlasmaBlast = 7

RadioactiveWaste = 8

SatelliteToss = 9

SawBlade = 10

Screwdriver = 11

Wrench = 12

Struct WeaponUpgradeData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to hold blob asset reference for blob array of WeaponLevelData components for a weapon's upgrade path.

```
public struct WeaponUpgradeData : IComponentData, IQueryTypeParameter
```

Remarks

Blob asset created during baking in WeaponAuthoring.

Fields

Value

Reference to the blob array of WeaponLevelData for a weapon's upgrade path. Element 0 of the array corresponds to the weapon level data for level 1 of the attack, index 1 is level 2, and so on.

```
public BlobAssetReference<BlobArray<WeaponLevelData>> Value
```

Field Value

BlobAssetReference<BlobArray<WeaponLevelData>>

Properties

this[int]

Custom indexer for easy access to the WeaponLevelData for a weapon.

```
public WeaponLevelData this[int i] { get; }
```

Parameters

i int

Index into the blob array. Index 0 is level 1 for a weapon, index 1 is level 2, and so on.

Property Value

WeaponLevelData

Class WeaponUpgradeProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

ScriptableObject to define the properties for a weapon and its upgrade path.

```
[CreateAssetMenu(fileName = "WeaponUpgradeProperties", menuName = "ScriptableObjects/Weapon
Upgrade Properties")]
public class WeaponUpgradeProperties : UpgradeProperties
```

Remarks

Inherits from UpgradeProperties.

Fields

AttackPrefab

GameObject prefab that will be spawned when the weapon is active via WeaponActiveFlag. This GameObject will be baked into an Entity in the WeaponAuthoring script and stored in the AttackPrefab on the associated weapon entity.

```
public GameObject AttackPrefab
```

Field Value

GameObject

LevelPropertiesArray

Array to store WeaponLevelData and descriptions for each level in the weapon's upgrade path.

```
public WeaponLevelInfo[] LevelPropertiesArray
```

Field Value

WeaponLevelInfo[]

WeaponType

Defines the type of the weapon. Often used as a key to obtain types relevant to the weapon in managed collections.

```
public WeaponType WeaponType
```

Field Value

WeaponType

Properties

MaxLevelIndex

Property to return the maximum level index for the weapon.

```
public override int MaxLevelIndex { get; }
```

Property Value

int

Methods

GetDescription(int)

Gets the description of the weapon upgrade at a given level.

```
public override string GetDescription(int levelIndex)
```

Parameters

`levelIndex` int

Index of the level to get the description from. Index 0 for level 1, index 1 for level 2, and so on.

Returns

string

Description of the weapon upgrade at the given level to be shown to the player in the level up UI.

Struct WeaponUpgradePropertiesReference

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to reference the WeaponUpgradeProperties for a weapon.

```
public struct WeaponUpgradePropertiesReference : IComponentData, IQueryTypeParameter
```

Remarks

This is primarily used for UI display of weapon upgrades as runtime weapon data is stored in WeaponUpgradeData.

Fields

Value

```
public UnityObjectRef<WeaponUpgradeProperties> Value
```

Field Value

UnityObjectRef<WeaponUpgradeProperties>

Struct WeightedUpgradeProperties

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Helper struct used for a pairing of UpgradeProperties with a weighted random value. Used in the PlayerUpgradeData to retrieve a random upgrade.

```
public struct WeightedUpgradeProperties
```

Fields

UpgradeProperties

Upgrade Properties associated with this weighted random value.

```
public UpgradeProperties UpgradeProperties
```

Field Value

UpgradeProperties

WeightedRandomValue

Highest random value associated with this UpgradeProperties instance.

```
public int WeightedRandomValue
```

Field Value

int

Remarks

Weighted random value is assigned in the GameStartSystem. During initialization, all UpgradeProperties ScriptableObjects are iterated, this weighted random value is calculated by adding the running total of

all weighted random values to the Rarity property. When determining which upgrades should be shown during level up, a random value between 0 and the maximum weighted random value is chosen; each element of WeightedUpgradeProperties is iterated to see if that number is less than its assigned weighted random value. If it is, then the upgrade is selected to be shown.

Class WorldUICanvasController

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

UI controller associated with the world space UI canvas used to display damage numbers above enemies taking damage.

```
public class WorldUICanvasController : MonoBehaviour
```

Fields

Instance

```
public static WorldUICanvasController Instance
```

Field Value

WorldUICanvasController

Methods

DisplayDamageNumber(int, Vector3, bool)

```
public void DisplayDamageNumber(int damageValue, Vector3 startPosition, bool isHealing
= false)
```

Parameters

`damageValue` int

`startPosition` Vector3

`isHealing` bool

ReturnDamageNumberToPool(DamageNumberUIController)

```
public void ReturnDamageNumberToPool(DamageNumberUIController damageNumberUIController)
```

Parameters

`damageNumberUIController` DamageNumberUIController

Struct WrenchAttackSystem

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

System to handle instantiating AttackPrefabs of wrenches into the game world.

```
[UpdateInGroup(typeof(DS_AttackSystemGroup))]
public struct WrenchAttackSystem : ISystem, ISystemCompilerGenerated
```

Remarks

Wrenches are affected by modifications to the player's additional attack projectiles, damage dealt, and attack projectile speed stat modifications. The initial wrench attack will spawn in the direction towards a random enemy on screen. Subsequent wrenches will be offset from that initial angle by the angle defined in RadianAngleBetweenAttacks. Wrenches move in a boomerang pattern, first moving in the direction of spawn then reversing on its initial path. System executes on the persistent weapon entity responsible for spawning attacks, not the in-world attack entity itself. This system will only execute on the weapon entity once its CooldownTimer expires and has its WeaponActiveFlag set to true in the WeaponActivationSystem.

Methods

OnCreate(ref SystemState)

```
public void OnCreate(ref SystemState state)
```

Parameters

`state` SystemState

OnUpdate(ref SystemState)

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
```

Parameters

`state` SystemState

Class WrenchEntityAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add components necessary for wrench in-world attack entity to function.

```
[RequireComponent(typeof(EntityInteractionAuthoring))]
[RequireComponent(typeof(DestructibleEntityAuthoring))]
[RequireComponent(typeof(DestroyOffCameraAuthoring))]
public class WrenchEntityAuthoring : MonoBehaviour
```

Remarks

Requires EntityInteractionAuthoring, DestructibleEntityAuthoring, and DestroyOffCameraAuthoring to ensure additional components are added to perform desired behaviors.

See Also

DealHitPointsOnInteraction
BoomerangMovementData

Class WrenchWeaponAuthoring

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Authoring script to add WrenchWeaponData to the entity.

```
[RequireComponent(typeof(WeaponAuthoring))]
[RequireComponent(typeof(EntityRandomAuthoring))]
public class WrenchWeaponAuthoring : MonoBehaviour
```

Remarks

Note that this script should be added to the weapon entity that will spawn in-world attacks, not the in-world attack entity itself. Although this entity will not be rendered in the game world, it is marked with the dynamic transform usage flags as it will be a child of the player entity. Requires the Weapon Authoring script to ensure all components required for executing attacks are added to the entity. Requires the EntityRandomAuthoring script for random number generation in WrenchAttackSystem.

Fields

DegreeAngleBetweenAttacks

The initial wrench attack will spawn in the direction towards a random enemy on screen. Subsequent wrenches will be offset from that initial angle by the angle defined in this field.

```
public float DegreeAngleBetweenAttacks
```

Field Value

float

Remarks

Authored in degrees for ease of use, stored in radians as that is what unity expects for the quaternion.Euler() method

Struct WrenchWeaponData

Namespace: TMG.DOTSSurvivors

Assembly: DOTSSurvivors.dll

Data component to store data relevant to spawning wrench attacks.

```
public struct WrenchWeaponData : IComponentData, IQueryTypeParameter
```

Fields

RadianAngleBetweenAttacks

An offset to apply to the angle at which a wrench attack spawns in a group. Value will be multiplied by its spawn index in the attack group.

```
public float RadianAngleBetweenAttacks
```

Field Value

float

Remarks

Authored in degrees for ease of use, stored in radians as that is what unity expects for the quaternion.Euler() method

RadianAngleToFirstEnemy

Stores the angle towards the first enemy the weapon will be firing a wrench towards. Subsequent wrenches will fire at an offset from this angle.

```
public float RadianAngleToFirstEnemy
```

Field Value

float

Remarks

As this value will change with each attack group, normally I would store something like this in a separate "state" component. However, as this component is only used in the WrenchAttackSystem I don't have any concern over potential data dependency issues so there is no problem having this field in here too.

Namespace TMG.DOTSSurvivors.EditorScripts

Classes

BurstCompilationChecker

Static class to check to see if Burst compilation is enabled in the editor.

SpawnEventPropertiesEditor

Custom editor script to display EnemySpawnEventProperties in the editor.

Class BurstCompilationChecker

Namespace: TMG.DOTSSurvivors.EditorScripts

Assembly: Assembly-CSharp-Editor.dll

Static class to check to see if Burst compilation is enabled in the editor.

```
public static class BurstCompilationChecker
```

Remarks

If Burst is disabled, a warning will be printed in the editor, giving the user more information on how to enable.

Class SpawnEventPropertiesEditor

Namespace: TMG.DOTSSurvivors.EditorScripts

Assembly: Assembly-CSharp-Editor.dll

Custom editor script to display EnemySpawnEventProperties in the editor.

```
[CustomEditor(typeof(EnemySpawnEventProperties))]
public class SpawnEventPropertiesEditor : Editor
```

Remarks

EnemySpawnEventProperties is a ScriptableObject that holds data related to enemy spawn events. This editor script makes editing these spawn events easier as it only displays fields relevant to the chosen spawn formation.

Methods

OnInspectorGUI()

Implement this function to make a custom inspector.

```
public override void OnInspectorGUI()
```